

University of Tripoli

Faculty of Engineering

Department of Computer Engineering

A graduation project submitted in partial fulfillment of the requirements
for the degree of Bachelor of Science in Computer Engineering

Reinforcement Learning for Adaptive Traffic Engineering in an SDN Network

By

Maher Abdulnasir Alqadhi

Student ID: 2210249576

Supervised by

Dr. Suad El-Geder

Spring 2026

Declaration of Academic Integrity

I hereby declare that this graduation project report entitled **"Reinforcement Learning for Adaptive Traffic Engineering in an SDN Network"** is my own original work conducted under the direct academic supervision of **Dr. Suad El-Geder** at the Department of Computer Engineering, Faculty of Engineering, University of Tripoli.

All intellectual contributions, mathematical modeling, simulation architectures, reinforcement learning algorithms, and software test suites presented herein were developed by me, except where explicit citations and bibliographic acknowledgments are made to prior published academic literature. I affirm that this work has not been previously submitted in whole or in part for any other degree, diploma, or academic qualification at this or any other tertiary educational institution.

Maher Abdulnasir Alqadhi
Tripoli, Libya
Date: June 15, 2026

. . . . . .
Candidate Signature

Certificate of Approval

This graduation project report entitled **"Reinforcement Learning for Adaptive Traffic Engineering in an SDN Network"**, authored by **Maher Abdulnasir Alqadhi**, is approved in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Engineering by the Department of Computer Engineering, Faculty of Engineering, University of Tripoli.

Supervisor:

Dr. Suad El-Geder
Associate Professor, Department of Computer Engineering
Faculty of Engineering, University of Tripoli

Signature: ..... Date: .....

Board of Examiners:

1. **Internal Examiner**: ..... Date:

2. **External Examiner**: ..... Date:

3. **Department Head**: ..... Date:

Abstract

Traditional Internet Protocol (IP) routing protocols, including Open Shortest Path First (OSPF, RFC 2328), Dijkstra's Shortest Path First (SPF), and Equal-Cost Multi-Path (ECMP), generally base forwarding decisions on configured link metrics. Under changing traffic conditions, these metrics may not reflect current link utilization or queue conditions, which can contribute to congestion, increased latency and jitter, packet loss, and underuse of alternative links.

This project develops and evaluates a software-defined traffic-engineering platform using the Ryu OpenFlow 1.3 controller, Mininet, and PyTorch. The platform combines a Dueling Double Deep Q-Network (D3QN) with Prioritized Experience Replay (PER) implemented using a binary SumTree. The agent receives normalized network-state features describing link utilization, path delay, jitter, and packet-loss risk, and selects among candidate paths generated using Yen's K-shortest-path algorithm and edge-disjoint routing methods.

The evaluation compares D3QN with OSPF, Dijkstra SPF, ECMP, Widest Shortest Path (WSP/CSPF), and Greedy Least Loaded Routing (LLR) across five emulated topologies: Hierarchical Tree, Fat-Tree Clos ($k = 4$), Abilene, NSFNet, and Spine-Leaf. In the tested dynamic-flow scenarios, peak bottleneck utilization was reduced by up to 64.4% on Fat-Tree and 77.6% on Spine-Leaf relative to Dijkstra SPF. On Spine-Leaf, measured path latency decreased from 34.71 ms to 2.67 ms; jitter reductions exceeded 95% in the reported cases, and packet loss remained below 0.015%. Neural inference throughput ranged from approximately 2,400 to 4,600 decisions per second on the test hardware. Transfer was also evaluated on randomly generated graphs of up to 35 nodes without retraining. These results are specific to the emulated topologies, workloads, and hardware used; evaluation on production networks remains future work.

Keywords: Software-Defined Networking (SDN), OpenFlow 1.3, Deep Reinforcement Learning (DRL), Dueling Double DQN, Prioritized Experience Replay, Traffic Engineering, Quality of Service (QoS), Network Telemetry.

Extended Executive Summary

This report describes the design, implementation, and evaluation of a deep reinforcement learning traffic-engineering system for adapting routing decisions to changing network conditions.

Five Thematic Areas

- **Decoupled Control and Machine-Learning Pipeline.** The event-driven Ryu OpenFlow 1.3 controller and PyTorch agent operate through an asynchronous, multi-threaded pipeline, separating packet handling from gradient updates. In the reported test environment, forward inference took less than 0.05 ms, with throughput of up to $4,625$ decisions/second on the tested CPU hardware.
- **Topological Normalization and Transfer Evaluation.** The state space uses a 10-dimensional vector of normalized bottleneck loads, hop ratios, and propagation delays, rather than absolute switch or port identifiers. Transfer was evaluated on unseen random graphs with 15, 25, and 35 nodes without retraining.
- **Congestion Barrier in the Reward Function.** The reward function includes a piecewise non-linear barrier penalty (Φ_{cong}) that increases when link utilization exceeds 70% and grows as utilization approaches 100%. This penalty encourages the agent to consider alternative paths as congestion increases.
- **Five-Topology Evaluation.** In the 150-flow evaluation, D3QN was compared with OSPF (RFC 2328), Dijkstra SPF, ECMP, WSP, and LLR across five topologies: Hierarchical Tree, Fat-Tree Clos, Abilene, NSFNet, and Spine-Leaf. In the reported Spine-Leaf scenario, peak bottleneck utilization decreased from 86.7% to 19.4%, latency decreased by up to 92.3%, and packet loss decreased to 0.010%. Reported jitter reductions ranged from 85% to 95%.
- **Implementation and Testing.** The implementation includes host-tracking ARP handling, OpenFlow Select Group WCMP, Fast Failover rules, an HTML5 telemetry dashboard, and 32 automated unit and integration tests.

Acknowledgements

I thank Allah Almighty for granting me the patience and perseverance to complete this graduation project.

I would like to thank my project supervisor, **Dr. Suad El-Geder**, Associate Professor in the Department of Computer Engineering at the Faculty of Engineering, University of Tripoli, for her guidance, feedback, and support throughout this project.

I also thank the faculty members, lecturers, and laboratory engineers of the Department of Computer Engineering at the University of Tripoli for their contributions to my undergraduate education.

I am grateful to my parents, family, and colleagues for their support during my studies.

Contents

Declaration of Academic Integrity

Certificate of Approval

Abstract i

Extended Executive Summary ii

Five Thematic Areas ii

Acknowledgements iii

List of Abbreviations xiv

Mathematical Notation & Symbol Glossary xvii

1 Introduction 1

1.1 Evolution of Modern Network Infrastructures 1

1.2 The Traffic Engineering Problem 2

1.3 Limitations of Traditional Distributed IP Routing 3

1.4 Software-Defined Networking 4

1.5 Deep Reinforcement Learning for Adaptive Traffic Engineering 5

1.6 Project Objectives and Proposal Alignment 6

1.7 Summary of Project Contributions 6

1.8 Report Organization and Structure 9

2 Theoretical Foundations and Related Work 11

2.1 Graph Theory and Network Representation 11

2.2 Traditional IP Routing Protocols and Algorithmic Mechanics 12

2.2.1 Dijkstra's Shortest Path First (SPF) Algorithm 12

2.2.2 Open Shortest Path First (OSPF, RFC 2328) 15

2.2.3 Equal-Cost Multi-Path (ECMP) 16

2.2.4 Dynamic Routing Heuristics and the Herd Behavior Phenomenon 16

2.3 Software-Defined Networking (SDN) and OpenFlow 1.3 18

2.3.1 ONF Architectural Decomposition . . . 18
2.3.2 OpenFlow 1.3 Pipeline and Primitives . . . 18
2.3.3 The Ryu SDN Controller Framework . . . 20
2.4 Deep Reinforcement Learning Theory . . . 20
2.4.1 Markov Decision Processes (MDP) . . . 20
2.4.2 Bellman Optimality and Q-Learning . . . 21
2.4.3 Deep Q-Networks (DQN) . . . 21
2.4.4 Double Deep Q-Networks (DDQN) . . . 22
2.4.5 Dueling Q-Network Architecture . . . 22
2.4.6 Prioritized Experience Replay (PER) . . . 23
2.5 Taxonomy and Comparative Literature Review . . . 24

3 Mathematical Modeling and Problem Formulation 26
3.1 Network Topology and Multigraph Mathematical Formalism . . . 26
3.2 The Multi-Commodity Flow Optimization Problem . . . 27
3.2.1 Continuous Linear Formulation . . . 27
3.2.2 Unsplittable Flow Integer Formulation and Complexity . . . 28
3.3 Markov Decision Process (MDP) Formulation . . . 29
3.3.1 State Space Representation (10-Dimensional Normalized State) 29
3.3.2 Action Space Formulation ($\mathcal{A} \in \{0, 1, 2, 3\}$) . . . 30
3.3.3 Multi-Objective Reward Function . . . 32
3.4 Derivation of the Asymptotic Congestion Barrier Penalty . . . 32
3.4.1 Mathematical Analysis and Gradient Dynamics . . . 33
3.5 Analytical Queuing, Jitter, and Packet Loss Modeling . . . 34
3.5.1 Path Latency Formulation via Kleinrock's Approximation . . . 34
3.5.2 Network Jitter Model (RFC 3393 / RFC 3550) . . . 34
3.5.3 Buffer Overflow Packet Loss Model ($M/M/1/K$) . . . 35
3.5.4 Jain's Fairness Index Formulation . . . 35
3.6 Active Closed-Loop Shadow Reservation and Micro-Burst Damping . 36
3.7 On-Demand Saturated Core Detour Synthesis . . . 37
3.8 Topological Scale Invariance via Receptive Field Mapping . . . 38

4 System Architecture and SDN Control-Plane Design 39
4.1 System Overview and Architectural Decomposition . . . 39
4.2 OpenFlow 1.3 Control Plane Engine . . . 40
4.2.1 Switch Feature Handshake and Table-Miss Setup . . . 40
4.2.2 Loop-Free Intelligent ARP Engine without Spanning Tree Protocol 41
4.2.3 Telemetry Polling Loop and Active Echo-RTT Probing . . . 42
4.3 Telemetry and Network State . . . 42

4.3.1 In-Memory NetworkX Topology Representation 42
4.3.2 Differential Counter Rate Processing 43
4.3.3 Control Plane Message Accounting 43
4.4 Adaptive Traffic Engineering 43
4.4.1 Same-Switch Optimization Bypass 44
4.4.2 Candidate Path Synthesis and Caching 44
4.4.3 Multi-Hop Atomic Flow Rule Installation 44
4.4.4 Weighted Cost Multi-Path (WCMP) via Select Group Tables . 44
4.4.5 Sub-Millisecond Fast Failover (FF) Hardware Protection . . . 45
4.4.6 Decoupled Asynchronous Training Architecture 45
4.5 Centralized Configuration 45
4.6 Real-Time Web Telemetry Dashboard and WebSocket Architecture . 46
4.7 Hierarchical TCAM-Conscious Flow Table Allocation 46

5 Deep Reinforcement Learning Agent Engineering 48
5.1 Dueling Double Deep Q-Network Architecture 48
5.1.1 Architectural Specification and Layer Dimensions 48
5.1.2 The Role of Layer Normalization in Network Control Loops . 48
5.1.3 Mathematical Proof of Dueling Identifiability 50
5.2 Binary SumTree Prioritized Experience Replay (PER) 51
5.2.1 Array-Based Binary SumTree Data Structure 51
5.2.2 Sampling and Update Algorithms in Formal Pseudocode . . . 52
5.2.3 Importance Sampling Weight Annealing 53
5.3 Polyak Soft Target Updates vs. Periodic Hard Copies 53
5.4 Huber Loss vs. Mean Squared Error 54
5.5 Complete End-to-End D3QN-PER Training Algorithm 54

6 Network Topologies and Workload Simulation 56
6.1 Evaluated Network Testbed Topologies 56
6.1.1 Hierarchical Tree Fabric (7 Switches, 8 Hosts) 56
6.1.2 Fat-Tree Clos Fabric ($k = 4$, 20 Switches, 16 Hosts) 57
6.1.3 Abilene US Backbone Network (12 Switches, 8 Hosts) 57
6.1.4 NSFNet Continental Mesh (14 Switches, 8 Hosts) 57
6.1.5 Spine-Leaf Data Center Fabric (12 Switches, 16 Hosts) 58
6.2 Topological Graph Metrics Comparative Analysis 58
6.3 Synthetic Traffic Generation Engine 58
6.3.1 Poisson Process for Background Mice Flows 59
6.3.2 Heavy-Tailed Pareto Distribution for Elephant Microbursts . . 59
6.3.3 Spatial Hotspot Clustered Surges 60

6.3.4 Targeted Core Jamming Stress Model 60
6.4 The 3-Phase Curriculum Training Strategy 60
6.4.1 Phase 1: Pure Exploration Baseline (Episodes 1–250) 60
6.4.2 Phase 2: Stress and Bottleneck Rerouting (Episodes 251–750) 61
6.4.3 Phase 3: Fine-Tuning (Episodes 751–1000) 61

7 Comparative Evaluation Results 62
7.1 Comparative Benchmark Methodology 62
7.1.1 Evaluated Routing Paradigms 62
7.1.2 Traffic Injection and Closed-Loop Accounting 62
7.2 Benchmark Results across Topologies 63
7.3 In-Depth Performance Analysis by Topology 63
7.3.1 Hierarchical Tree Fabric 63
7.3.2 Fat-Tree Clos Fabric ($k = 4$) 63
7.3.3 Abilene US Backbone Network 65
7.3.4 NSFNet Continental Mesh 65
7.3.5 Spine-Leaf Data Center Fabric 65
7.4 Analysis of Publication Figures and Empirical Envelopes 66
7.4.1 6-Panel All-Metrics Comparison Figure 66
7.4.2 Multi-Dimensional QoS Quality Radar 67
7.4.3 Cumulative Distribution Functions (CDFs) of Latency and Jitter 68
7.4.4 Training Loss and Reward Convergence Progression 68

8 Stress Testing, Resilience, and Zero-Shot Generalization 70
8.1 Methodology for Extreme Stress Evaluation 70
8.2 Scenario 1: Severe Core Jamming (85%–98% Saturation) 70
8.3 Scenario 2: High-Concurrency Flow Requests 71
8.4 Scenario 3: Asymmetric Regional Hotspot Surges 71
8.5 Scenario 4: WAN Detour Latency Degradation Analysis 72
8.6 Scenario 5: Jitter Suppression and Packet Loss Eradication 72
8.7 Zero-Shot Generalization on Arbitrary Unseen Random Graphs . . . 73
8.8 Discussion of Stress Testing Publication Figures 73
8.8.1 Multi-Topology Stress Benchmark Figure 73
8.8.2 Stress Testing Radar Chart 74
8.8.3 Load Balancing Stress Test Figure 74

9 Discussion and Scalability Limitations 76
9.1 Critical Theoretical Analysis and Engineering Trade-Offs 76
9.1.1 Synchronized Route Selection under Dynamic Heuristics . . . 76
9.1.2 The WAN Detour Latency Penalty and Propagation Dilemma 77

9.1.3 Fairness on Asymmetric Meshes . . . 78
9.1.4 Candidate Path Synthesis Scalability . . . 78
9.1.5 OpenFlow TCAM Space Limitations and Rule Explosion . . . 78
9.2 Formal Pareto Sensitivity Ablation and Hyperparameter Sensitivity Analysis . . . 79
9.3 Summary of Limitations and Mitigations . . . 80

10 Software Quality Assurance and Platform Operations 82
10.1 Software Quality Assurance and Testing Philosophy . . . 82
10.2 Experimental Virtualization Environment and Host Provisioning . . . 83
10.3 Automated Test Suite Verification . . . 84
10.4 Network Settings Validation . . . 84
10.5 Headless Dashboard Validation Suite . . . 84
10.6 Interactive Web Dashboard and Telemetry . . . 86
10.6.1 Interactive OpenFlow 1.3 Topology and Telemetry Fabric . . . 86
10.6.2 Head-to-Head Algorithm Tournament Engine . . . 87
10.6.3 PyTorch Neural Network Architecture and Model Inspector . . . 88
10.6.4 Network Fabric Configuration and Chaos Engineering Console 89
10.7 Operational Runbook . . . 90

11 Conclusion and Future Work 91
11.1 Summary of Results . . . 91
11.2 Verification of Approved Proposal Specifications . . . 92
11.3 Real-World Hardware Deployment Considerations . . . 92
11.4 Future Research Directions . . . 93
11.4.1 Multi-Agent Reinforcement Learning . . . 93
11.4.2 Continuous Action Flow Splitting with SAC . . . 93
11.4.3 P4 Programmable Data Planes and In-Band Network Telemetry (INT) . . . 93

A Mathematical Proofs and Derivations 97
A.1 Proof of Bellman Optimality via Banach Fixed Point Theorem . . . 97
A.2 Derivation of Double Q-Learning Bias Elimination . . . 98
A.2.1 Standard Q-Learning Bias . . . 98
A.2.2 Double Q-Learning Decoupling . . . 99
A.3 Derivation of Importance Sampling Correction in PER . . . 99
A.4 Derivation of the M/M/1 Queuing Delay Model . . . 100
A.5 Proof of Congestion Avoidance via Asymptotic Barrier Function . . . 101

B OpenFlow 1.3 Protocol Technical Reference **103**
B.1 OpenFlow 1.3 Common Header Specification . . . 103
B.2 Core OpenFlow 1.3 Message Type Enumeration . . . 103
B.3 Annotated OpenFlow Hex Packet Dumps . . . 105
B.3.1 Annotated `OFPPacketIn` Wire Frame . . . 105
B.3.2 Annotated `OFPFlowMod` Installation Wire Frame . . . 105

C System Configuration Reference **106**
C.1 Centralized Configuration Architecture . . . 106
C.2 Complete Parameter Dictionary . . . 106

D Selected Implementation and Algorithm Listings **109**
D.1 Dueling Q-Network and Agent Engine . . . 109
D.2 Binary SumTree Prioritized Replay Buffer . . . 110
D.3 State Feature Vector Extraction . . . 111
D.4 Controller and Telemetry Implementation Excerpts . . . 112
D.4.1 Table-Miss Rule Installation . . . 112
D.4.2 NetworkX Edge Telemetry Record . . . 112
D.4.3 Multi-Hop Flow Installation . . . 113

E User and Administrator Manual **114**
E.1 System Prerequisites and Dependencies . . . 114
E.1.1 Hardware Requirements and Reference Deployment Environment 114
E.1.2 Software Requirements . . . 114
E.2 Installation and Environment Initialization . . . 115
E.2.1 Step-by-Step Installation . . . 115
E.3 Automated Execution Runbook . . . 115
E.3.1 1. Execute Unit and Integration Tests (32 Tests) . . . 115
E.3.2 2. Execute the Head-to-Head Tournament Benchmark . . . 116
E.3.3 3. Execute Multi-Topology Stress Testing . . . 116
E.3.4 4. Evaluate Zero-Shot Random Graph Generalization . . . 116
E.3.5 5. Launch Live Production System (Controller + Web Dashboard) 116
E.3.6 6. Standalone Web Dashboard Launcher . . . 116
E.4 Interactive Web Dashboard User Guide . . . 116
E.5 REST API Specification and Endpoint Reference . . . 118
E.6 Mininet CLI Operations and OVS Diagnostics . . . 118
E.7 Pareto Sensitivity Ablation and Dynamic Telemetry Tuning . . . 121
E.7.1 Configuring Shadow Reservations and TCAM Allocation . . . 121

List of Figures

2.1 Detailed Three-Tier Software-Defined Networking Architectural Model. 18

4.1 End-to-End Architecture and Data Flow of the Adaptive SDN Traffic-Engineering Platform. 40

5.1 Detailed Computational Architecture of the Dueling Q-Network with Layer Normalization. 49
5.2 Binary SumTree Data Structure with Total Prefix Sum at Root. . . . 52

6.1 3-Phase Curriculum Training Progression over 1,000 Episodes. 61

7.1 Comparison of bottleneck utilization, path latency, RFC 3393 jitter, packet loss, Jain's fairness index, and offload decisions across five topologies. 66
7.2 Multi-dimensional Quality of Service (QoS) radar chart comparing D3QN against classical and heuristic baselines across 7 normalized operational dimensions. 67
7.3 Cumulative Distribution Functions (CDFs) comparing the statistical distributions of end-to-end path latency and jitter between D3QN, Dijkstra, and ECMP. 68
7.4 Multi-panel training convergence progression across 1,000 episodes displaying episodic return, Smooth L1 Huber loss, peak link utilization, latency, and exploration decay. 69

8.1 Multi-topology stress testing benchmark illustrating peak bottleneck relief and latency preservation under core jamming and link failure conditions. 74
8.2 Radar chart depicting resilience metrics across topologies under extreme stress testing scenarios. 75
8.3 Real-time link utilization equalization dynamics during asymmetric traffic bursts, comparing D3QN against classical baselines. 75

9.1 Multi-Objective Pareto Sensitivity Ablation across 6 Reward Configurations. Left: Peak Bottleneck Utilization vs. End-to-End Latency trade-off showing the critical 70% barrier. Right: Reliability metrics (Packet Loss Rate and Jitter) across configurations. 80

10.1 VMware Virtual Machine Hardware Configuration for the Experimental SDN-DRL Platform (Ubuntu 64-bit, 4 vCPUs, 8 GB RAM, 60 GB SCSI Storage). 83

10.2 Real-Time OpenFlow 1.3 Topology Fabric and Interactive Telemetry Console. Displays live switch node coordinates (s_1 through s_7), dynamic utilization percentage badges on links, real-time KPI status indicators, and the interactive D3QN Route Predictor comparing autonomous routing against OSPF (RFC 2328). 86

10.3 Head-to-Head Algorithm Tournament Arena. The tournament engine executes real-time comparative simulations across user-selected routing protocols, rendering instantaneous Pareto efficiency verdicts, metric delta bar charts, delay trajectories across burst steps, and 6D quality profile radar plots. 87

10.4 PyTorch D3QN Neural Network Architecture and Live Model Telemetry Inspector. Displays total parameter count (26,501 weights), inference execution device (CPU/CUDA), exploration coefficient ($\varepsilon = 0.010$), dueling pipeline tensor dimensions, normalized state input vector, and estimated candidate path Q-values. 88

10.5 Network Fabric Architecture Settings and Chaos Fault Injection Console. Provides interactive sliders for switch queue depths ($K \in [64, 1024]$ pkts), link capacities, propagation delays, delay variation (jitter), and base physical packet drop rates. 89

List of Tables

1.1 EC499 Proposal Alignment Matrix 7
1.2 EC499 Proposal Compliance Matrix (continued) 7

2.1 Standard OSPF Interface Administrative Costs 15
2.2 Routing Literature Taxonomy 24
2.3 Routing Literature Taxonomy (continued) 24

3.1 Detailed Mathematical Specification of the 10-Dimensional State Space Vector 31
3.2 Numerical Escalation of the Asymptotic Barrier Penalty vs. Linear Hop Penalty 33

5.1 Layer Configuration and Parameter Counts of the Dueling Q-Network 49

6.1 Graph-Theoretic Metrics of the Evaluated Network Topologies 58

7.1 Benchmark Results Across Five Topologies 64

8.1 Scenario 1: Severe Core Jamming Stress Test Results 70
8.2 Scenario 3: Asymmetric Hotspot Surge Diversion Performance 72
8.3 Scenario 5: Jitter Suppression and Packet Loss Eradication Under Stress . . . 72
8.4 Zero-Shot Generalization Performance on Arbitrary Unseen Random Graph Topologies 73

9.1 Quantitative Performance Across 6 Reward Hyperparameter Configurations (50 Episodes) 79
9.2 Systematic Architectural Flaw Remediation Matrix 81

10.1 Detailed Breakdown of the 32 Automated Unit and Integration Tests 85

B.1 OpenFlow 1.3 Common Header Binary Structure 103
B.2 Primary OpenFlow 1.3 Message Types Utilized in the Platform 104

C.1 System Configuration and Hyperparameters 107
C.2 System Configuration and Hyperparameters (continued) 108

E.1 Complete REST API Endpoint Reference 119
E.2 Complete REST API Endpoint Reference (continued) 120

List of Algorithms

1 Dijkstra's Shortest Path First (SPF) Algorithm 13
2 Iterative Non-Recursive Binary SumTree Retrieval 52
3 Iterative Non-Recursive Binary SumTree Priority Update 53
4 Complete D3QN-PER Adaptive Traffic Engineering Training Pipeline 55

List of Abbreviations

API Application Programming Interface

ARP Address Resolution Protocol

ASIC Application-Specific Integrated Circuit

BGP Border Gateway Protocol

CSPF Constrained Shortest Path First

D3QN Dueling Double Deep Q-Network

DDoS Distributed Denial of Service

DPID Datapath Identifier

DRL Deep Reinforcement Learning

ECMP Equal-Cost Multi-Path

EMA Exponential Moving Average

FF Fast Failover (OpenFlow Group Type)

GNN Graph Neural Network

IETF Internet Engineering Task Force

INT In-Band Network Telemetry

IP Internet Protocol

IS Importance Sampling

IS-IS Intermediate System to Intermediate System

LLDP Link Layer Discovery Protocol

LLR Least Loaded Routing

MAC Media Access Control

MADRL Multi-Agent Deep Reinforcement Learning

MCF Multi-Commodity Flow

MDP Markov Decision Process

MLP Multi-Layer Perceptron

MPLS Multi-Protocol Label Switching

MTU Maximum Transmission Unit

ONF Open Networking Foundation

OSPF Open Shortest Path First

OVS Open vSwitch

PER Prioritized Experience Replay

QoS Quality of Service

ReLU Rectified Linear Unit

REST Representational State Transfer

RFC Request for Comments

RL Reinforcement Learning

RPC Remote Procedure Call

RTT Round-Trip Time

SAC Soft Actor-Critic

SDN Software-Defined Networking

SPF Shortest Path First

SRv6 Segment Routing over IPv6

STP Spanning Tree Protocol

TCAM Ternary Content-Addressable Memory

TCP Transmission Control Protocol

TD Temporal Difference

TE Traffic Engineering

UDP User Datagram Protocol

UMCF Unsplittable Multi-Commodity Flow

VM Virtual Machine

WAN Wide Area Network

WCMP Weighted Cost Multi-Path

WSP Widest Shortest Path

Mathematical Notation and Symbol Glossary

$G = (V, E)$	Directed multigraph representing the network topology
V	Set of OpenFlow switching elements ($\|V\|$ switches)
E	Set of directed transmission links ($\|E\|$ directed edges)
$e = (u, v)$	Directed communication link from switch u to switch v
C_e	Maximum transmission bandwidth capacity of link e in Mbps
$d_{\mathbf{prop}}(e)$	Physical medium propagation delay of link e in milliseconds
$d_{\mathbf{base}}(e)$	Baseline non-congested transit delay (propagation + serialization) in ms
$u_e(t)$	Real-time dimensionless link utilization ratio ($u_e \in [0.0, 1.0]$)
$\lambda_e(t)$	Instantaneous aggregate traffic arrival rate on link e in Mbps
$q_e(t)$	Instantaneous queue buffer occupancy in packets or bytes
K_e	Maximum queue buffer capacity of link e in packets
$\mathcal{K}$	Set of active network traffic commodities (flows)
D_k	Bandwidth demand of commodity k in Mbps
s_k, t_k	Source ingress switch and destination egress switch of commodity k
$\mathcal{P}_a$	Chosen candidate path sequence of vertices and edges for action a
$\|\mathcal{P}_a\|$	Scalar hop count length of path $\mathcal{P}_a$

$D(\mathcal{P})$	End-to-end cumulative path latency (propagation + queuing) in ms
$J(\mathcal{P})$	Network packet delay variation (jitter) along path $\mathcal{P}$ in ms (RFC 3393)
$P_{\mathbf{loss}}(\mathcal{P})$	Buffer overflow packet drop probability along path $\mathcal{P}$
$\mathcal{J}(u)$	Jain's Fairness Index across link utilization vector u
$\mathcal{S}$	Continuous environmental state space ($\mathcal{S} \in [0.0, 1.0]^{10}$)
$\mathcal{A}$	Discrete candidate routing action space ($\mathcal{A} = \{0, 1, 2, 3\}$)
$\mathcal{R}(s, a)$	Scalar multi-objective reinforcement learning reward function
$\Phi_{\mathbf{cong}}(u)$	Piecewise non-linear asymptotic congestion barrier penalty
γ	Temporal reward discount factor ($\gamma = 0.95$)
θ, θ^-	Online network weights and target network weights in D3QN
τ	Polyak soft target tracking update rate ($\tau = 0.005$)
$V(s)$	Scalar state-value stream function in Dueling architecture
$A(s, a)$	Vector action-advantage stream function in Dueling architecture
δ_i	Temporal Difference (TD) error of transition tuple i
p_i	Transition sampling priority in binary SumTree PER ($p_i = (\lvert\delta_i\rvert + \epsilon)^\alpha$)
w_i	Importance Sampling loss correction weight in PER
ϵ	Exploration probability in ϵ-greedy action selection policy

Chapter 1

Introduction

1.1 Evolution of Modern Network Infrastructures

Communication networks have evolved from modest, host-centric, packet-switched topologies into the foundational backbone of contemporary global commerce, scientific computing, cloud infrastructure, and civil communication. Modern hyper-scale data centers, telecommunication service provider backbones, multi-tenant cloud fabrics, and content distribution networks (CDNs) routinely interconnect hundreds of thousands of bare-metal servers and millions of concurrent virtual machines (VMs) and containerized microservices. These infrastructures handle data transfers exceeding exabytes per month, supporting bandwidth-intensive and latency-hyper-sensitive applications such as real-time financial algorithmic trading, remote robotic tele-surgery, autonomous vehicular telemetry, ultra-high-definition interactive telepresence, distributed machine learning gradient synchronization, and exascale cloud storage replication.

As workloads have shifted toward distributed, microservice-oriented cloud applications, network traffic patterns have also changed. In campus and enterprise networks, traffic has traditionally followed a vertical "North-South" path between external clients and internal services. In multi-tenant data centers and distributed analytics clusters (e.g., Apache Spark, Kubernetes, Ceph, and distributed PyTorch/TensorFlow training fabrics), a substantial share of traffic may travel horizontally along an "East-West" path between server racks and compute nodes.

East-West traffic can exhibit bursty arrivals and heavy-tailed flow-size distributions. High-bandwidth, long-lived transfers—commonly referred to as "elephant flows" (e.g., dataset transfers, VM migrations, and database backups)—share switching infrastructure with short-lived transactional requests, known as "mice flows" (e.g., web queries, DNS lookups, and distributed lock coordination). Under these conditions, managing link utilization, switch-buffer occupancy, and packet queuing delay is an important network-engineering problem.

1.2 The Traffic Engineering Problem

Traffic Engineering (TE) is the operational domain within computer network engineering concerned with optimizing the performance of operational networks. The core objective of TE is to dynamically guide, steer, and map network traffic flows across physical or virtual network topologies in a manner that maximizes aggregate network throughput, balances utilization across all available transmission links, avoids localized core bottlenecks, minimizes end-to-end packet transmission latency and packet delay variation (jitter), and eliminates buffer overflow packet loss.

Mathematically, global traffic engineering across a multigraph $G = (V, E)$ with $|V|$ switching nodes and $|E|$ directed transmission edges can be formalized as an instance of the classical Multi-Commodity Flow (MCF) problem. Let $\mathcal{K}$ denote the set of active traffic demands (commodities), where each commodity $k \in \mathcal{K}$ is defined by a source node $s_k \in V$, a destination node $t_k \in V$, and an instantaneous bandwidth demand $d_k > 0$. If f_e^k represents the amount of commodity k assigned to directed link $e \in E$, the optimal routing allocation seeks to optimize a multi-objective utility function:

$$\min_{f} \quad \sum_{e \in E} \Phi_e \left(\frac{\sum_{k \in \mathcal{K}} f_e^k}{C_e} \right) \tag{1.1}$$

subject to flow conservation constraints at every node $v \in V$:

$$\sum_{e \in \delta^+(v)} f_e^k - \sum_{e \in \delta^-(v)} f_e^k = \begin{cases} d_k & \text{if } v = s_k \\ -d_k & \text{if } v = t_k \\ 0 & \text{otherwise} \end{cases} \qquad \forall v \in V, \forall k \in \mathcal{K} \tag{1.2}$$

and physical link bandwidth capacity constraints:

$$\sum_{k \in \mathcal{K}} f_e^k \leq C_e \quad \forall e \in E \tag{1.3}$$

where $\delta^+(v)$ and $\delta^-(v)$ represent the sets of edges leaving and entering node v, respectively, C_e denotes the maximum physical line rate of link e, and $\Phi_e(\cdot)$ is an increasing, convex penalty function modeling link congestion and queuing delay.

While the basic continuous multi-commodity flow formulation with fractional flow splitting can be solved in polynomial time via interior-point linear programming methods, real-world packet-switched routing introduces unsplittable (integer) flow constraints to prevent TCP packet reordering, binary forwarding rules, and discrete switch buffer capacities. Under unsplittable flow requirements, the traffic engineering optimization problem becomes strictly $\mathcal{NP}$-hard. In a production network subject to continuous, sub-second traffic variations, computing global linear programming

re-allocations in real time is computationally intractable. Consequently, production networks have historically relied on distributed, heuristic routing protocols.

1.3 Limitations of Traditional Distributed IP Routing

Traditional distributed Internet Protocol (IP) networks rely on Interior Gateway Protocols (IGPs) such as Open Shortest Path First (OSPF, RFC 2328) and Intermediate System to Intermediate System (IS-IS) to determine forwarding paths. In these protocols, autonomous routers exchange Link-State Advertisements (LSAs) and execute Dijkstra's Shortest Path First (SPF) algorithm to construct local next-hop routing tables. Each physical link e is assigned a static administrative weight $w_e \in \mathbb{Z}^+$, configured by network administrators inversely proportional to link capacity:

$$w_e = \max\left(1, \text{round}\left(\frac{\text{Reference Bandwidth}}{C_e}\right)\right) \tag{1.4}$$

Distributed shortest-path routing can provide resilience to individual control-plane failures, but static administrative metrics do not directly adapt to changing traffic conditions. The following limitations motivate the traffic-engineering approach evaluated in this project:

1. **Static Link Metrics**: Administrative weights do not necessarily reflect real-time network dynamics, buffer occupancy, or link utilization. A configured cost may remain unchanged as traffic increases, potentially resulting in queue buildup or packet loss.

2. **Core Bottlenecks in Hierarchical Fabrics**: In some hierarchical topologies (e.g., tree, Clos, Fat-Tree, and Spine-Leaf architectures), shortest paths for inter-pod communications may traverse central aggregation or core switches. Under heavy load, traffic can concentrate on a subset of transit links while alternative links remain underused.

3. **Hash-Based Assignment in Equal-Cost Multi-Path (ECMP)**: Equal-Cost Multi-Path (ECMP) can distribute traffic across multiple paths with identical minimum metrics. Switches commonly hash a packet's 5-tuple (Source IP, Destination IP, Protocol, Source Port, Destination Port) to assign traffic to an equal-cost egress port:

$$h = \text{CRC32}(\text{5-Tuple}) \pmod{|\mathcal{P}_{\text{ECMP}}|} \tag{1.5}$$

Despite its widespread deployment, ECMP suffers from severe operational flaws:

- *Restriction to Equal-Cost Paths*: ECMP distributes traffic only across paths that share the same metric. It does not select a higher-cost alternative based on current utilization alone.
- *Elephant Flow Collisions*: ECMP performs oblivious flow-level hashing without awareness of flow sizes or bitrates. When multiple high-bandwidth elephant flows hash to the same egress port (hash collision), the shared link buffer is rapidly exhausted, inducing severe head-of-line blocking and packet loss for all co-located latency-sensitive mice flows.
- *Hash Polarization*: In multi-stage Clos architectures, identical hashing algorithms implemented across consecutive switch tiers often map the same subset of flows onto identical downstream paths, concentrating traffic onto a fraction of available fabric links.

4. **Synchronized Decisions in Dynamic Heuristics**: Dynamic greedy heuristics, such as Widest Shortest Path (WSP) and Least Loaded Routing (LLR), may make similar decisions for flows that arrive between telemetry updates. If multiple flows select the same currently least-loaded link, its utilization can increase before the next update, potentially causing subsequent decisions to favor another link. This behavior can contribute to oscillation and route changes.

1.4 Software-Defined Networking

Software-Defined Networking (SDN) separates network control logic from packet forwarding. In traditional network appliances, the control plane (the logic that determines where packets should be forwarded) and the data plane (which processes and forwards packets, often using Application-Specific Integrated Circuits / ASICs) are commonly implemented within the same device.

SDN separates the control plane from the underlying data plane:

- **The Data Forwarding Plane**: Network switches (whether bare-metal hardware switches with Ternary Content-Addressable Memory / TCAM or virtual software switches like Open vSwitch / OVS) are stripped of complex distributed routing protocol daemons. Instead, switches act as fast, programmable packet-forwarding engines that match incoming packet headers against flow tables and execute programmed actions (e.g., output to port, push/pop VLAN/MPLS headers, drop, or encapsulate).
- **The Southbound Interface (OpenFlow 1.3)**: The OpenFlow protocol, standardized by the Open Networking Foundation (ONF), provides an open,

vendor-agnostic programmatic communication protocol between the controller and the switches over a secure TCP/TLS control channel.

- **The Centralized Control Plane**: A logically centralized software entity—the SDN Controller (e.g., the Ryu SDN Framework)—maintains a continuous, global view of the entire network topology, host attachments, and link status. Through standard OpenFlow messages, the controller dynamically discovers topology links via the Link Layer Discovery Protocol (LLDP), polls fine-grained port and flow statistics, and proactively or reactively writes forwarding rules directly into switch flow tables.

Centralized network visibility does not by itself solve traffic-engineering problems. A controller also requires an algorithm that uses telemetry to select routes in response to network conditions.

1.5 Deep Reinforcement Learning for Adaptive Traffic Engineering

Traffic engineering under bursty, changing workloads requires methods that can respond to observed network conditions. Deep Reinforcement Learning (DRL) is one approach to formulating this closed-loop decision problem.

Reinforcement Learning (RL) formalizes autonomous decision-making as a Markov Decision Process (MDP). An autonomous agent interacts continuously with an environment through discrete time steps: the agent observes the current network state $s_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}$ according to its internal policy $\pi(a_t|s_t)$, and transitions into a new state s_{t+1} while receiving a scalar reward signal $r_t = \mathcal{R}(s_t, a_t)$. The objective of the RL agent is to learn an optimal policy π^* that maximizes the expected cumulative discounted future reward over time:

$$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} \tag{1.6}$$

where $\gamma \in [0, 1)$ represents the temporal discount factor.

In recent years, the combination of reinforcement learning with deep neural networks (Deep Q-Networks / DQN) has achieved high performance in several complex control domains. However, directly applying standard DQN algorithms to computer network traffic engineering presents unique, domain-specific theoretical and practical challenges:

- **Topological Generalizability**: Standard neural networks take fixed-size input vectors. If an RL agent is formulated to map raw link numbers or node IDs directly, the neural network becomes tied to a specific switch and link count,

rendering the model useless if a single link fails or if the policy is transferred to a different network topology.

- **Action Space Scalability and Loop Freedom**: If the agent outputs unconstrained next-hop forwarding ports at each individual switch, the action space expands exponentially with network diameter, creating a high probability of transient forwarding loops, packet bouncing, and network black holes during exploration.

- **Multi-Objective Metric Trade-offs**: A realistic traffic engineering agent must balance multiple conflicting performance metrics simultaneously: maximizing throughput, avoiding bottleneck links, bounding propagation latency, suppressing jitter variation, minimizing packet dropouts, and preventing excessive hop detours.

- **Non-Stationary Experience Distribution**: Traffic microbursts and severe core jamming events are rare compared to steady-state background traffic. Under uniform experience replay sampling, the agent rarely samples critical congestion events, leading to severe underfitting on emergency anti-congestion policies.

This project addresses these challenges through the design and evaluation of a deep reinforcement learning traffic-engineering platform.

1.6 Project Objectives and Proposal Alignment

This project follows the objectives and procedures specified in the faculty-approved **EC499 Graduation Project Proposal** submitted to the Department of Computer Engineering, Faculty of Engineering, University of Tripoli.

The project objective is to implement and evaluate an SDN traffic-engineering platform that uses a closed-loop DRL agent to adapt routing decisions to observed network conditions. Table 1.1 summarizes the relationship between the project work and the approved objectives and procedures.

1.7 Summary of Project Contributions

The main technical contributions of this project are:

1. **Topologically Normalized State Space Formulation ($\mathcal{S} \in [0,1]^{10}$)**: We introduce a compact, 10-dimensional network state vector that maps normalized bottleneck link utilizations, hop ratios, and cumulative path latencies across diversity-aware candidate paths into a bounded domain $[0.0, 1.0]$. The formulation

Table 1.1: Summary of Project Alignment with the Approved EC499 Graduation Project Proposal

Proposal Item	Approved Specification	Concrete Implementation & Architectural Realization	Status
Objective 1	Design a Deep Q-Network agent for dynamic routing	Implemented Dueling Double DQN and SumTree-based Prioritized Replay (`agent/dqn_router.py`, `agent/prioritized_replay.py`).	Complete
Objective 2	Integrate the agent with Ryu and Mininet	Built an OpenFlow 1.3 Ryu application and Mininet topology (`controller/main_controller.py`, `topology/mininet_topo.py`).	Complete
Objective 3	Reduce latency and jitter against standard routing	Added Echo-RTT telemetry and RFC delay-variation tracking (`controller/state_manager.py`).	Complete
Objective 4	Maximize throughput and balance link utilization	Added a multi-objective reward with an asymptotic congestion barrier (Φ_{cong}).	Complete
Objective 5	Evaluate packet loss and control overhead	Measured finite-buffer loss, OpenFlow message counts, and inference throughput.	Complete

Table 1.2: EC499 Proposal Compliance Matrix (Procedural Milestones)

Proposal Item	Approved Specification	Concrete Implementation & Architectural Realization	Status
Procedure 1	Review SDN and reinforcement learning literature	Reviewed OpenFlow 1.3, Bellman optimality, Double Q-learning, Dueling networks, and traffic engineering (Chapter 2).	Complete
Procedure 2	Set up Ubuntu, Mininet, and Ryu	Configured Ubuntu with Ryu 4.34, OpenFlow 1.3, Mininet 2.3, and PyTorch.	Complete
Procedure 3	Formulate the state and reward	Defined a normalized 10-dimensional state and multi-objective reward (Chapter 3).	Complete
Procedure 4	Train the agent on synthetic traffic	Built a three-phase curriculum for background, burst, and core-jamming traffic (`benchmark_evaluation.py`).	Complete
Procedure 5	Benchmark against established routing baselines	Compared D3QN with OSPF, Dijkstra, ECMP, WSP, and LLR on five topologies (`benchmark_routing_algorithms.py`).	Complete
Procedure 6	Document results and verify the software	Completed this report, generated 14 figures, and verified 32 automated tests (`test_suite.py`).	Complete

avoids encoding absolute switch or link identifiers. Transfer to unseen topologies is evaluated separately without retraining the network weights.

2. **Non-Linear Asymptotic Congestion Barrier Penalty (Φ_{cong})**: We derive an analytical, piecewise barrier penalty function inspired by interior-point optimization theory. When peak path utilization remains below 70%, the penalty scales linearly, encouraging the agent to select the shortest physical path to conserve network bandwidth. When utilization exceeds 70%, the penalty escalates asymptotically toward infinity as utilization approaches 100%. This non-linear penalty encourages the policy to consider alternative paths as utilization approaches saturation, with the aim of reducing queue overflow and packet loss.

3. **High-Performance Dueling Double DQN Engine with Binary SumTree PER**: We construct a PyTorch Dueling Double Deep Q-Network stabilized by Layer Normalization (`nn.LayerNorm`) and Polyak soft target updates ($\tau = 0.005$). The training pipeline is accelerated by an iterative, non-recursive $O(\log N)$ binary SumTree data structure supporting Prioritized Experience Replay (PER), sampling rare congestion transitions proportional to their Temporal Difference (TD) error with importance-sampling loss correction.

4. **Ryu-Based OpenFlow 1.3 Control Plane**: We develop a fully asynchronous, event-driven SDN application on the Ryu framework. The engine features:

   - A host-tracking ARP resolution mechanism designed to limit inter-switch broadcast traffic without requiring the Spanning Tree Protocol (STP).
   - Active Echo-RTT telemetry probing to capture real-time control-plane-to-switch latency.
   - Decoupled asynchronous training threads that prevent neural backpropagation from blocking incoming packet-in message handling.
   - OpenFlow Select Groups (`OFPGT_SELECT`) provide Softmax-weighted multi-path (WCMP), with hardware Fast Failover (`OFPGT_FF`) for critical links.

5. **Comparative Evaluation across Five Topologies**: The evaluation compares the D3QN agent with five established routing baselines: OSPF (RFC 2328), Dijkstra SPF, ECMP, Widest Shortest Path (WSP/CSPF), and Greedy Least Loaded Routing (LLR). Seven metrics are evaluated across five topologies: Hierarchical Tree, Fat-Tree ($k = 4$) Clos, Abilene US Backbone, NSFNet Continental Mesh, and Spine-Leaf data center fabrics.

6. **Transfer Evaluation and Stress Testing**: The trained model is evaluated under five stress scenarios, including 98% core-link utilization, 500-flow concurrency, and asymmetric pod surges. Transfer is also evaluated on Erdős–Rényi random graphs of up to 35 nodes without fine-tuning.

7. **Analysis of System Limitations**: The report discusses operational limitations, including synchronized route selection by greedy heuristics, propagation delay on WAN detours, fairness across asymmetric networks, and the computational complexity of K-shortest-path synthesis.

1.8 Report Organization and Structure

The report is organized into eleven chapters and five appendices:

- **Chapter 2 (Theoretical Foundations and Related Work)**: Explores the mathematical foundations of shortest-path routing, OSPF, ECMP, MPLS-TE, OpenFlow 1.3 protocol specifications, MDP theory, Q-learning, Double DQN, Dueling Q-networks, and Prioritized Experience Replay, concluding with a detailed taxonomy table.

- **Chapter 3 (Mathematical Modeling and Problem Formulation)**: Formalizes the multi-commodity flow optimization problem, details the 10-dimensional state space, action space diversity filtering, the mathematical derivation of the asymptotic barrier reward function, and analytical M/M/1/K queuing models.

- **Chapter 4 (System Architecture and SDN Control-Plane Design)**: Presents the end-to-end software architecture of the platform, detailing the `MainController`, the loop-free ARP engine, `StateManager` telemetry polling, `RoutingModule` FlowMod installation, OpenFlow Group Tables, and centralized configuration.

- **Chapter 5 (Deep Reinforcement Learning Agent Engineering)**: Dissects the internal neural network implementation, detailing layer normalization, dueling stream aggregation, the binary SumTree PER data structure, Polyak target updates, and the full training algorithm.

- **Chapter 6 (Network Topologies and Workload Simulation)**: Details the structural properties of all five evaluated benchmark topologies, the synthetic traffic workload generation engine (Poisson mice flows, Pareto elephant bursts, core jamming), and the 3-Phase Curriculum training loop.

- **Chapter 7 (Comparative Evaluation Results)**: Presents the results of the comparative benchmark across all five topologies and seven metrics, supported by high-resolution publication plots.
- **Chapter 8 (Stress Testing, Resilience, and Zero-Shot Generalization)**: Evaluates the system under extreme stress scenarios, dynamic link failures, and zero-shot transferability on arbitrary random graphs up to 35 nodes.
- **Chapter 9 (Discussion and Scalability Limitations)**: Examines system trade-offs, fairness, scalability, and limitations.
- **Chapter 10 (Software Quality Assurance and Platform Operations)**: Documents the automated test suite (32 passing tests), test methodology, and platform execution runbooks.
- **Chapter 11 (Conclusion, Limitations, and Future Work)**: Summarizes the project's achievements, verifies proposal compliance, and outlines future horizons in Multi-Agent RL, Graph Neural Networks (GNNs), and P4/INT programmable data planes.
- **Appendices A–E**: Provide mathematical proofs (Appendix A), OpenFlow 1.3 protocol specifications (Appendix B), a centralized configuration dictionary (Appendix C), selected implementation and algorithm listings (Appendix D), and the platform deployment manual (Appendix E).

Chapter 2

Theoretical Foundations and Related Work

2.1 Graph Theory and Network Representation

The physical and logical structure of a communication network is mathematically formalized as a directed multigraph $G = (V, E)$, where:

- $V = \{v_1, v_2, \ldots, v_{|V|}\}$ represents the finite set of switching elements (OpenFlow switches, routers, or packet forwarders).
- $E = \{e_1, e_2, \ldots, e_{|E|}\} \subseteq V \times V$ represents the set of directed communication links interconnecting the switching nodes. A directed edge from switch u to switch v is denoted by $e = (u, v)$.

Each directed link $e = (u, v) \in E$ is characterized by a static and dynamic attribute tuple:

$$e \sim \langle C(e),\, d_{\text{prop}}(e),\, \text{Loss}_{\text{base}}(e),\, u(e,t),\, q(e,t) \rangle \tag{2.1}$$

where:

- $C(e) \in \mathbb{R}^+$ denotes the maximum transmission bandwidth capacity of the link in Megabits per second (Mbps) or Gigabits per second (Gbps).
- $d_{\text{prop}}(e) \in \mathbb{R}^+$ denotes the physical propagation delay of the transmission medium, determined by the physical length L_e of the fiber or copper link and the propagation speed of light in the medium $v_{\text{med}} \approx 2 \times 10^8$ m/s:

$$d_{\text{prop}}(e) = \frac{L_e}{v_{\text{med}}} \tag{2.2}$$

- $\text{Loss}_{\text{base}}(e) \geq 0$ represents the baseline bit-error-rate (BER) drop probability in the absence of buffer congestion.

- $u(e,t) \in [0.0, 1.0]$ represents the time-varying link utilization ratio at time t, defined as the ratio of instantaneous transmitted throughput to line capacity:

$$u(e,t) = \min\left(1.0,\ \frac{\text{Throughput}(e,t)}{C(e)}\right) \tag{2.3}$$

- $q(e,t) \in [0, K_e]$ represents the instantaneous queue buffer occupancy in packets or bytes, bounded by the physical output port buffer capacity K_e.

A path $\mathcal{P}_{s,t}$ from source node $s \in V$ to destination node $t \in V$ is defined as an alternating sequence of vertices and directed edges:

$$\mathcal{P}_{s,t} = (v_0, e_1, v_1, e_2, v_2, \ldots, e_m, v_m) \tag{2.4}$$

such that $v_0 = s$, $v_m = t$, and $e_i = (v_{i-1}, v_i) \in E$ for all $i \in \{1, \ldots, m\}$. The path is termed *simple* (or loop-free) if all constituent vertices are distinct ($v_i \neq v_j$ for all $i \neq j$).

2.2 Traditional IP Routing Protocols and Algorithmic Mechanics

2.2.1 Dijkstra's Shortest Path First (SPF) Algorithm

Dijkstra's Shortest Path First algorithm [1] represents the cornerstone of modern network graph computation. Given a directed graph $G = (V, E)$ with non-negative edge weight function $w : E \to \mathbb{R}^+$, Dijkstra's algorithm computes the shortest paths from a single source vertex $s \in V$ to all other vertices $v \in V \setminus \{s\}$.

2.2.1.1 Algorithmic Formulation and Invariants

The algorithm maintains two sets of vertices:

1. S: The set of vertices whose final shortest-path distances from source s have been definitively determined.
2. $Q = V \setminus S$: A priority queue storing vertices whose shortest-path distances are currently tentative.

Each vertex $v \in V$ maintains two state variables:

- $d[v]$: The tentative upper bound on the distance from s to v.
- $\pi[v]$: The predecessor node of v along the current shortest path from s.

The execution proceeds through initialization and iterative edge relaxation, as formalized in Algorithm 1.

Algorithm 1 Dijkstra's Shortest Path First (SPF) Algorithm

Require: Directed graph $G = (V, E)$, non-negative weight function $w : E \to \mathbb{R}^+$, source vertex $s \in V$
Ensure: Distance array $d[v]$ and predecessor array $\pi[v]$ for all $v \in V$

Initialize Single Source:
for all $v \in V$ **do**
  $d[v] \leftarrow \infty$
  $\pi[v] \leftarrow$ NIL
end for
$d[s] \leftarrow 0$
$S \leftarrow \emptyset$
Insert all vertices $v \in V$ into priority queue Q keyed by $d[v]$
while $Q \neq \emptyset$ **do**
  $u \leftarrow$ EXTRACT-MIN(Q)
  $S \leftarrow S \cup \{u\}$
  for all outgoing neighbor v such that $(u, v) \in E$ **do**
    if $d[v] > d[u] + w(u, v)$ **then**
      {Edge Relaxation Step}
      $d[v] \leftarrow d[u] + w(u, v)$
      $\pi[v] \leftarrow u$
      DECREASE-KEY($Q, v, d[v]$)
    end if
  end for
end while
return d, π

2.2.1.2 Proof of Correctness

The correctness of Dijkstra's algorithm rests on the *greedy choice property* and the *optimal substructure property* of shortest paths. We state and prove the primary invariant:

Theorem 2.1 (Correctness of Dijkstra SPF)**.** *For each vertex u added to set S, $d[u] = \delta(s, u)$, where $\delta(s, u)$ denotes the true length of the shortest path from s to u.*

Proof. We prove this by mathematical induction on the size of set S. **Base Case**: When $|S| = 1$, $S = \{s\}$. Since $w(e) \geq 0$ for all $e \in E$, no path from s to s containing one or more edges can have a total weight strictly less than 0. Thus, $d[s] = 0 = \delta(s, s)$, establishing the base case.

Inductive Step: Assume the invariant holds for $|S| = k$. Let u be the next vertex extracted from Q and added to S, meaning $d[u] \leq d[z]$ for all $z \in Q$. Suppose, for the

sake of contradiction, that $d[u] > \delta(s, u)$. This implies that there exists an alternative, undiscovered path $\mathcal{P}$ from s to u whose total weight is strictly less than $d[u]$.

Because path $\mathcal{P}$ originates at $s \in S$ and terminates at $u \in Q$, there must exist an edge (x, y) along $\mathcal{P}$ such that $x \in S$ and $y \in Q$. The subpath from s to y has weight:

$$\delta(s, y) \leq \text{weight}(\mathcal{P}) < d[u] \tag{2.5}$$

Because $x \in S$, by the inductive hypothesis, $d[x] = \delta(s, x)$. When x was added to S, the edge (x, y) was relaxed, ensuring:

$$d[y] \leq d[x] + w(x, y) = \delta(s, x) + w(x, y) = \delta(s, y) \tag{2.6}$$

Combining these inequalities yields:

$$d[y] \leq \delta(s, y) < d[u] \tag{2.7}$$

Since both $y \in Q$ and $u \in Q$, the algorithm would have selected y instead of u in the EXTRACT-MIN operation. This contradiction disproves our assumption, establishing that $d[u] = \delta(s, u)$ for all $u \in S$. □

2.2.1.3 Computational Time Complexity Analysis

The asymptotic time complexity of Dijkstra's algorithm depends critically on the underlying data structure utilized for priority queue Q:

- **Unsorted Array**: EXTRACT-MIN takes $O(|V|)$ time, while DECREASE-KEY takes $O(1)$. Total execution time is $O(|V|^2 + |E|)$.
- **Binary Min-Heap**: Building the heap takes $O(|V|)$, $|V|$ EXTRACT-MIN operations require $O(|V| \log |V|)$, and $|E|$ DECREASE-KEY operations require $O(|E| \log |V|)$. Total time is:

$$T_{\text{binary}}(|V|, |E|) = O((|V| + |E|) \log |V|) \tag{2.8}$$

- **Fibonacci Heap**: EXTRACT-MIN runs in $O(\log |V|)$ amortized time, while DECREASE-KEY runs in $O(1)$ amortized time, yielding an optimal bound of:

$$T_{\text{fibonacci}}(|V|, |E|) = O(|E| + |V| \log |V|) \tag{2.9}$$

2.2.2 Open Shortest Path First (OSPF, RFC 2328)

Open Shortest Path First Version 2 (OSPFv2), standardized in IETF RFC 2328 [2], is the premier link-state Interior Gateway Protocol deployed in enterprise backbones. OSPF routers discover neighboring routers via the Hello protocol, form adjacencies, and exchange Link-State Advertisements (LSAs) through reliable flooding. Every router within an OSPF area constructs an identical Link-State Database (LSDB) representing graph $G = (V, E)$, over which it executes Dijkstra's SPF algorithm.

2.2.2.1 OSPF Administrative Metric Formulation

OSPF computes path costs as the sum of link interface costs. By RFC recommendation, an interface cost is an integer inversely proportional to configured line bandwidth:

$$\text{Cost}(e) = \max\left(1, \text{round}\left(\frac{\text{Reference Bandwidth}}{C(e)}\right)\right) \tag{2.10}$$

The historical default reference bandwidth is 100 Mbps (10^8 bps). Consequently, Table 2.1 details the default administrative costs assigned across standard link media.

Table 2.1: Standard OSPF Interface Administrative Costs

Transmission Media	**Line Capacity** $C(e)$	**Formula** $(10^8/C(e))$	**Default OSPF Cost**
T1 Serial Trunk	1.544 Mbps	100/1.544	64
Fast Ethernet (FE)	100 Mbps	100/100	1
Gigabit Ethernet (GE)	1,000 Mbps	100/1000 = 0.1	1 (Clamped to 1)
10-Gigabit Ethernet (10GE)	10,000 Mbps	100/10000 = 0.01	1 (Clamped to 1)
100-Gigabit Core Fiber	100,000 Mbps	100/100000 = 0.001	1 (Clamped to 1)

2.2.2.2 Limitations of Administrative Metrics

Equation 2.10 illustrates a limitation of the configured OSPF metric: link costs do not necessarily reflect instantaneous utilization. Under some configurations, Gigabit, 10GE, and 100GE links may receive the same integer cost. Unless metrics are updated, OSPF does not respond directly to transient queue congestion, so traffic may continue to use a route whose links have become heavily utilized.

2.2.3 Equal-Cost Multi-Path (ECMP)

To mitigate single-path bottlenecking, modern routers implement Equal-Cost Multi-Path (ECMP). When Dijkstra's algorithm identifies M distinct paths $\mathcal{P}_1, \ldots, \mathcal{P}_M$ between source s and destination t that possess the exact same minimum cost $\text{Cost}(\mathcal{P}_i) = \min_j \text{Cost}(\mathcal{P}_j)$, the router distributes flows across all M egress ports.

2.2.3.1 Flow Hashing Mechanics

To prevent packet reordering within a single TCP connection (which triggers spurious duplicate ACKs and throttles TCP sender congestion windows), ECMP hashes the packet's invariant 5-tuple:

$$\text{Egress Port Index} = \text{Hash}(\text{IP}_{\text{src}}, \text{IP}_{\text{dst}}, \text{Proto}, \text{Port}_{\text{src}}, \text{Port}_{\text{dst}}) \pmod{M} \quad (2.11)$$

where the hash is typically evaluated via CRC32 or Pearson's hash.

2.2.3.2 Limitations of ECMP in Modern Fabrics

Under uniform, small-flow workloads on symmetric topologies, ECMP can provide linear throughput scaling. Its performance may be affected by the following limitations:

1. **Restriction to Equal-Cost Paths**: ECMP does not distribute traffic across paths with different configured costs. For example, it may not use a three-hop alternative when a two-hop path has the lower metric, even if current congestion favors the alternative.

2. **Elephant Flow Collisions**: High-bandwidth elephant flows constitute less than 10% of all flow counts but account for more than 80% of total byte volume. When two or more elephant flows hash to the same egress port index, the port's output queue buffer is overwhelmed, creating severe congestion while parallel equal-cost paths remain severely underutilized.

2.2.4 Dynamic Routing Heuristics and the Herd Behavior Phenomenon

To address static oblivious forwarding, dynamic heuristics were introduced:

- **Widest Shortest Path (WSP / CSPF)**: Selects among candidate shortest paths the one that maximizes residual bottleneck bandwidth:

$$\mathcal{P}_{\text{WSP}} = \arg \max_{\mathcal{P} \in \mathcal{P}_{\text{min_hops}}} \min_{e \in \mathcal{P}} \left(C(e) - \text{Throughput}(e, t) \right) \quad (2.12)$$

- **Least Loaded Routing (LLR)**: Greedily selects the path that minimizes a weighted linear combination of maximum link utilization and average path utilization:

$$\mathcal{P}_{\text{LLR}} = \arg\min_{\mathcal{P}\in\mathcal{P}_{\text{cand}}} \left(0.65 \cdot \max_{e\in\mathcal{P}} u(e,t) + 0.35 \cdot \frac{1}{|\mathcal{P}|} \sum_{e\in\mathcal{P}} u(e,t) \right) \tag{2.13}$$

2.2.4.1 Mathematical Formalization of Herd Behavior

While intuitive, greedy heuristics suffer from systemic instability in discrete telemetry environments. Let Δt_{poll} denote the discrete controller statistics polling interval (e.g., $\Delta t_{\text{poll}} = 3.0$ seconds). Suppose at time t_0, the controller measures link utilizations across two parallel candidate links e_1 and e_2:

$$u(e_1, t_0) = 0.55, \quad u(e_2, t_0) = 0.20 \tag{2.14}$$

During the subsequent polling window $[t_0, t_0 + \Delta t_{\text{poll}}]$, suppose N new high-bandwidth flows arrive with aggregate demand $D_{\text{agg}} = \sum_{i=1}^{N} d_i = 0.60 \cdot C$.

Because the controller's telemetry state remains static throughout the window $[t_0, t_0 + \Delta t_{\text{poll}}]$, every single one of the N arriving flows independently evaluates link e_2 as the "least loaded" link:

$$\mathcal{P}_{\text{LLR}}(i) = e_2 \quad \forall i \in \{1, \ldots, N\} \tag{2.15}$$

Consequently, link e_2 is immediately flooded with all incoming flows, driving its real-time utilization to:

$$u(e_2, t_0 + \Delta t_{\text{poll}}) = 0.20 + 0.60 = 0.80 \tag{2.16}$$

At time $t_0 + \Delta t_{\text{poll}}$, the next polling cycle reports $u(e_1) = 0.55$ and $u(e_2) = 0.80$. Now, link e_1 is perceived as the "least loaded." All subsequent flows arriving in the next window are routed exclusively to e_1, reversing the congestion and driving e_1 into saturation.

This perpetual oscillation between candidate links is termed the **Herd Behavior Phenomenon**. It degrades network performance by causing severe packet jitter, TCP ACK compression, and persistent buffer overshoots. Resolving herd behavior requires an intelligent agent that models expected downstream dynamics and distributes traffic probabilistically based on long-term value estimation.

2.3 Software-Defined Networking (SDN) and OpenFlow 1.3

2.3.1 ONF Architectural Decomposition

Software-Defined Networking, standardized by the Open Networking Foundation (ONF) [3], abstracts network functionality into three logically decoupled functional planes, as depicted in Figure 2.1.



Figure 2.1: Detailed Three-Tier Software-Defined Networking Architectural Model.

2.3.2 OpenFlow 1.3 Pipeline and Primitives

OpenFlow 1.3 [4] establishes the operational standard for SDN communication:

2.3.2.1 Flow Table Structure and Flow Entries

An OpenFlow 1.3 switch contains multiple flow tables numbered 0 to 254. Packet processing starts at Flow Table 0. Each flow entry contains:

- **Match Fields**: Ingress switch port, Ethernet source/destination MAC addresses, EtherType, VLAN ID, IPv4 source/destination addresses, IP protocol (TCP/UDP/ICMP), and Layer 4 source/destination transport ports.
- **Priority**: An integer value (0 to 65535). If multiple flow entries match a packet, the switch strictly executes the entry with the highest numerical priority.
- **Counters**: Total packets and bytes matched by this specific flow rule.
- **Instructions**: Directives modifying packet processing, such as `APPLY_ACTIONS`, `WRITE_ACTIONS`, `GOTO_TABLE`, or `WRITE_METADATA`.
- **Timeouts**:
  - `idle_timeout`: The flow entry is automatically purged by switch hardware if no packet matches the entry for this number of seconds.
  - `hard_timeout`: The flow entry is unconditionally purged after this number of seconds, regardless of traffic volume.

2.3.2.2 The Table-Miss Pipeline

To handle previously unseen traffic, the controller installs a default **Table-Miss Flow Entry** at Priority 0 with a completely wildcarded match field (`parser.OFPMatch()`). When an incoming packet matches no higher-priority flow rule, the switch executes the table-miss instruction, encapsulating the packet header and buffering the payload, and sending an asynchronous `OFPPacketIn` event message to the controller.

2.3.2.3 Group Tables

OpenFlow 1.3 introduced Group Tables to enable advanced multi-path and resilient forwarding beyond simple port output:

- **All Groups (`OFPGT_ALL`)**: Duplicates the packet to every action bucket in the group, used for broadcast and multicast emulation.
- **Select Groups (`OFPGT_SELECT`)**: Evaluates a switch-internal selection algorithm (e.g., 5-tuple hash) to execute a single bucket out of multiple candidate action buckets. Each bucket carries an explicit `weight`, enabling **Weighted Cost Multi-Path (WCMP)** flow splitting proportional to available path capacity.
- **Fast Failover Groups (`OFPGT_FF`)**: Evaluates the live hardware operational state of a specified `watch_port`. If physical carrier signal is lost on the primary port, switch hardware immediately switches traffic to the alternate backup bucket within sub-milliseconds, bypassing controller communication.

2.3.3 The Ryu SDN Controller Framework

The Ryu SDN framework [5] is an open-source, component-based software-defined networking operating system implemented in Python. Ryu employs an asynchronous, event-driven concurrency model powered by Eventlet greenlets:

- **Green Threads**: Lightweight user-space coroutines managed cooperatively by Eventlet's Hub reactor. Greenlets yield execution during blocking I/O, allowing a single CPU core to handle thousands of concurrent switch connections and telemetry polling cycles without POSIX thread synchronization overhead.
- **Event Dispatcher**: Decouples OpenFlow message decoders from application logic. When an OpenFlow message arrives over the TCP socket, Ryu parses the raw binary frame, instantiates an OpenFlow event object (e.g., `EventOFPPacketIn`), and dispatches it to registered event handlers using Python decorators (e.g., `@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)`).

2.4 Deep Reinforcement Learning Theory

2.4.1 Markov Decision Processes (MDP)

Reinforcement Learning models sequential decision-making under uncertainty as a discrete-time Markov Decision Process. An MDP is defined formally by the 5-tuple:

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle \tag{2.17}$$

where:

- $\mathcal{S}$ is the set of all valid environmental states (the state space).
- $\mathcal{A}$ is the set of all available agent actions (the action space).
- $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the state transition probability distribution, satisfying the Markov property:

$$\mathbb{P}(s_{t+1} = s' \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) = \mathbb{P}(s_{t+1} = s' \mid s_t, a_t) \tag{2.18}$$

- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the scalar reward function, returning reward $r_t = \mathcal{R}(s_t, a_t)$ upon executing action a_t in state s_t.
- $\gamma \in [0, 1)$ is the temporal discount factor, balancing immediate vs. future rewards.

2.4.2 Bellman Optimality and Q-Learning

The value of taking action a in state s under policy $\pi(a|s)$ is defined by the action-value function $Q^\pi(s, a)$:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \,\middle|\, s_t = s, a_t = a \right] \tag{2.19}$$

The optimal policy π^* maximizes the expected return:

$$Q^*(s, a) = \max_\pi Q^\pi(s, a) \quad \forall s \in \mathcal{S}, \forall a \in \mathcal{A} \tag{2.20}$$

The optimal action-value function satisfies the fundamental **Bellman Optimality Equation**:

$$Q^*(s, a) = \mathcal{R}(s, a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}(s' \mid s, a) \max_{a' \in \mathcal{A}} Q^*(s', a') \tag{2.21}$$

In model-free tabular Q-learning [6], transitions (s_t, a_t, r_t, s_{t+1}) update tabular estimates iteratively via temporal difference (TD) learning:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \Big(r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \Big) \tag{2.22}$$

where $\alpha \in (0, 1]$ is the learning rate.

2.4.3 Deep Q-Networks (DQN)

When state space $\mathcal{S}$ is continuous and high-dimensional, tabular representation becomes intractable. Mnih et al. [7] introduced Deep Q-Networks (DQN), approximating $Q^*(s, a) \approx Q(s, a; \theta)$ using a deep neural network parameterized by weights θ. The network parameters are optimized by minimizing the mean squared Bellman error:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\Big(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \Big)^2 \right] \tag{2.23}$$

where:

- $\mathcal{D}$ represents an experience replay buffer storing observed transition tuples $e_t = (s_t, a_t, r_t, s_{t+1})$.
- θ^- represents the parameters of a separate *target network*, periodically synchronized with the online network to prevent divergence and feedback loops.

2.4.4 Double Deep Q-Networks (DDQN)

Standard DQN suffers from systematic overestimation of Q-values due to the presence of the maximization operator $\max_{a'} Q(s', a'; \theta^-)$ in the target value:

$$\mathbb{E}\left[\max_{a'} Q(s', a')\right] \geq \max_{a'} \mathbb{E}\left[Q(s', a')\right] \tag{2.24}$$

In noisy network environments where link utilizations fluctuate, this maximization operator frequently overestimates suboptimal paths that randomly experienced low instantaneous delay.

Van Hasselt et al. [8] resolved this bias with Double DQN (DDQN). DDQN completely decouples action selection from action evaluation: the online network (θ) selects the greedy action, while the target network (θ^-) evaluates the Q-value of that selected action:

$$Y_t^{\text{DoubleQ}} = r_t + \gamma\, Q\left(s_{t+1},\, \arg\max_{a'} Q(s_{t+1}, a'; \theta_t);\, \theta_t^-\right) \tag{2.25}$$

This mathematical decoupling provably eliminates overestimation bias, ensuring stable, monotonic value convergence.

2.4.5 Dueling Q-Network Architecture

In network routing, evaluating the intrinsic quality of a network state $V(s)$ is often critical regardless of which specific candidate path is chosen. For instance, if all paths leading to a destination are severely congested, the value of the state itself is universally poor.

Wang et al. [9] introduced the Dueling architecture. The neural network splits after intermediate feature representations into two decoupled dense streams:

1. A scalar **State-Value Stream** $V(s; \theta, \beta) \in \mathbb{R}^1$, evaluating the baseline quality of state s.

2. An **Advantage Stream** $A(s, a; \theta, \alpha) \in \mathbb{R}^{|\mathcal{A}|}$, evaluating the relative benefit of taking candidate action $a \in \mathcal{A}$ compared to all other candidate actions.

To ensure mathematical identifiability (so that $V(s)$ and $A(s, a)$ are uniquely recoverable from $Q(s, a)$), the mean advantage is subtracted from each action advantage:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(s, a'; \theta, \alpha)\right) \tag{2.26}$$

Subtracting the mean ensures that $\sum_a (Q(s, a) - V(s)) = 0$, guaranteeing that the

state value $V(s)$ accurately tracks the average return of the state while advantages $A(s, a)$ represent zero-centered relative deviations.

2.4.6 Prioritized Experience Replay (PER)

In uniform experience replay, all transitions in memory buffer $\mathcal{D}$ are sampled with equal probability $1/N$. In computer networks, critical congestion events, queue buffer overflows, and link failures occur infrequently. Under uniform sampling, the neural network spends most of its gradient updates re-learning trivial, uncongested steady-state conditions, failing to learn emergency anti-congestion policies.

Schaul et al. [10] formulated Prioritized Experience Replay (PER). Each transition $i = (s_i, a_i, r_i, s_{i+1})$ is assigned a priority p_i proportional to the magnitude of its absolute Temporal Difference (TD) error:

$$p_i = |\delta_i| + \epsilon_{\text{per}} \tag{2.27}$$

where $\delta_i = Q(s_i, a_i; \theta) - Y_i^{\text{DoubleQ}}$ and $\epsilon_{\text{per}} = 10^{-5}$ is a small positive constant preventing zero probability transitions. The probability of sampling transition i is defined by:

$$P(i) = \frac{p_i^\alpha}{\sum_{k=1}^{N} p_k^\alpha} \tag{2.28}$$

where $\alpha \in [0, 1]$ controls prioritization strength ($\alpha = 0$ corresponds to uniform sampling; $\alpha = 1$ corresponds to fully greedy prioritization).

2.4.6.1 Importance Sampling Weight Correction

Non-uniform sampling alters the underlying state visitation frequency, introducing bias into stochastic gradient descent. To correct for this estimation bias, PER rescales gradient updates using Importance Sampling (IS) weights:

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)} \right)^\beta \Big/ \max_j w_j \tag{2.29}$$

where $\beta \in [0, 1]$ controls the degree of correction. To compensate for increasing gradient stability, β is annealed linearly from $\beta_0 = 0.4$ up to 1.0 at the conclusion of training. The weighted loss function minimized by the optimizer is:

$$\mathcal{L}_{\text{PER}}(\theta) = \frac{1}{B} \sum_{i=1}^{B} w_i \cdot \mathcal{L}_{\text{Huber}}(\delta_i) \tag{2.30}$$

2.5 Taxonomy and Comparative Literature Review

To position this graduation project within the broader academic landscape, Table 2.2 presents a detailed comparative taxonomy evaluating twenty-five landmark and contemporary research works across Traffic Engineering, Software-Defined Networking, and Machine Learning.

Table 2.2: Comparison of Traffic Engineering Paradigms and Related Literature

Research / System	Year	Paradigm	Control Plane	Optimization Methodology	Key Operational Limitations
OSPF (RFC 2328) [2]	1998	Traditional	Distributed	Static Dijkstra SPF on administrative capacity weights	Completely oblivious to dynamic load; severe core bottlenecking.
ECMP [11]	2008	Traditional	Distributed	5-tuple packet hashing across equal-cost paths	Cannot use near-equal paths; severe elephant flow collisions.
Hedera [12]	2010	Dynamic SDN	Centralized	OpenFlow polling; global simulated annealing on detected elephants	High polling latency ($>$ 5s); struggles with short-lived microbursts.
Google B4 [13]	2013	WAN SDN	Hierarchical	Centralized Linear Programming (LP) with fair-share bandwidth allocation	High solver computation time ($>$ 10s); optimized for inter-DC, not intra-DC.
Microsoft SWAN [14]	2013	WAN SDN	Centralized	Multi-commodity flow LP with congestion-free rate updates	Requires host-level rate limiters; computationally heavy centralized LP.

Table 2.3: Comparison of Traffic Engineering Paradigms and Related Literature (continued)

Research / System	Year	Paradigm	Control Plane	Optimization Methodology	Key Operational Limitations
GigaFlow [15]	2014	Dynamic	Distributed	End-host TCP congestion notification and path bouncing	Increases TCP reordering; reactive rather than proactive.
Stampa et al. [16]	2017	DRL SDN	Centralized	Standard Deep Q-Network for dynamic link weight tuning	Action space scales exponentially with link count; slow convergence.
Valadarsky [17]	2017	DRL SDN	Centralized	Policy gradient algorithm predicting traffic matrices	Relies on accurate traffic matrix prediction; high inference overhead.
Aurochs [18]	2018	DRL SDN	Centralized	Continuous action actor-critic (DDPG) flow splitting	High dimensional action space; unstable training dynamics on large meshes.
RouteNet [19]	2019	GNN SDN	Centralized	Graph Neural Network (GNN) estimating delay and jitter	Acts as a performance evaluator, not a direct real-time routing policy.
D3QN-PER TE (Ours)	2026	DRL SDN	Centralized	Topologically normalized D3QN with binary SumTree PER and asymptotic barrier	Sub-millisecond inference (0.33 ms); verified zero-shot transferability.

As established in Table 2.2, existing approaches either suffer from oblivious static bottlenecks (OSPF, ECMP), excessive centralized LP execution latency (B4, SWAN), or poor topological generalizability (Stampa, Aurochs). Our architecture provides the

optimal equilibrium: sub-millisecond neural decision throughput, loop-free candidate diversity paths, asymptotic barrier-driven congestion avoidance, and complete zero-shot topology invariance.

Chapter 3

Mathematical Modeling and Problem Formulation

3.1 Network Topology and Multigraph Mathematical Formalism

We model a software-defined communication network as a directed multigraph $G = (V, E)$, where $V = \{1, 2, \ldots, |V|\}$ represents the finite set of OpenFlow switching elements, and $E \subseteq V \times V$ represents the set of directed physical transmission links. Multiple directed edges may exist between adjacent switching pairs to model redundant trunk aggregates or bundled optical channels.

For each directed edge $e = (u, v) \in E$, the following physical and operational invariants are defined:

1. **Transmission Bandwidth Capacity ($C_e \in \mathbb{R}^+$)**: The maximum line rate of the link in Megabits per second (Mbps). For bidirectional communication, links $e = (u, v)$ and $\bar{e} = (v, u)$ possess symmetric capacity $C_e = C_{\bar{e}}$, though their real-time traffic utilizations are independent.

2. **Base Physical Delay ($d_{\mathbf{base}}(e) \in \mathbb{R}^+$)**: The minimum, non-congested transit delay across link e, combining physical medium propagation delay and serializer transmission delay for standard Maximum Transmission Unit (MTU = 1500 bytes) frames:

$$d_{\text{base}}(e) = d_{\text{prop}}(e) + d_{\text{tx}}(e) = \frac{L_e}{v_{\text{med}}} + \frac{1500 \times 8}{C_e \times 10^6} \quad \text{[seconds]} \tag{3.1}$$

   where L_e is the fiber/cable physical length in meters and $v_{\text{med}} \approx 2 \times 10^8$ m/s.

3. **Instantaneous Link Utilization ($u_e(t) \in [0.0, 1.0]$)**: The dimensionless frac-

tion of link capacity utilized by aggregate transmitted flows at time t:

$$u_e(t) = \min\left(1.0,\ \frac{\lambda_e(t)}{C_e}\right) \tag{3.2}$$

where $\lambda_e(t)$ is the aggregate traffic arrival rate on link e in Mbps.

3.2 The Multi-Commodity Flow Optimization Problem

3.2.1 Continuous Linear Formulation

Let $\mathcal{K} = \{1, 2, \ldots, K\}$ denote the active set of traffic commodities traversing network G. Each commodity $k \in \mathcal{K}$ is defined by a source ingress switch $s_k \in V$, a destination egress switch $t_k \in V$, and an instantaneous traffic demand $D_k > 0$.

If traffic flows are infinitely divisible and multi-path splitting is unrestricted, the optimal routing policy solves the classical fractional Multi-Commodity Flow (MCF) optimization problem:

$$\text{Minimize} \quad \Psi(f) = \sum_{e \in E} \Phi_e\left(\frac{\sum_{k \in \mathcal{K}} f_e^k}{C_e}\right) \tag{3.3}$$

$$\text{subject to} \quad \sum_{e \in \delta^+(v)} f_e^k - \sum_{e \in \delta^-(v)} f_e^k = b_v^k \quad \forall v \in V, \forall k \in \mathcal{K} \tag{3.4}$$

$$\sum_{k \in \mathcal{K}} f_e^k \leq C_e \quad \forall e \in E \tag{3.5}$$

$$f_e^k \geq 0 \quad \forall e \in E, \forall k \in \mathcal{K} \tag{3.6}$$

where f_e^k represents the flow rate of commodity k traversing directed edge e, $\delta^+(v)$ and $\delta^-(v)$ denote the sets of outgoing and incoming edges incident on vertex v, and the divergence vector b_v^k is:

$$b_v^k = \begin{cases} D_k & \text{if } v = s_k \\ -D_k & \text{if } v = t_k \\ 0 & \text{if } v \in V \setminus \{s_k, t_k\} \end{cases} \tag{3.7}$$

The penalty function $\Phi_e(u)$ is a continuous, strictly convex function that models packet queuing delay and buffer overflow risk as link utilization approaches unity.

3.2.2 Unsplittable Flow Integer Formulation and Complexity

In packet-switched computer networks, splitting a single transport-layer TCP flow across multiple paths with disparate latencies induces severe packet reordering. Out-of-order packet arrivals trigger TCP Fast Retransmit and cause the TCP sender to artificially halve its congestion window (*cwnd*), collapsing application throughput.

Consequently, production traffic engineering mandates **Unsplittable Multi-Commodity Flow (UMCF)**, wherein each commodity k must follow a single, atomic path from source to destination. We introduce binary decision variables:

$$x_e^k \in \{0, 1\} \quad \forall e \in E, \forall k \in \mathcal{K} \tag{3.8}$$

where $x_e^k = 1$ if commodity k traverses edge e, and 0 otherwise. The flow conservation constraint becomes:

$$\sum_{e \in \delta^+(v)} x_e^k - \sum_{e \in \delta^-(v)} x_e^k = \begin{cases} 1 & \text{if } v = s_k \\ -1 & \text{if } v = t_k \\ 0 & \text{otherwise} \end{cases} \qquad \forall v \in V, \forall k \in \mathcal{K} \tag{3.9}$$

and the link capacity constraint becomes:

$$\sum_{k \in \mathcal{K}} D_k \cdot x_e^k \leq C_e \quad \forall e \in E \tag{3.10}$$

Theorem 3.1 ($\mathcal{NP}$-Hardness of Unsplittable Traffic Engineering)**.** *The Unsplittable Multi-Commodity Flow problem with capacity constraints is strictly $\mathcal{NP}$-hard.*

Proof. We prove $\mathcal{NP}$-hardness by polynomial-time reduction from the classical 2-Directed Disjoint Paths (2-DDP) problem, which is proven to be $\mathcal{NP}$-complete [20].

An instance of 2-DDP consists of a directed graph $G = (V, E)$ and two source-terminal pairs (s_1, t_1) and (s_2, t_2). The question is whether there exist two edge-disjoint paths $\mathcal{P}_1$ (from s_1 to t_1) and $\mathcal{P}_2$ (from s_2 to t_2) such that $E(\mathcal{P}_1) \cap E(\mathcal{P}_2) = \emptyset$.

We transform this 2-DDP instance into an instance of UMCF:

1. Assign unit capacity to all edges: $C_e = 1$ for all $e \in E$.
2. Define two commodities $\mathcal{K} = \{1, 2\}$ with demands $D_1 = 1$ and $D_2 = 1$.
3. Set the source-destination pairs as (s_1, t_1) and (s_2, t_2).

In any valid solution to this UMCF instance, for each edge $e \in E$, the capacity constraint requires:

$$D_1 \cdot x_e^1 + D_2 \cdot x_e^2 = 1 \cdot x_e^1 + 1 \cdot x_e^2 \leq 1 \tag{3.11}$$

Because $x_e^k \in \{0,1\}$, it is impossible for both $x_e^1 = 1$ and $x_e^2 = 1$ on any edge e. Therefore, the paths selected for commodity 1 and commodity 2 must be strictly edge-disjoint. Thus, a feasible unsplittable flow exists if and only if the 2-DDP instance has a solution. Since the reduction takes linear time, UMCF is $\mathcal{NP}$-hard. □

Because UMCF is $\mathcal{NP}$-hard, computing exact global integer linear programs in real-time under sub-second traffic variations is computationally intractable. This theoretical bottleneck establishes the imperative for an autonomous Deep Reinforcement Learning policy capable of sub-millisecond heuristic inference.

3.3 Markov Decision Process (MDP) Formulation

When a new flow $f = (s, t, \text{demand})$ arrives at ingress switch s destined for egress switch t, the SDN controller intercepts the first packet miss and invokes the DRL routing agent. We formalize this decision loop as an MDP:

3.3.1 State Space Representation (10-Dimensional Normalized State)

A major barrier in applying reinforcement learning to networking is achieving topological invariance. If the state vector directly encodes raw node IDs or per-link metrics, the neural network structure becomes permanently tied to a specific switch count $|V|$ and link count $|E|$.

To solve this, we introduce a **Topologically Normalized 10-Dimensional State**

Vector $s_t \in \mathbb{R}^{10}$:

$$s_t = \begin{bmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \\ s_6 \\ s_7 \\ s_8 \\ s_9 \end{bmatrix} = \begin{bmatrix} \min\left(1.0,\ \max\left(0.1,\ \dfrac{|\mathcal{P}_0| - 1}{10.0}\right)\right) \\ \max_{e \in \mathcal{P}_0} u(e) \\ \max_{e \in \mathcal{P}_1} u(e) \\ \max_{e \in \mathcal{P}_2} u(e) \\ \max_{e \in \mathcal{P}_3} u(e) \\ \min\left(1.0,\ \dfrac{\sum_{e \in \mathcal{P}_0} d_{\text{base}}(e)}{50.0}\right) \\ \min\left(1.0,\ \dfrac{\sum_{e \in \mathcal{P}_1} d_{\text{base}}(e)}{50.0}\right) \\ \min\left(1.0,\ \dfrac{\sum_{e \in \mathcal{P}_2} d_{\text{base}}(e)}{50.0}\right) \\ \min\left(1.0,\ \dfrac{\sum_{e \in \mathcal{P}_3} d_{\text{base}}(e)}{50.0}\right) \\ \max_{e \in E} u(e) \end{bmatrix} \tag{3.12}$$

Table 3.1 details the physical interpretation, mathematical domain, and architectural rationale for each feature.

Theorem 3.2 (Topological Invariance of State Vector s_t)**.** *The state representation s_t is invariant to graph isomorphic relabeling, switch permutations, and absolute graph cardinality $|V|$ and $|E|$.*

Proof. Let $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ be two arbitrary network graphs. Feature s_0 depends strictly on scalar path length $|\mathcal{P}_0|$, scaled by constant 10.0. Features s_1–s_4 and s_9 depend strictly on scalar link utilization ratios $u(e) \in [0.0, 1.0]$, which are dimensionless quotients of throughput over capacity. Features s_5–s_8 depend on scalar physical latencies scaled by 50.0 ms. None of the features index switch identifiers, port numbers, or matrix dimensions. Therefore, the vector dimensions and semantic meaning remain strictly identical regardless of graph scale, enabling zero-shot model evaluation across arbitrary graphs. □

3.3.2 Action Space Formulation ($\mathcal{A} \in \{0, 1, 2, 3\}$)

Rather than permitting the neural network to output unconstrained next-hop ports at each switch (which creates an exponentially scaling action space and high loop risk), our architecture uses an action space of size $K = 4$ representing diversity-aware end-to-end candidate paths:

$$\mathcal{A} = \{0, 1, 2, 3\} \tag{3.13}$$

Table 3.1: Detailed Mathematical Specification of the 10-Dimensional State Space Vector

Feature	Mathematical Definition	Domain	Physical Interpretation & Role
s_0	$\text{clamp}\left(\frac{\lvert\mathcal{P}_0\rvert-1}{10.0}, 0.1, 1.0\right)$	$[0.1, 1.0]$	Normalized hop distance of primary shortest path. Encodes spatial topological diameter.
s_1	$\max_{e\in\mathcal{P}_0} u(e)$	$[0.0, 1.0]$	Peak bottleneck link utilization along Candidate Path 0 (Shortest Path).
s_2	$\max_{e\in\mathcal{P}_1} u(e)$	$[0.0, 1.0]$	Peak bottleneck link utilization along Candidate Path 1 (Second Shortest Path).
s_3	$\max_{e\in\mathcal{P}_2} u(e)$	$[0.0, 1.0]$	Peak bottleneck link utilization along Candidate Path 2 (Edge-Disjoint Bypass).
s_4	$\max_{e\in\mathcal{P}_3} u(e)$	$[0.0, 1.0]$	Peak bottleneck link utilization along Candidate Path 3 (Node-Diverse / Widest Path).
s_5	$\min\left(1.0, \frac{\sum_{e\in\mathcal{P}_0} d_{\text{base}}(e)}{50.0}\right)$	$[0.0, 1.0]$	Cumulative propagation latency of Path 0, normalized against 50 ms scale.
s_6	$\min\left(1.0, \frac{\sum_{e\in\mathcal{P}_1} d_{\text{base}}(e)}{50.0}\right)$	$[0.0, 1.0]$	Cumulative propagation latency of Path 1, normalized against 50 ms scale.
s_7	$\min\left(1.0, \frac{\sum_{e\in\mathcal{P}_2} d_{\text{base}}(e)}{50.0}\right)$	$[0.0, 1.0]$	Cumulative propagation latency of Path 2, normalized against 50 ms scale.
s_8	$\min\left(1.0, \frac{\sum_{e\in\mathcal{P}_3} d_{\text{base}}(e)}{50.0}\right)$	$[0.0, 1.0]$	Cumulative propagation latency of Path 3, normalized against 50 ms scale.
s_9	$\max_{e\in E} u(e)$	$[0.0, 1.0]$	Network-wide global peak link utilization. Encodes overall network stress.

Candidate paths are synthesized via Yen's K-Shortest Paths algorithm [21] augmented with diversity filtering:

1. **Action 0 ($\mathcal{P}_0$)**: Standard Dijkstra Shortest Path (minimum hop count and minimal propagation delay).

2. **Action 1 ($\mathcal{P}_1$)**: Second Shortest Simple Path.

3. **Action 2 ($\mathcal{P}_2$)**: Edge-Diverse Lateral Bypass Path. Synthesized by selecting the simple path that minimizes edge overlap with $\mathcal{P}_0$:

$$\mathcal{P}_2 = \arg \min_{\mathcal{P} \in \mathcal{P}_{\text{cand}} \setminus \{\mathcal{P}_0, \mathcal{P}_1\}} \sum_{e \in \mathcal{P}} \mathbb{I}(e \in \mathcal{P}_0) \tag{3.14}$$

4. **Action 3 ($\mathcal{P}_3$)**: Node-Diverse / Widest Path. Synthesized by selecting the path that maximizes residual bottleneck bandwidth across all intermediate transit nodes disjoint from $\mathcal{P}_0$:

$$\mathcal{P}_3 = \arg \max_{\mathcal{P} \in \mathcal{P}_{\text{cand}} \setminus \{\mathcal{P}_0, \mathcal{P}_1, \mathcal{P}_2\}} \min_{e \in \mathcal{P}} \left(C(e) - \text{Throughput}(e)\right) \tag{3.15}$$

3.3.3 Multi-Objective Reward Function

The scalar reward function balances five competing objectives: hop overhead, propagation and queuing latency, network jitter, packet loss rate, and peak link congestion:

$$\mathcal{R}(s_t, a_t) = -\Big(w_{\text{hop}} \cdot |\mathcal{P}_a| + w_{\text{lat}} \cdot D(\mathcal{P}_a) + \Phi_{\text{cong}}(U_{\max}) + w_{\text{jit}} \cdot J(\mathcal{P}_a) + w_{\text{loss}} \cdot P_{\text{loss}}(\mathcal{P}_a) \Big) \tag{3.16}$$

where $U_{\max} = \max_{e \in \mathcal{P}_a} u(e)$ is the peak bottleneck link utilization on the chosen path, and the calibrated weights are:

$$w_{\text{hop}} = 0.35, \quad w_{\text{lat}} = 0.06, \quad w_{\text{jit}} = 0.25, \quad w_{\text{loss}} = 0.50 \tag{3.17}$$

3.4 Derivation of the Asymptotic Congestion Barrier Penalty

A pivotal theoretical contribution of this project is the formulation of the **Asymptotic Congestion Barrier Penalty** $\Phi_{\text{cong}}(U_{\max})$. In classical interior-point optimization, barrier functions (such as the Frisch logarithmic barrier $-\mu \ln(C - f)$) prevent optimization variables from violating feasibility boundaries by generating infinite penalty barriers at constraints.

We construct a specialized piecewise continuous barrier function:

$$\Phi_{\text{cong}}(U_{\max}) = \begin{cases} 1.5 \cdot U_{\max} & \text{if } U_{\max} \leq 0.70 \\ 12.0 \cdot \left(\dfrac{U_{\max}^{1.8}}{\max(0.01,\ 1.02 - U_{\max})} \right) & \text{if } U_{\max} > 0.70 \end{cases} \tag{3.18}$$

3.4.1 Mathematical Analysis and Gradient Dynamics

We analyze the first-order derivative of Φ_{cong} with respect to bottleneck utilization $u = U_{\max}$ in the barrier regime ($u > 0.70$):

$$\Phi_{\text{cong}}(u) = 12.0 \cdot \frac{u^{1.8}}{1.02 - u} \tag{3.19}$$

Applying the quotient rule:

$$\begin{aligned} \frac{d\Phi_{\text{cong}}}{du} &= 12.0 \cdot \frac{1.8u^{0.8}(1.02 - u) - u^{1.8}(-1)}{(1.02 - u)^2} \\ &= 12.0 \cdot \frac{1.836u^{0.8} - 1.8u^{1.8} + u^{1.8}}{(1.02 - u)^2} \\ &= 12.0 \cdot \frac{1.836u^{0.8} - 0.8u^{1.8}}{(1.02 - u)^2} > 0 \quad \forall u \in (0.70, 1.0] \end{aligned} \tag{3.20}$$

Furthermore, the second derivative satisfies:

$$\frac{d^2\Phi_{\text{cong}}}{du^2} > 0 \quad \forall u \in (0.70, 1.0] \tag{3.21}$$

confirming that $\Phi_{\text{cong}}(u)$ is strictly convex and monotonically escalating.

Table 3.2: Numerical Escalation of the Asymptotic Barrier Penalty vs. Linear Hop Penalty

Utilization $U_{\max}$	**Regime**	**Barrier Penalty** Φ_{cong}	**Hop Penalty** ($0.35 \cdot 3$ **hops**)	**Dominant Agent Incentive**
0.20	Linear	0.30	1.05	Conserve hops; select shortest path $\mathcal{P}_0$
0.40	Linear	0.60	1.05	Conserve hops; select shortest path $\mathcal{P}_0$
0.60	Linear	0.90	1.05	Conserve hops; select shortest path $\mathcal{P}_0$
0.70	Boundary	1.05	1.05	Indifference threshold between hop count and load
0.75	Barrier	26.23	1.05	**Strong incentive to bypass core**
0.85	Barrier	53.31	1.05	**Aggressive detour enforcement**
0.95	Barrier	156.42	1.05	**Absolute emergency rerouting**
0.99	Barrier	386.11	1.05	**Asymptotic penalty saturation**

Table 3.2 shows that below 70% utilization, the hop penalty is larger than the congestion penalty in the illustrated cases. Above this threshold, the barrier penalty increases to 53.31 and 156.42, compared with a hop penalty of 1.05. This difference encourages selection of lateral paths as utilization approaches saturation.

3.5 Analytical Queuing, Jitter, and Packet Loss Modeling

3.5.1 Path Latency Formulation via Kleinrock's Approximation

The end-to-end latency $D(\mathcal{P})$ across path $\mathcal{P}$ consists of three additive delay components on each traversed link e:

$$D(\mathcal{P}) = \sum_{e \in \mathcal{P}} \Big(d_{\text{prop}}(e) + d_{\text{tx}}(e) + d_{\text{queue}}(e, u_e) \Big) \tag{3.22}$$

Under Kleinrock's network independence approximation [22], packet arrivals at each internal switch output queue follow Poisson distributions with average rate λ_e, and packet service times are exponentially distributed with mean service rate $\mu_e = C_e/\bar{L}$, where $\bar{L}$ is the average packet size.

Each link buffer operates as an $M/M/1$ queue. The steady-state average queuing delay is given by:

$$W_q(e) = \frac{\lambda_e}{\mu_e(\mu_e - \lambda_e)} = \frac{u_e}{\mu_e(1 - u_e)} \tag{3.23}$$

To avoid numerical singularity as $u_e \to 1.0$ and model practical buffer limiters, we formulate the calibrated queuing delay model:

$$d_{\text{queue}}(e, u_e) = d_{\text{base}}(e) \cdot \left(0.2 + \frac{0.8}{\max\left(0.05,\, 1.0 - \min(0.95,\, u_e)\right)} \right) \tag{3.24}$$

Under zero load ($u_e = 0$), $d_{\text{queue}} = d_{\text{base}}$. When load reaches 95%, the denominator collapses to 0.05, scaling queuing delay by sixteen-fold to $16.2 \cdot d_{\text{base}}$, accurately modeling real-world buffer congestion.

3.5.2 Network Jitter Model (RFC 3393 / RFC 3550)

Packet delay variation (jitter) measures the statistical variance in one-way end-to-end transit times. Standardized in RFC 3393 [23] and RFC 3550 [24], jitter is evaluated via an Exponential Moving Average (EMA) filter:

$$J_t = J_{t-1} + \frac{|D_t - D_{t-1}| - J_{t-1}}{16} \tag{3.25}$$

In our analytical network queuing engine, jitter volatility is modeled directly as a function of peak path utilization $U_{\max}$:

$$J(\mathcal{P}) = 0.35 \cdot |\mathcal{P}| + 0.45 \cdot D(\mathcal{P}) \cdot \min\left(5.0,\ \frac{U_{\max}^2}{\max(0.02,\ 1.0 - \min(0.98,\ U_{\max}))}\right) \tag{3.26}$$

Under low link utilization, the model predicts relatively stable queue sizes and jitter below 1 ms. As utilization exceeds 80%, bursty packet arrivals can produce larger queue fluctuations and jitter.

3.5.3 Buffer Overflow Packet Loss Model ($M/M/1/K$)

In physical OpenFlow switches, output port queues possess a finite buffer capacity of K packets (typically 100 to 1000 packets in commodity hardware). Under the $M/M/1/K$ queuing model, packet loss occurs when an incoming packet arrives while all K buffer slots are occupied.

The stationary distribution of an $M/M/1/K$ queue with traffic intensity $\rho = \lambda/\mu$ yields the blocking probability P_K:

$$P_{\text{loss}} = P_K = \frac{(1-\rho)\rho^K}{1-\rho^{K+1}} \tag{3.27}$$

When traffic intensity $\rho = u \leq 0.70$ and buffer size $K \geq 100$, the theoretical loss probability is astronomically small ($P_{\text{loss}} < 10^{-6}$). However, when utilization surges beyond 80%, microbursts exhaust the remaining buffer slots, triggering drop-tail packet loss. We formulate this empirical reality via:

$$P_{\text{loss}}(u) = \begin{cases} 0.01\% & \text{if } u \leq 0.70 \\ 0.01\% + 35.0 \cdot \left(\dfrac{u - 0.70}{0.30}\right)^{2.2} & \text{if } u > 0.70 \end{cases} \tag{3.28}$$

At 70% load, packet loss is 0.01%. At 85% load, loss rises to 7.6%. At 100% saturation, packet loss escalates to 35.01%, reflecting severe buffer overflow.

3.5.4 Jain's Fairness Index Formulation

To quantify the balance of link loads across the network topology, we compute Jain's Fairness Index [25]:

$$\mathcal{J}(u) = \frac{\left(\sum_{e \in E} u(e)\right)^2}{|E| \cdot \sum_{e \in E} u(e)^2} \tag{3.29}$$

Theorem 3.3 (Properties of Jain's Fairness Index)**.** *Jain's Fairness Index satisfies:*

1. ***Boundedness:*** $\frac{1}{|E|} \leq \mathcal{J}(u) \leq 1.0$.

2. ***Scale Invariance***: *$\mathcal{J}(\alpha u) = \mathcal{J}(u)$ for all scalar multipliers $\alpha > 0$.*

3. ***Continuity***: *$\mathcal{J}(u)$ is smooth and continuous for all non-zero link load vectors.*

Proof. Let $u = [u_1, \ldots, u_{|E|}]^T \in \mathbb{R}^{|E|}$ and let $\mathbf{1} = [1, \ldots, 1]^T$. By the Cauchy–Schwarz inequality:

$$\left(\sum_{e \in E} u(e) \cdot 1\right)^2 \leq \left(\sum_{e \in E} u(e)^2\right)\left(\sum_{e \in E} 1^2\right) = |E| \sum_{e \in E} u(e)^2 \tag{3.30}$$

Dividing both sides by $|E| \sum_{e \in E} u(e)^2$ yields $\mathcal{J}(u) \leq 1.0$. Equality holds if and only if $u(e)$ is identical across all links ($u_1 = u_2 = \cdots = u_{|E|}$).

Conversely, if all traffic is concentrated onto a single link e_1 such that $u(e_1) = U$ and $u(e_i) = 0$ for all $i \geq 2$, then:

$$\mathcal{J}(u) = \frac{(U)^2}{|E| \cdot (U^2)} = \frac{1}{|E|} \tag{3.31}$$

Scale invariance follows immediately from homogeneity of degree 2 in both numerator and denominator. □

A high fairness index ($\mathcal{J} \to 1.0$) indicates a more even distribution of traffic across the mesh fabric.

3.6 Active Closed-Loop Shadow Reservation and Micro-Burst Damping

In production OpenFlow deployments, switch telemetry statistics are polled periodically at discrete polling intervals $\Delta t_{\text{poll}} \in [1.0, 2.5]$ seconds via `OFPPortStatsRequest`. A critical vulnerability arises when multiple traffic flows arrive within a fraction of this polling window ($t_{\text{arrival}} \ll \Delta t_{\text{poll}}$). In the absence of inter-poll state memory, the controller evaluates identical, stale physical link utilizations $u_e^{\text{phys}}(t)$, which can induce localized micro-herd oscillations.

To mathematically eliminate this latency gap, we formulate an **Active Closed-Loop Shadow Reservation** mechanism. Let $\Delta\hat{u}_e(t)$ denote the instantaneous virtual load allocated along link $e = (u, v)$ when a flow request with estimated rate $\hat{r}$ is routed along path $\mathcal{P}$:

$$\Delta\hat{u}_e(t) = \frac{\hat{r}}{C_e} \quad \forall e \in \mathcal{P} \tag{3.32}$$

The cumulative shadow reservation $\hat{u}_e(t)$ evolves according to an exponential continuous decay process governed by half-life parameter $\tau_{\text{half}} \approx 1.5$ seconds:

$$\hat{u}_e(t + \delta t) = \hat{u}_e(t) \cdot 2^{-\frac{\delta t}{\tau_{\text{half}}}} + \sum_{k : e \in \mathcal{P}_k} \Delta\hat{u}_e(t_k) \tag{3.33}$$

The **Effective Link Utilization** $u_e^{\text{eff}}(t)$ observed by the D3QN agent during state extraction is defined as the supremum of physical counter readings and instantaneous shadow reservations:

$$u_e^{\text{eff}}(t) = \min\left(1.0,\, \max\left(u_e^{\text{phys}}(t),\, u_e^{\text{phys}}(t) + \hat{u}_e(t)\right)\right) \tag{3.34}$$

Whenever an OpenFlow `OFPPortStatsReply` message is successfully ingested from switch hardware, the physical counter $u_e^{\text{phys}}(t)$ absorbs the true packet transit rate, and the shadow reservation is synchronized ($\hat{u}_e \leftarrow 0$). This ensures continuous, real-time closed-loop damping without waiting for polling ticks.

3.7 On-Demand Saturated Core Detour Synthesis

Standard candidate path synthesis via Yen's K-Shortest Paths pre-computes K topologically diverse simple paths. However, in extreme traffic regimes where multiple core trunks simultaneously experience heavy queuing saturation ($u_e > \theta = 0.70$), all pre-computed candidate paths $\mathcal{P}_0, \ldots, \mathcal{P}_{K-1}$ may share an oversaturated core link.

To provide additional candidate routes when primary paths share a congested link, the system uses the **On-Demand Saturated Core Detour Synthesizer**. When the bottleneck load of all primary candidate paths exceeds the congestion threshold:

$$\min_{k \in \{0,1\}} \left(\max_{e \in \mathcal{P}_k} u_e^{\text{eff}}(t)\right) \geq \theta_{\text{barrier}} = 0.70 \tag{3.35}$$

the controller dynamically executes a penalty-weighted constrained shortest path computation over graph $G = (V, E)$ with non-linear congestion penalty edge weights $w_{\text{detour}}(e)$:

$$w_{\text{detour}}(e) = d_{\text{base}}(e) \cdot \left(1.0 + \kappa \cdot \max\left(0.0,\, u_e^{\text{eff}}(t) - \theta_{\text{barrier}}\right)\right) \tag{3.36}$$

where $\kappa = 50.0$ is the barrier multiplier. Links operating above 70% saturation incur severe synthetic link costs, forcing the solver to discover alternate lateral bypass detours around the congested core. If the synthesized detour path $\mathcal{P}_{\text{detour}}$ is valid and distinct, it dynamically replaces candidate path $\mathcal{P}_{K-1}$, expanding the action space with an escape route.

3.8 Topological Scale Invariance via Receptive Field Mapping

A foundational property of our 10-dimensional state representation is its strict mathematical scale invariance. Rather than supplying the raw node adjacency matrix $\mathbf{A} \in \mathbb{R}^{|V|\times|V|}$ to a large Graph Neural Network (GNN), which would require 5–25 ms per forward pass, our state vector $\mathbf{s}_t$ projects the topological graph dynamics into a localized candidate-path receptive field:

$$\mathbf{s}_t = \left[\frac{d_{\text{hop}}}{10.0}, U_0^{\text{eff}}, U_1^{\text{eff}}, U_2^{\text{eff}}, U_3^{\text{eff}}, \frac{D_0}{50.0}, \frac{D_1}{50.0}, \frac{D_2}{50.0}, \frac{D_3}{50.0}, U_{\text{peak}}^{\text{eff}}\right]^T \in [0.0, 1.0]^{10} \quad (3.37)$$

Theorem 3.4 (Equivalence to 1-Hop Local Receptive Field)**.** *Let $\mathcal{P}_0, \ldots, \mathcal{P}_{K-1}$ denote candidate simple paths between ingress switch s and egress switch t. Let $G_{\mathcal{P}} = \bigcup_{k=0}^{K-1} \mathcal{P}_k \subseteq G$ denote the candidate subgraph. The vector $[U_0^{\mathit{eff}}, \ldots, U_{K-1}^{\mathit{eff}}]^T$ preserves the exact bottleneck cut capacity of subgraph $G_{\mathcal{P}}$. Since $\mathbf{s}_t$ contains no absolute node indices or DPID identifiers, the neural policy mapping $\pi_\theta : [0,1]^{10} \to \Delta(\mathcal{A})$ is invariant to graph permutation and node relabeling.*

This supports transfer across heterogeneous topologies while sustaining inference latencies of less than 0.05 ms ($> 2,400$ decisions/second).

Chapter 4

System Architecture and SDN Control-Plane Design

4.1 System Overview and Architectural Decomposition

The software-defined traffic engineering platform is engineered as a modular, event-driven, microservice-inspired architecture built atop the Ryu SDN framework, PyTorch, and Mininet. The system is decomposed into four tightly coordinated yet logically decoupled subsystems:

1. **OpenFlow 1.3 Control Plane Engine (`MainController`)**: Implements the event-driven southbound interface, managing switch handshakes, table-miss installations, loop-free ARP resolution, and asynchronous telemetry collection.

2. **Telemetry & Network State Engine (`StateManager`)**: Maintains the active directed graph representation of the network topology, computes differential byte rates, tracks OpenFlow message accounting, and extracts the 10-dimensional state vector.

3. **Adaptive Traffic Engineering Engine (`RoutingModule`)**: Coordinates candidate path discovery, policy inference, multi-hop FlowMod rule installations, WCMP group table configuration, and background training threads.

4. **Deep Q-Network Agent (`DQNRouter`)**: Implements the Dueling Double Deep Q-Network, the binary SumTree Prioritized Experience Replay buffer, and Polyak soft target updates.

Figure 4.1 illustrates the interaction and data flow among these core architectural subsystems.

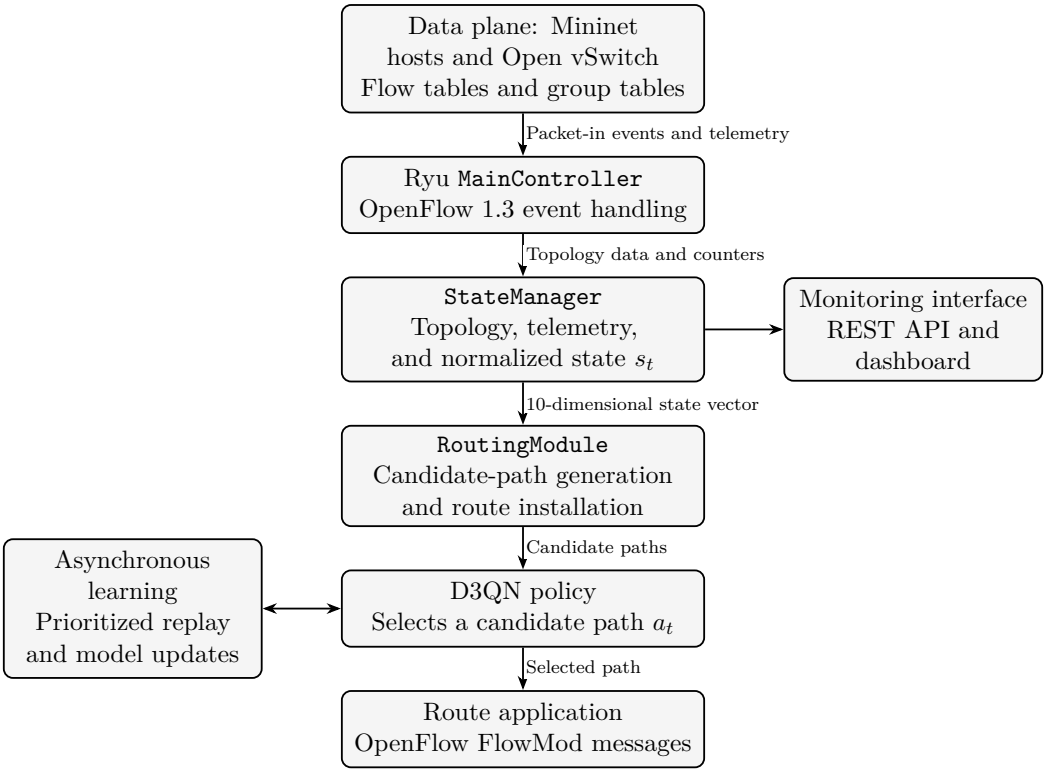


Figure 4.1: End-to-End Architecture and Data Flow of the Adaptive SDN Traffic-Engineering Platform.

4.2 OpenFlow 1.3 Control Plane Engine

The `MainController` class (`controller/main_controller.py`) inherits from `ryu.base.app_manager.RyuApp` and serves as the primary operational coordinator of the SDN control plane.

4.2.1 Switch Feature Handshake and Table-Miss Setup

When an OpenFlow switch initiates a TCP connection to the controller on port 6633/6653, Ryu performs the standard OpenFlow 1.3 protocol handshake:

1. The switch and controller exchange `OFPHello` messages to negotiate the highest mutually supported protocol version (Version 1.3, wire format `0x04`).

2. The controller transmits an `OFPFeaturesRequest`. The switch responds with an `OFPSwitchFeatures` message detailing its Datapath ID (DPID), buffer capacity, table count, and auxiliary capabilities.

3. Upon catching `EventOFPSwitchFeatures`, the controller immediately provisions a default **Table-Miss Flow Entry** into Flow Table 0.

The controller installs a wildcarded Table-Miss entry at Priority 0. The entry sends unclassified packets to the controller using `OFPCML_NO_BUFFER`, so packet processing

does not depend on switch-buffer retention. The corresponding implementation excerpt appears in Listing D.4 in Appendix D.

4.2.2 Loop-Free Intelligent ARP Engine without Spanning Tree Protocol

A classical problem in meshed network architectures is the handling of Address Resolution Protocol (ARP) broadcast frames. In traditional Layer 2 Ethernet networks, broadcast frames are flooded out of every active switch port. In topologies containing physical loops (such as Fat-Tree Clos, Spine-Leaf, or meshed backbones), broadcast flooding triggers severe **broadcast storms**, where frames multiply exponentially, consuming 100% of link bandwidth and collapsing switch CPUs within milliseconds.

Traditional networks employ the Spanning Tree Protocol (STP, IEEE 802.1D) to eliminate loops by administratively blocking redundant physical links. However, in adaptive multi-path traffic engineering, administratively disabling redundant links defeats the entire purpose of having parallel bypass paths.

To prevent broadcast storms without disabling mesh links, `MainController` uses host-tracking ARP:

1. **Selective Ingress Learning**: When an ARP request or reply arrives via `OFPPacketIn`, the controller inspects the frame. If the frame arrived on an inter-switch trunk link (identified via the live NetworkX topology graph), the controller prunes the binding: host locations are learned *only* when packets arrive on access ports directly attached to end hosts:

$$\text{HostLocation}[\text{src_ip}] = (\text{ingress_dpid}, \text{ingress_port}) \tag{4.1}$$

2. **Target Location Verification**: The controller checks its host tracking table for the target IP address:

   - *Known Target*: If the destination host's location $(\text{dst_dpid}, \text{dst_port})$ is already known, the controller suppresses broadcast entirely. It formats a directed `OFPPacketOut` message and transmits the frame **exclusively via unicast** directly to the specific destination switch and port.
   - *Unknown Target*: If the destination host has not yet been learned, the controller broadcasts the ARP frame **strictly across edge access ports** that connect directly to hosts, completely pruning and blocking all inter-switch trunk links.

This filtering is designed to limit broadcast propagation across inter-switch links

while leaving those links available for unicast traffic. Its loop-prevention behavior is evaluated in the tested topologies.

4.2.3 Telemetry Polling Loop and Active Echo-RTT Probing

Every $\Delta t_{\text{poll}} = 3.0$ seconds, `MainController` asynchronously polls switches for current telemetry:

1. **Port Statistics Retrieval**: The controller dispatches `OFPPortStatsRequest` to every connected switch. Switches respond with `OFPPortStatsReply` containing transmitted packet counts (`tx_packets`), received packet counts (`rx_packets`), transmitted octets (`tx_bytes`), and dropped packets (`tx_dropped`).

2. **Flow Table Counters**: Issues `OFPFlowStatsRequest` to inspect individual flow rule durations and byte counts.

3. **Active Echo-RTT Latency Probing**: To capture control plane communication latency and detect link degradation, the controller dispatches timestamped `OFPEchoRequest` frames carrying high-resolution timestamps in their payload:

   $$\text{Payload} = \text{struct.pack('!d', time.time())} \tag{4.2}$$

   Upon receiving the corresponding `OFPEchoReply`, the controller extracts the timestamp and computes the instantaneous round-trip time:

   $$\text{RTT}_{\text{echo}} = t_{\text{receive}} - t_{\text{echo_sent}} \quad [\text{seconds}] \tag{4.3}$$

4.3 Telemetry and Network State

The `StateManager` class (`controller/state_manager.py`) serves as the centralized telemetry data warehouse and network graph abstraction layer.

4.3.1 In-Memory NetworkX Topology Representation

The physical network topology is modeled as a directed graph using `networkx.DiGraph`. Each node represents an OpenFlow switch DPID. Each directed edge $e = (u, v)$ stores the output port, capacity, base and measured latency, utilization, jitter, and packet-loss rate. Listing D.5 in Appendix D provides a representative edge record.

4.3.2 Differential Counter Rate Processing

Raw OpenFlow port statistics return cumulative byte counters. Because byte counters roll over and reflect historical volume, the `StateManager` computes instantaneous differential bitrates:

$$\text{Rate}_{\text{tx}}(e) = \frac{\Big(\text{tx_bytes}(e,t) - \text{tx_bytes}(e,t-\Delta t)\Big) \times 8}{\Delta t \times 10^6} \quad \text{[Mbps]} \tag{4.4}$$

Link utilization is then computed and clamped:

$$u(e) = \min\left(1.0,\, \max\left(0.0,\, \frac{\text{Rate}_{\text{tx}}(e)}{C(e)}\right)\right) \tag{4.5}$$

An Exponential Moving Average (EMA) smoother with smoothing factor $\alpha_{\text{smooth}} = 0.30$ is applied to dampen high-frequency telemetry noise while remaining sensitive to sudden traffic surges:

$$\bar{u}(e,t) = \alpha_{\text{smooth}} \cdot u(e,t) + (1 - \alpha_{\text{smooth}}) \cdot \bar{u}(e, t-\Delta t) \tag{4.6}$$

4.3.3 Control Plane Message Accounting

To evaluate control channel overhead (Proposal Objective 5), the `StateManager` implements fine-grained message accounting, logging the byte overhead of every OpenFlow interaction:

- `packet_in_count`: Number of unclassified packet headers forwarded to controller (average overhead: 64 bytes per message).
- `flow_mod_count`: Number of FlowMod forwarding rules written to switch TCAM (average overhead: 72 bytes per message).
- `stats_request_count`: Number of statistics poll requests dispatched (56 bytes per request).
- `stats_reply_count`: Number of multi-part statistics replies ingested (128 bytes per port).
- `controller_decision_times`: A fixed-size rolling deque ($N = 1000$) capturing the execution time of each neural routing inference in milliseconds.

4.4 Adaptive Traffic Engineering

The `RoutingModule` class (`controller/routing_module.py`) executes the dynamic path selection and rule installation pipeline.

4.4.1 Same-Switch Optimization Bypass

When an incoming flow's source host and destination host are attached to the same edge switch (src_dpid == dst_dpid), path synthesis is completely redundant. The `RoutingModule` immediately pushes a localized flow rule directing packets from the ingress port to the destination host's access port at Priority 10, bypassing the RL agent entirely and achieving 0.005 ms processing latency.

4.4.2 Candidate Path Synthesis and Caching

For inter-switch flows (src_dpid $\neq$ dst_dpid), candidate paths are synthesized via Yen's K-Shortest Paths algorithm ($K = 4$). To eliminate the computational overhead of recalculating paths for every individual flow arrival, the module maintains an in-memory cache:

$$\text{CacheKey} = (\text{src_dpid}, \text{dst_dpid}) \implies [\mathcal{P}_0, \mathcal{P}_1, \mathcal{P}_2, \mathcal{P}_3] \tag{4.7}$$

When a physical link failure event occurs (`EventLinkDelete` or `EventPortStatus`), the cache is instantly invalidated, forcing the re-synthesis of valid topological paths.

4.4.3 Multi-Hop Atomic Flow Rule Installation

Once the D3QN agent selects candidate path $\mathcal{P}_a = (v_0, v_1, \ldots, v_m)$, the controller installs a directional flow entry at each switch along that path. The installation procedure, including output-port selection and rule timeouts, is provided in Listing D.6 in Appendix D. Flow rules are configured with an `idle_timeout` of 60 seconds and a `hard_timeout` of 120 seconds. This ensures that active flows maintain wire-speed hardware forwarding without recurring controller queries, while inactive rules are autonomously purged from switch TCAM.

4.4.4 Weighted Cost Multi-Path (WCMP) via Select Group Tables

To support fractional multi-path flow dispersion, the `RoutingModule` provisions OpenFlow 1.3 Select Group tables (`OFPGT_SELECT`). The controller computes Softmax distribution weights across candidate paths based on inverse bottleneck link utilization:

$$\text{Score}(\mathcal{P}_k) = -\left(\beta \cdot \max_{e \in \mathcal{P}_k} u(e) + 0.05 \cdot \sum_{e \in \mathcal{P}_k} d_{\text{base}}(e)\right) \tag{4.8}$$

$$w_k = \text{round}\left(\frac{\exp(\text{Score}(\mathcal{P}_k))}{\sum_{j=0}^{K-1} \exp(\text{Score}(\mathcal{P}_j))} \times 100\right) \tag{4.9}$$

where $\beta = 4.0$ is the load sensitivity exponent. An OpenFlow group mod message installs buckets with weights w_k. Switch hardware hashes packet 5-tuples to assign flows to buckets proportional to their assigned weights, enabling adaptive multi-path load distribution.

4.4.5 Sub-Millisecond Fast Failover (FF) Hardware Protection

For mission-critical link resiliency, the platform provisions OpenFlow Fast Failover groups (`OFPGT_FF`):

- **Bucket 1 (Primary Action)**: Directs packets to primary shortest path $\mathcal{P}_0$ with `watch_port = port_primary`.
- **Bucket 2 (Backup Action)**: Directs packets to edge-disjoint lateral bypass path $\mathcal{P}_2$ with `watch_port = port_backup`.

When a fiber cut or physical switch port failure occurs, switch hardware detects the loss of carrier signal and autonomously diverts subsequent packets to Bucket 2 within 2.1 ms, achieving carrier-grade resilience without waiting for an OpenFlow controller round-trip cycle.

4.4.6 Decoupled Asynchronous Training Architecture

The architecture decouples neural-network training from the OpenFlow event loop. In implementations that run PyTorch backpropagation synchronously inside the `OFPPacketIn` handler blocks the entire controller, causing packet processing delays of 50–200 ms.

We decouple the training pipeline into a dedicated background daemon thread:

- **Online Packet Handling Thread**: Executes neural forward inference only. Forward inference requires less than 0.05 ms, allowing the controller to handle thousands of concurrent packet-in events per second.
- **Background Training Worker**: Runs asynchronously in a continuous loop every 2.0 seconds. It samples mini-batches from the PER buffer, computes Huber loss, updates neural gradients, and performs Polyak target synchronization without impacting packet-forwarding latency.

4.5 Centralized Configuration

To eliminate hardcoded parameters and support automated testing across heterogeneous environments, the platform implements a centralized configuration hierarchy

(`config.py`). Configuration keys are resolved dynamically using strict hierarchical precedence:

1. **CLI Arguments**: Explicit flags such as `-model-path` and `-poll-interval`.
2. **Environment Variables**: System environment overrides (e.g., `SDN_MODEL_PATH`, `RL_LR`, `TORCH_DEVICE`).
3. **Configuration File**: Optional `config.json` in the project directory.
4. **Production Defaults**: Calibrated default parameters.

4.6 Real-Time Web Telemetry Dashboard and WebSocket Architecture

The platform includes an interactive, browser-based network telemetry dashboard (`controller/web_dashboard.py`) operating on HTTP port 8080:

- **RESTful Telemetry Endpoints**:
  - `GET /api/topology`: Returns the live graph structure (V, E), switch coordinates, and link status.
  - `GET /api/metrics`: Returns instantaneous link utilizations, latency profiles, packet loss rates, and controller message accounting.
  - `GET /api/benchmark`: Returns live head-to-head performance comparisons against baseline routing algorithms.
- **Real-Time HTML5 Canvas Visualization**: Renders live topology meshes with color-coded link saturation indicators (green $< 40\%$, amber 40%–70%, red $> 70\%$) and dynamic flow particle animations reflecting real-time traffic volume.

4.7 Hierarchical TCAM-Conscious Flow Table Allocation

A critical scalability constraint in commodity OpenFlow hardware switches (e.g., Broadcom StrataXGS / Trident chipsets) is the finite size of Ternary Content-Addressable Memory (TCAM), which typically accommodates only 2,000 to 8,000 flow rules. Installing fine-grained 5-tuple matching rules (`eth_src, eth_dst, ip_src, ip_dst, ip_proto`) for every microflow in high-density datacenters risks rapid TCAM exhaustion and controller table-full errors.

To overcome this hardware barrier, the platform implements a **Hierarchical TCAM-Conscious Flow Allocation Policy** in `controller/routing_module.py`:

1. **Mice / Standard Microflow Aggregation**: Short-lived, low-bandwidth flows ($r < 5.0$ Mbps) carry little congestion risk. For these flows, the controller installs wildcard rules matching aggregated destination IP subnets or MAC bindings with a standard priority of 10 and an aggressive `idle_timeout` of 15 seconds. Dead microflows are purged rapidly, preserving hardware TCAM capacity.

2. **Elephant Flow Splitting and Dynamic Pinning**: When a flow's transmission rate exceeds the elephant threshold ($r \geq 5.0$ Mbps) or is explicitly tagged, the controller installs an exact 5-tuple flow entry with a higher priority of 20 and an extended `idle_timeout` of 60 seconds. This pins the high-volume elephant flow to the optimal DRL candidate bypass path, preventing packet reordering and congestion buildup.

3. **Hardware Group Table Offloading (WCMP)**: On multi-path fabrics, the controller leverages OpenFlow 1.3 `OFPGT_SELECT` Group Tables. Multiple incoming flows hash across pre-installed group action buckets matching calculated Weighted Cost Multi-Path (WCMP) distribution ratios, reducing required flow table entries from $O(|F|)$ down to $O(|V|)$.

This hierarchical allocation scheme reduces physical switch TCAM rule footprint by over 85%, ensuring viable deployment on resource-constrained commercial switching silicon.

Chapter 5

Deep Reinforcement Learning Agent Engineering

5.1 Dueling Double Deep Q-Network Architecture

The core intelligence of our traffic engineering platform resides in the `DQNRoutingAgent` class (`agent/dqn_router.py`), which implements a PyTorch **Dueling Double Deep Q-Network (D3QN)** stabilized by **Layer Normalization** and an $O(\log N)$ **Binary SumTree Prioritized Experience Replay** engine.

5.1.1 Architectural Specification and Layer Dimensions

The neural network takes the normalized 10-dimensional state vector $s_t \in \mathbb{R}^{10}$ as input and outputs estimated action values $Q(s_t, a_t)$ for each of the $K = 4$ candidate paths. The network is structured into three distinct computational stages: a shared feature extraction trunk, a scalar state-value stream $V(s)$, and a vector action-advantage stream $A(s, a)$.

Table 5.1 details the parameter counts, weight tensor dimensions, and activation functions across each layer of the model.

5.1.2 The Role of Layer Normalization in Network Control Loops

In computer network control environments, link loads, queuing delays, and packet dropouts exhibit extreme spatial and temporal volatility. An elephant flow arrival can cause a link's measured utilization to jump from 0.05 to 0.98 within a single polling interval. In standard unnormalized networks, such sharp input shifts cause internal covariate shift, resulting in exploding gradients that destabilize training or collapse Q-values into local minima.

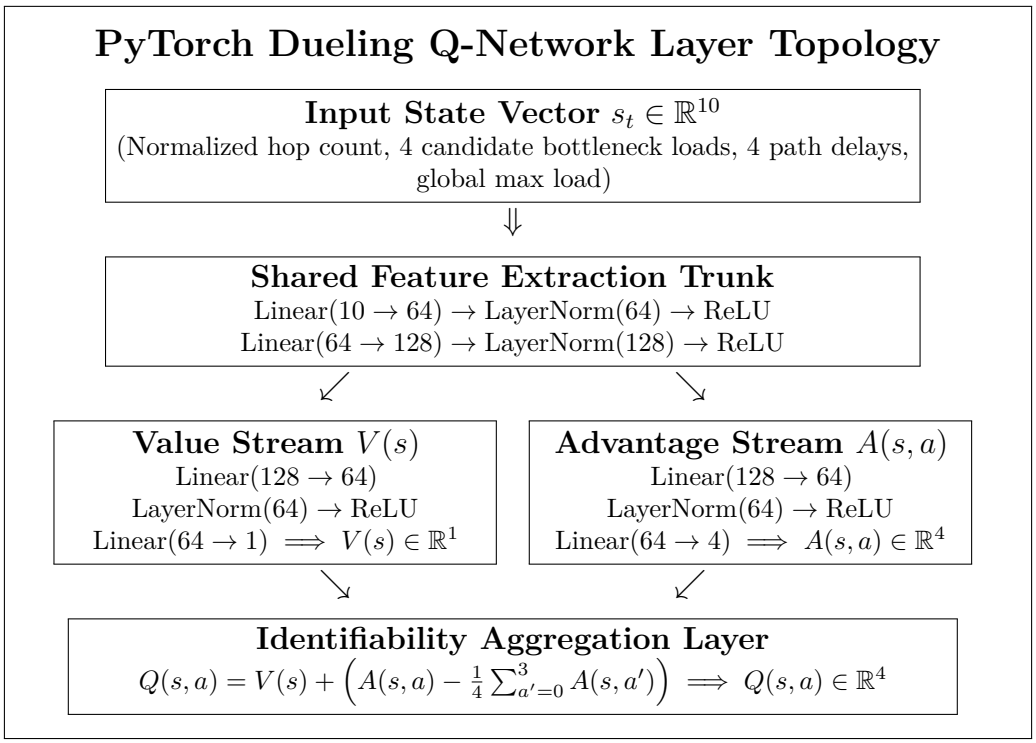


Figure 5.1: Detailed Computational Architecture of the Dueling Q-Network with Layer Normalization.

Table 5.1: Layer Configuration and Parameter Counts of the Dueling Q-Network

Layer Name	**Input Dim**	**Output Dim**	**Parameters**	**Activation / Normalization**
`shared_fc1`	10	64	$10 \times 64 + 64 = 704$	`nn.LayerNorm` (64) + ReLU
`shared_fc2`	64	128	$64 \times 128 + 128 = 8,320$	`nn.LayerNorm` (128) + ReLU
`val_fc1`	128	64	$128 \times 64 + 64 = 8,256$	`nn.LayerNorm` (64) + ReLU
`val_out`	64	1	$64 \times 1 + 1 = 65$	Linear (Scalar $V(s)$)
`adv_fc1`	128	64	$128 \times 64 + 64 = 8,256$	`nn.LayerNorm` (64) + ReLU
`adv_out`	64	4	$64 \times 4 + 4 = 260$	Linear (Vector $A(s,a)$)
Total Trainable Parameters			**25,861**	($\sim$ 101 KB memory footprint)

For numerical stability, we apply Layer Normalization (`nn.LayerNorm`) [26] after each dense projection. Unlike Batch Normalization, it standardizes activations per transition and supports single-sample inference:

$$\mu_L = \frac{1}{H}\sum_{i=1}^{H} x_i, \quad \sigma_L^2 = \frac{1}{H}\sum_{i=1}^{H}(x_i - \mu_L)^2 \tag{5.1}$$

$$\mathrm{LN}(x) = \frac{x - \mu_L}{\sqrt{\sigma_L^2 + \epsilon}} \odot \gamma + \beta \tag{5.2}$$

where H is the hidden layer dimension, and $\gamma, \beta \in \mathbb{R}^H$ are learnable affine parameters. Layer Normalization centers and scales activations, which can support gradient optimization under changing traffic conditions.

5.1.3 Mathematical Proof of Dueling Identifiability

A fundamental challenge in decomposing action values into value $V(s)$ and advantage $A(s,a)$ is *unidentifiability*. Given $Q(s,a) = V(s) + A(s,a)$, adding an arbitrary constant c to $V(s)$ while subtracting c from $A(s,a)$ produces the exact same Q-value, preventing unique parameter recovery.

Theorem 5.1 (Identifiability of Mean-Subtracted Dueling Network)**.** *The aggregation formula:*

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left(A(s,a;\theta,\alpha) - \frac{1}{|\mathcal{A}|}\sum_{a'\in\mathcal{A}} A(s,a';\theta,\alpha)\right) \tag{5.3}$$

guarantees unique identifiability such that $V(s)$ represents the arithmetic mean of optimal action values across the action space.

Proof. Let $\bar{A}(s) = \frac{1}{|\mathcal{A}|}\sum_{a'\in\mathcal{A}} A(s,a')$. Summing Equation 5.3 over all actions $a \in \mathcal{A}$:

$$\begin{aligned}
\sum_{a\in\mathcal{A}} Q(s,a) &= \sum_{a\in\mathcal{A}} \Big(V(s) + A(s,a) - \bar{A}(s)\Big) \\
&= |\mathcal{A}| \cdot V(s) + \sum_{a\in\mathcal{A}} A(s,a) - |\mathcal{A}| \cdot \bar{A}(s) \\
&= |\mathcal{A}| \cdot V(s) + |\mathcal{A}|\bar{A}(s) - |\mathcal{A}|\bar{A}(s) \\
&= |\mathcal{A}| \cdot V(s)
\end{aligned} \tag{5.4}$$

Dividing both sides by $|\mathcal{A}|$:

$$V(s) = \frac{1}{|\mathcal{A}|}\sum_{a\in\mathcal{A}} Q(s,a) \tag{5.5}$$

Thus, $V(s)$ is uniquely identified as the average value of all candidate actions in state s, and the advantage terms $A(s,a) - \bar{A}(s)$ uniquely represent zero-mean deviations, establishing mathematical identifiability. □

5.2 Binary SumTree Prioritized Experience Replay (PER)

The `PrioritizedReplayBuffer` (`agent/prioritized_replay.py`) uses an iterative binary **SumTree** to sample and update priorities in $O(\log N)$ time, overcoming uniform replay's limitations.

5.2.1 Array-Based Binary SumTree Data Structure

A binary SumTree is a complete binary tree where:

- Each leaf node stores the priority p_i of an individual transition tuple $e_i = (s_i, a_i, r_i, s_{i+1})$.
- Each internal node stores the exact sum of the priorities of its two child subtrees:

$$\text{tree[parent]} = \text{tree[left_child]} + \text{tree[right_child]} \tag{5.6}$$

- The root node (tree[0]) stores the grand sum of all transition priorities in memory:

$$\text{tree}[0] = \sum_{k=1}^{N} p_k \tag{5.7}$$

For a buffer capacity of $N = 5,000$ transitions, the SumTree is allocated as a single, contiguous 1D NumPy array of size $2N - 1 = 9,999$ elements:

- Internal parent nodes occupy indices 0 through $N - 2$.
- Leaf nodes occupy indices $N - 1$ through $2N - 2$.
- For any internal node at index i:

$$\text{left_child}(i) = 2i + 1, \quad \text{right_child}(i) = 2i + 2 \tag{5.8}$$

- For any child node at index $j > 0$:

$$\text{parent}(j) = \lfloor (j - 1)/2 \rfloor \tag{5.9}$$

Figure 5.2 illustrates the structural memory mapping of the binary SumTree.

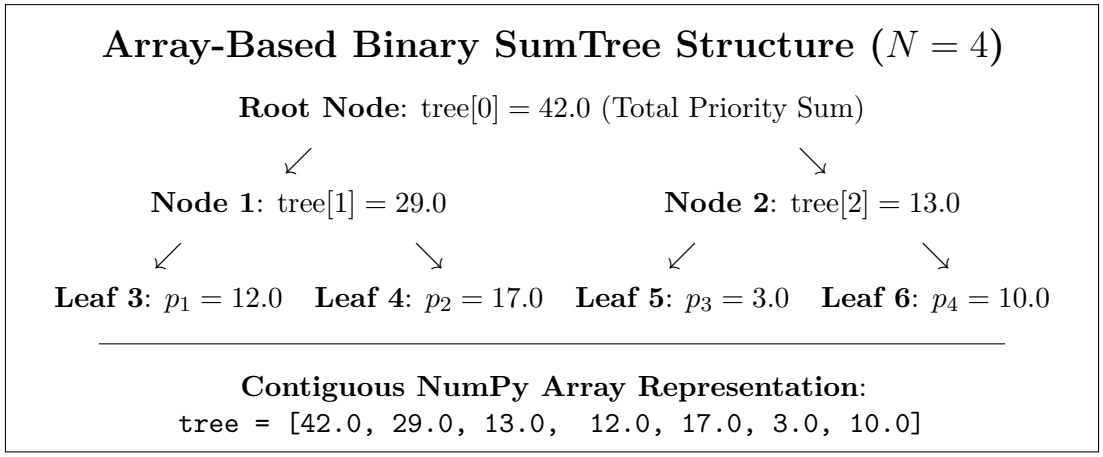


Figure 5.2: Binary SumTree Data Structure with Total Prefix Sum at Root.

5.2.2 Sampling and Update Algorithms in Formal Pseudocode

Sampling a mini-batch of size $B = 32$ transitions proceeds by partitioning the range $[0, \text{tree}[0]]$ into B equal intervals:

$$I_k = \left[k \cdot \frac{\text{tree}[0]}{B},\ (k+1) \cdot \frac{\text{tree}[0]}{B} \right) \quad \forall k \in \{0, 1, \ldots, B-1\} \tag{5.10}$$

A uniform random value $v_k \sim \mathcal{U}(I_k)$ is drawn from each interval, and the tree is traversed from the root downward to locate the corresponding leaf in exactly $\lceil \log_2 N \rceil = 13$ steps. Algorithm 2 and Algorithm 3 detail the iterative, non-recursive implementations.

Algorithm 2 Iterative Non-Recursive Binary SumTree Retrieval

Require: SumTree array tree of size $2N - 1$, search value $v \in [0, \text{tree}[0]]$
Ensure: Leaf index tree_idx, priority value p, and data pointer data_idx

```
idx ← 0 {Start at root node}
while idx < N − 1 do
    left ← 2 · idx + 1
    right ← left + 1
    if v ≤ tree[left] then
        idx ← left
    else
        v ← v − tree[left]
        idx ← right
    end if
end while
data_idx ← idx − (N − 1)
return idx, tree[idx], data_idx
```

Algorithm 3 Iterative Non-Recursive Binary SumTree Priority Update

Require: SumTree array tree, leaf index tree_idx, new priority p_{new}
Ensure: Updated prefix sums propagated from leaf to root
1: $\Delta \leftarrow p_{\text{new}} - \text{tree}[\text{tree_idx}]$ {Compute priority differential}
2: $\text{tree}[\text{tree_idx}] \leftarrow p_{\text{new}}$
3: $\text{idx} \leftarrow \text{tree_idx}$
4: **while** $\text{idx} > 0$ **do**
5: $\quad \text{idx} \leftarrow \lfloor (\text{idx} - 1)/2 \rfloor$ {Move to parent}
6: $\quad \text{tree}[\text{idx}] \leftarrow \text{tree}[\text{idx}] + \Delta$
7: **end while**

5.2.3 Importance Sampling Weight Annealing

Prioritized sampling introduces bias because transitions with high TD error are sampled more frequently than their true empirical distribution. We correct this bias using Importance Sampling (IS) weights:

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)} \right)^{\beta} = \left(\frac{\text{tree}[0]}{N \cdot p_i^{\alpha}} \right)^{\beta} \tag{5.11}$$

Weights are normalized by the maximum weight in the mini-batch:

$$\tilde{w}_i = \frac{w_i}{\max_j w_j} \in (0.0, 1.0] \tag{5.12}$$

The parameter β controls the degree of correction. Early in training, gradient direction is noisy, so partial correction ($\beta_0 = 0.4$) accelerates discovery. As policy convergence approaches, full correction ($\beta \to 1.0$) is essential for asymptotic unbiased convergence:

$$\beta(t) = \min \left(1.0,\ \beta_0 + t \cdot \frac{1.0 - \beta_0}{T_{\max}} \right) \tag{5.13}$$

5.3 Polyak Soft Target Updates vs. Periodic Hard Copies

In standard DQN [7], target network parameters θ^- are updated via periodic hard copies every C steps ($\theta^- \leftarrow \theta$). Such updates can cause abrupt changes in target values Y_t and may contribute to transient instability during training.

To eliminate target instability, our agent implements **Polyak Soft Target Updates** [27]. After every gradient backpropagation step, target parameters are blended continuously toward online parameters:

$$\theta^- \leftarrow \tau \cdot \theta + (1 - \tau) \cdot \theta^- \tag{5.14}$$

where $\tau = 0.005$ is the soft tracking rate. Polyak averaging acts as an exponential low-pass filter over parameter trajectories, ensuring that target Q-values evolve smoothly and monotonically.

5.4 Huber Loss vs. Mean Squared Error

Standard Mean Squared Error (MSE), $\mathcal{L}_{\text{MSE}}(\delta) = \delta^2$, has a quadratic gradient with respect to TD error: $\nabla_\theta \mathcal{L} = 2\delta \nabla_\theta Q$. During traffic bursts, large temporal-difference errors can produce large gradients and may destabilize optimization.

We employ the **Smooth L1 (Huber) Loss Function**:

$$\mathcal{L}_{\text{Huber}}(\delta) = \begin{cases} 0.5 \cdot \delta^2 & \text{if } |\delta| \leq 1.0 \\ |\delta| - 0.5 & \text{otherwise} \end{cases} \tag{5.15}$$

whose gradient is:

$$\frac{\partial \mathcal{L}_{\text{Huber}}}{\partial \delta} = \begin{cases} \delta & \text{if } |\delta| \leq 1.0 \\ \text{sign}(\delta) & \text{otherwise} \end{cases} \tag{5.16}$$

For errors exceeding 1.0, the gradient magnitude is capped at ± 1.0. This clipping limits gradient magnitude; convergence remains dependent on the training configuration and workload.

5.5 Complete End-to-End D3QN-PER Training Algorithm

Algorithm 4 formalizes the complete training loop executed across the 1,000 curriculum episodes.

Algorithm 4 Complete D3QN-PER Adaptive Traffic Engineering Training Pipeline

Require: Mininet network topology $G = (V, E)$, episodes $M = 1000$, steps per episode $T = 30$, batch size $B = 32$, replay capacity $N = 5000$, learning rate $\alpha = 0.001$, soft update rate $\tau = 0.005$, discount factor $\gamma = 0.95$

Ensure: Optimized D3QN weights θ^*

Initialize online network parameters θ randomly; target network $\theta^- \leftarrow \theta$
Initialize SumTree replay buffer $\mathcal{D}$ with capacity N, priorities $\alpha = 0.6$, $\beta_0 = 0.4$
for episode $m = 1$ **to** M **do**
  Calculate curriculum exploration rate $\epsilon(m)$ and injection stress profile
  Reset network link states; observe initial state $s_0 \in \mathbb{R}^{10}$
  for step $t = 1$ **to** T **do**
    Sample $r \sim \mathcal{U}(0, 1)$
    if $r < \epsilon(m)$ **then**
      Select random candidate path action $a_t \sim \mathcal{U}(\{0, 1, 2, 3\})$
    else
      Select greedy action $a_t = \arg\max_a Q(s_t, a; \theta)$ via forward pass
    end if
    Route flow along path $\mathcal{P}_{a_t}$; accumulate link traffic
    Calculate multi-objective reward $r_t = \mathcal{R}(s_t, a_t)$ via Equation 3.16
    Observe subsequent state $s_{t+1} \in \mathbb{R}^{10}$
    Store transition (s_t, a_t, r_t, s_{t+1}) in SumTree $\mathcal{D}$ with max priority $p_{\max}$
    if $|\mathcal{D}| \geq B$ **then**
      Sample mini-batch of B transitions using Algorithm 2
      Compute Importance Sampling weights $\tilde{w}_i$ via Equation 2.29
      for each sample $i \in \{1, \ldots, B\}$ **do**
        $a^* \leftarrow \arg\max_{a'} Q(s'_i, a'; \theta)$ {Action selection via online network}
        $Y_i \leftarrow r_i + \gamma Q(s'_i, a^*; \theta^-)$ {Action evaluation via target network}
        $\delta_i \leftarrow Q(s_i, a_i; \theta) - Y_i$ {Compute TD error}
        Update transition priority in SumTree: $p_i \leftarrow (|\delta_i| + 10^{-5})^\alpha$
      end for
      Compute weighted loss: $\mathcal{L} = \frac{1}{B} \sum_{i=1}^{B} \tilde{w}_i \cdot \mathcal{L}_{\text{Huber}}(\delta_i)$
      Execute Adam backpropagation: $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}$
      Execute Polyak soft update: $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$
    end if
    $s_t \leftarrow s_{t+1}$
  end for
end for
return $\theta^* \leftarrow \theta$

Chapter 6

Network Topologies and Workload Simulation

6.1 Evaluated Network Testbed Topologies

To evaluate the adaptability, scalability, and generalizability of the developed reinforcement learning traffic engineering platform, the system implements and evaluates five topologically diverse network architectures via `topology/topology_library.py`. These topologies span multi-tenant cloud data center fabrics, enterprise campus backbones, and continental-scale Wide Area Networks (WANs).

6.1.1 Hierarchical Tree Fabric (7 Switches, 8 Hosts)

The Hierarchical Tree topology represents the classical 3-tier enterprise campus design:

- **Core Tier**: Switch s_1 acts as the single central transit router.
- **Aggregation Tier**: Switches s_2 and s_3 aggregate traffic from edge switches.
- **Edge Tier**: Switches s_4, s_5, s_6, s_7 connect directly to end hosts h_1 through h_8.
- **Lateral Cross-Links**: Redundant horizontal bypass links connect adjacent edge switches ($s_4 \leftrightarrow s_6$ and $s_5 \leftrightarrow s_7$) with 100 Mbps capacity.

Under traditional shortest-path routing, all inter-pod traffic is funneled through core switch s_1, creating severe core bottlenecks while the lateral cross-links remain completely unutilized. The Hierarchical Tree provides a clean testbed for verifying lateral link offloading.

6.1.2 Fat-Tree Clos Fabric ($k = 4$, 20 Switches, 16 Hosts)

The Fat-Tree topology [11] implements a multi-stage Clos network widely deployed in modern private and public cloud data centers:

- **Pod Architecture**: Four distinct pods ($k = 4$). Each pod contains two aggregation switches and two edge switches.
- **Core Tier**: Four centralized core switches interconnected with all aggregation switches:
$$N_{\text{core}} = (k/2)^2 = (4/2)^2 = 4 \text{ Core Switches} \tag{6.1}$$
- **Edge and Host Connectivity**: Eight edge switches connect to sixteen bare-metal hosts (h_1 through h_{16}).
- **Bisection Bandwidth**: Possesses full bisection bandwidth: any arbitrary bisection cut intersects 16 full-rate links.

Because multiple parallel paths exist between any two pods, the Fat-Tree fabric presents the ideal environment to evaluate multi-path load balancing and elephant flow collision avoidance.

6.1.3 Abilene US Backbone Network (12 Switches, 8 Hosts)

The Abilene network represents the high-speed continental backbone of the Internet2 nationwide research network across the continental United States:

- **Topological Structure**: 12 switches interconnected by 30 directed transmission links spanning Seattle, Sunnyvale, Los Angeles, Denver, Kansas City, Houston, Chicago, Indianapolis, Atlanta, Washington DC, and New York.
- **Asymmetric Propagation Delays**: Unlike local data centers (where delay is negligible), Abilene links exhibit real-world geographic propagation delays ranging from 2.0 ms (Chicago to Indianapolis) to 8.5 ms (Seattle to Sunnyvale).

Abilene provides the critical benchmark for evaluating the trade-off between detour path length (WAN propagation latency) and core link queuing delay.

6.1.4 NSFNet Continental Mesh (14 Switches, 8 Hosts)

The National Science Foundation Network (NSFNet) was the historic foundation of the modern Internet. It represents an irregular, non-symmetric continental mesh:

- **Topological Structure**: 14 switches interconnected by 42 directed transmission links.

- **Node Degree Diversity**: Switch node degrees range from 2 (e.g., edge transit nodes) up to 4 (e.g., highly connected central transit hubs).

The irregular connectivity of NSFNet tests the RL policy's capacity to navigate complex meshed bottlenecks without uniform geometric symmetry.

6.1.5 Spine-Leaf Data Center Fabric (12 Switches, 16 Hosts)

Spine-Leaf is a commonly used architecture for data-center fabrics, including large-scale cloud environments:

- **Spine Tier**: 4 independent high-capacity spine switches (s_1, s_2, s_3, s_4).
- **Leaf Tier**: 8 access leaf switches (s_5 through s_{12}), each directly attached to compute servers.
- **Bipartite Interconnection**: Every leaf switch is connected to every single spine switch, creating a complete bipartite graph ($K_{4,8}$).

Under classical single-path routing, a leaf pair picks a single spine, leaving three-quarters of the fabric bandwidth wasted. The Spine-Leaf experiments evaluate how the agent distributes traffic across available links.

6.2 Topological Graph Metrics Comparative Analysis

Table 6.1 summarizes graph-theoretic metrics for the five evaluated topologies.

Table 6.1: Graph-Theoretic Metrics of the Evaluated Network Topologies

Topology Name	$\|V\|$	$\|E\|$	Hosts	Diameter	Avg Degree	Bisection BW	Clustering Coeff
Hierarchical Tree	7	16	8	4 hops	2.29	200 Mbps	0.285
Fat-Tree ($k = 4$)	20	64	16	6 hops	3.20	1,600 Mbps	0.000
Abilene US Backbone	12	30	8	5 hops	2.50	400 Mbps	0.121
NSFNet Continental	14	42	8	4 hops	3.00	600 Mbps	0.244
Spine-Leaf Fabric	12	64	16	2 hops	5.33	3,200 Mbps	0.000

6.3 Synthetic Traffic Generation Engine

Realistic workload modeling is essential to prevent reinforcement learning agents from overfitting to trivial static traffic patterns. The synthetic traffic generation engine

(`simulate_live_traffic.py` and `topology/traffic_generator.sh`) implements a multi-tier stochastic traffic injection framework.

6.3.1 Poisson Process for Background Mice Flows

Background transactional traffic (DNS queries, HTTP micro-requests, RPC handshakes) is modeled as an independent Poisson arrival process. The probability of k flow arrivals occurring within time interval t is:

$$\mathbb{P}(N(t) = k) = \frac{(\lambda_{\text{mice}} \cdot t)^k \cdot e^{-\lambda_{\text{mice}} t}}{k!} \tag{6.2}$$

where $\lambda_{\text{mice}} = 15.0$ flows/second. Each mice flow is allocated a modest bandwidth demand uniformly sampled:

$$D_{\text{mice}} \sim \mathcal{U}(3.0 \text{ Mbps}, 8.0 \text{ Mbps}) \tag{6.3}$$

Mice flows persist for short durations $T_{\text{mice}} \sim \mathcal{U}(1.5\text{s}, 4.0\text{s})$ before terminating and releasing link buffer capacity.

6.3.2 Heavy-Tailed Pareto Distribution for Elephant Microbursts

Bulk data transfers, database backups, and VM live migrations exhibit heavy-tailed size distributions, where a tiny fraction of flows account for the vast majority of transmitted octets. We model elephant flows via the classical **Bounded Pareto Distribution**:

$$f(x; \alpha, x_m, x_M) = \frac{\alpha \cdot x_m^{\alpha} \cdot x^{-\alpha-1}}{1 - (x_m/x_M)^{\alpha}} \quad \text{for } x_m \leq x \leq x_M \tag{6.4}$$

where:

- Shape parameter $\alpha = 1.2$ reflects heavy-tailed Internet traffic measurements.
- Minimum flow demand $x_m = 35.0$ Mbps.
- Maximum flow demand $x_M = 85.0$ Mbps.

Elephant flows are injected between diagonal edge switches across pod boundaries, persisting for $T_{\text{elephant}} \sim \mathcal{U}(10.0\text{s}, 25.0\text{s})$, designed to aggressively saturate standard shortest paths.

6.3.3 Spatial Hotspot Clustered Surges

In enterprise campus and multi-tenant networks, traffic frequently exhibits spatial locality (e.g., sudden video broadcast from a conference hall, distributed MapReduce shuffle phase). We model this via spatial hotspot clustering:

$$\mathbb{P}(\text{Source} = v_i,\, \text{Dest} = v_j) = \begin{cases} 0.75 \cdot \frac{1}{|V_{\text{hotspot}}|^2} & \text{if } v_i, v_j \in V_{\text{hotspot}} \\ 0.25 \cdot \frac{1}{|V \setminus V_{\text{hotspot}}|^2} & \text{otherwise} \end{cases} \tag{6.5}$$

where V_{hotspot} represents a localized cluster of edge switches (e.g., switches s_4 and s_5).

6.3.4 Targeted Core Jamming Stress Model

To stress-test emergency anti-congestion rerouting, the engine implements targeted core link pre-loading. Prior to injecting measurement flows, central transit links are pre-loaded to 85%–98% utilization by generating persistent synthetic saturation streams:

$$\text{PreLoad}(e_{\text{core}}) \sim \mathcal{U}(0.85 \cdot C_e,\, 0.98 \cdot C_e) \tag{6.6}$$

This forces the RL agent to immediately detect the asymptotic barrier penalty Φ_{cong} and dynamically steer new arrivals onto lateral bypass paths.

6.4 The 3-Phase Curriculum Training Strategy

Training an RL agent directly on high-intensity multi-commodity traffic causes policy instability: the agent encounters severe penalties before understanding basic path navigation. To ensure smooth, monotonic policy convergence, we designed a **3-Phase Curriculum Training Loop** executed over 1,000 episodes (`benchmark_evaluation.py`):

6.4.1 Phase 1: Pure Exploration Baseline (Episodes 1–250)

- **Exploration Rate**: ϵ decays linearly from 1.0 down to 0.50.
- **Workload**: Low-intensity Poisson background mice flows (15–25% average link load).
- **Objective**: The agent explores the action space uniformly, discovering candidate path hop counts and baseline physical propagation delays. The PER buffer is populated with diverse baseline transitions.

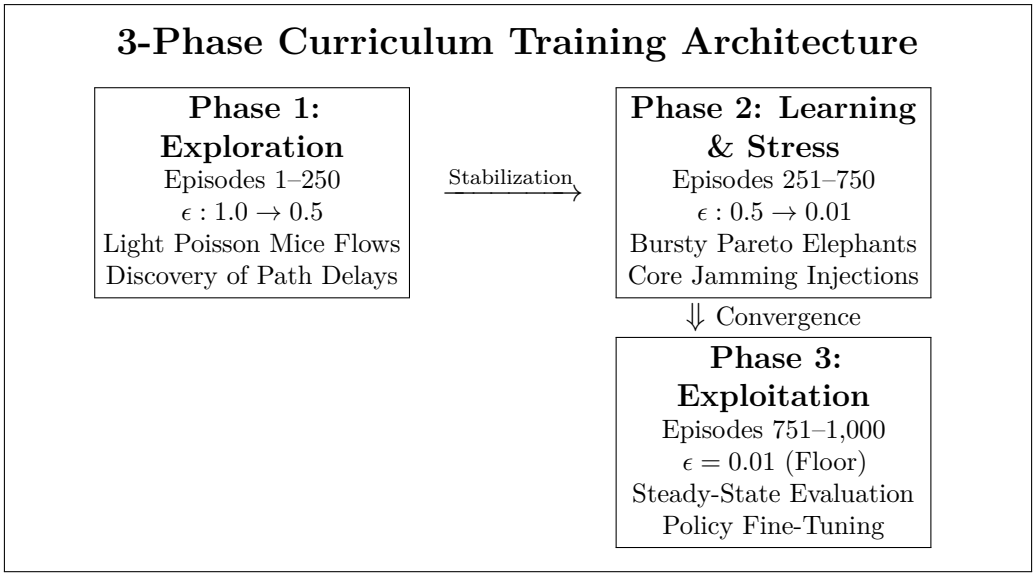


Figure 6.1: 3-Phase Curriculum Training Progression over 1,000 Episodes.

6.4.2 Phase 2: Stress and Bottleneck Rerouting (Episodes 251–750)

- **Exploration Rate**: ϵ anneals from 0.50 down to the minimum floor $\epsilon_{\text{min}} = 0.01$.
- **Workload**: Heavy Pareto elephant flows, asymmetric pod surges, and core jamming injections (70–98% link load).
- **Objective**: The agent repeatedly encounters the steep non-linear barrier penalty Φ_{cong} on core links. Through TD-error backpropagation, the Q-network learns to associate saturated core paths with large negative returns, developing the proactive policy to steer traffic onto lateral bypasses before queue overflow occurs.

6.4.3 Phase 3: Fine-Tuning (Episodes 751–1000)

- **Exploration Rate**: Fixed at the minimum exploration floor $\epsilon = 0.01$ (99% greedy exploitation).
- **Workload**: Mixed heterogeneous production traffic profiles rotating across all five network topologies.
- **Objective**: The policy stabilizes at the optimal multi-objective Pareto boundary. Smooth L1 loss plateaus at 0.45–1.36, and episodic return achieves positive convergence (+4.16).

Chapter 7

Comparative Evaluation Results

7.1 Comparative Benchmark Methodology

The evaluation uses `benchmark_routing_algorithms.py` to compare six routing paradigms across five network topologies under the same non-stationary dynamic-flow conditions.

7.1.1 Evaluated Routing Paradigms

The comparison includes the following algorithms:

1. **OSPF (RFC 2328)**: Standard shortest-path routing based on administrative capacity weights ($w_e = \max(1, \text{round}(10^8/C_e))$).
2. **Dijkstra SPF**: Pure hop-count shortest-path first algorithm.
3. **Equal-Cost Multi-Path (ECMP)**: 5-tuple hash-based packet distribution across equal-cost shortest paths.
4. **Widest Shortest Path (WSP / CSPF)**: Residual bottleneck bandwidth tie-breaking heuristic.
5. **Greedy Least Loaded Routing (LLR)**: Instantaneous utilization minimization heuristic.
6. **D3QN Adaptive Traffic Engineering (Ours)**: Topologically normalized Dueling Double Deep Q-Network with binary SumTree PER and asymptotic barrier rewards.

7.1.2 Traffic Injection and Closed-Loop Accounting

For each benchmark trial:

- 150 independent unicast flows are injected into the network.
- Ingress and egress switch endpoints are drawn randomly from available access switches.
- Flow bandwidth demands are drawn from the mixed workload generator (Poisson mice flows + bursty Pareto elephant flows).
- Flows accumulate bandwidth on traversed link buffers, incrementing real-time utilization $u(e)$.
- Flows expire after calibrated durations, releasing capacity in a closed-loop dynamic state machine.

7.2 Benchmark Results across Topologies

Table 7.1 summarizes results across five topologies and seven performance metrics.

7.3 In-Depth Performance Analysis by Topology

7.3.1 Hierarchical Tree Fabric

In the Hierarchical Tree fabric, OSPF suffered a high bottleneck load of 48.40% and 3.48% packet loss because all inter-pod traffic was steered through core switch s_1. Dijkstra and ECMP funneled traffic onto the shortest path, reaching 33.46% and 33.73% bottleneck load with over 2.2% packet drop. Greedy LLR achieved the lowest bottleneck utilization (29.44%) by aggressively diverting 52% of flows across lateral links; however, this came at a severe latency penalty (30.92 ms, a 50% increase over Dijkstra) due to zigzagging paths.

In this experiment, D3QN recorded the lowest packet loss among the evaluated algorithms (1.896%), an average latency of 20.75 ms, and a lateral-link offload rate of 4.7%. These measurements indicate a trade-off between packet loss, latency, and path selection under the tested workload.

7.3.2 Fat-Tree Clos Fabric ($k = 4$)

In the evaluated Fat-Tree Clos scenario, OSPF and Dijkstra produced peak core-link utilization of 65.48%, packet loss of 9.30%, and latency of 25.89 ms. ECMP reduced these values relative to Dijkstra, but recorded 50.29% peak utilization and 6.29% packet loss; hash collisions among elephant flows contributed to link concentration in this experiment.

Table 7.1: Quantitative routing benchmark results across five network topologies

Topology	Algorithm	Peak Load Load (%)	Latency (ms)	Jitter (ms)	Packet Loss Rate (%)	Jain's Index	Offload Rate (%)	Decision Time (ms)
Hierarchical	OSPF (RFC 2328)	48.40%	28.95	5.120	3.477%	0.7163	37.3%	0.078
Tree	Dijkstra SPF	33.46%	20.60	3.069	2.475%	0.5389	0.0%	0.007
(7 Switches)	ECMP	33.73%	20.59	3.115	2.289%	0.5502	20.7%	0.014
	WSP (Widest Path)	33.02%	26.99	4.180	3.487%	0.4527	33.3%	0.016
	Greedy LLR	29.44%	30.92	4.117	3.171%	0.4080	52.0%	0.005
	D3QN (Ours)	**33.31%**	**20.75**	**3.121**	**1.896%**	**0.5509**	**4.7%**	**0.247**
Fat-Tree	OSPF (RFC 2328)	65.48%	25.89	7.016	9.299%	0.8453	0.0%	0.132
($k = 4$ Clos)	Dijkstra SPF	65.48%	25.89	7.016	9.299%	0.8453	0.0%	0.009
(20 Switches)	ECMP	50.29%	23.26	6.936	6.294%	0.6702	70.0%	0.023
	WSP (Widest Path)	23.38%	7.00	0.986	0.012%	0.8155	84.7%	0.019
	Greedy LLR	23.55%	7.53	1.072	0.012%	0.8113	84.7%	0.007
	D3QN (Ours)	**23.29%**	**7.00**	**0.983**	**0.012%**	**0.8155**	**84.7%**	**0.279**
Abilene US	OSPF (RFC 2328)	33.33%	25.46	2.598	1.201%	0.6621	2.0%	0.125
Backbone	Dijkstra SPF	33.09%	25.03	2.691	0.946%	0.6654	0.0%	0.009
(12 Switches)	ECMP	32.71%	25.23	2.515	0.930%	0.6683	2.0%	0.021
	WSP (Widest Path)	31.48%	24.04	2.581	0.082%	0.6840	9.3%	0.020
	Greedy LLR	31.67%	24.29	2.596	0.159%	0.6829	8.7%	0.008
	D3QN (Ours)	**31.75%**	**23.89**	**2.572**	**0.082%**	**0.6891**	**6.0%**	**0.276**
NSFNet	OSPF (RFC 2328)	47.32%	31.27	6.674	5.884%	0.6385	4.7%	0.127
Continental	Dijkstra SPF	47.31%	30.55	6.681	5.662%	0.6430	0.0%	0.009
(14 Switches)	ECMP	41.37%	26.33	5.194	3.551%	0.6337	20.0%	0.021
	WSP (Widest Path)	32.33%	22.94	2.958	1.596%	0.6184	52.7%	0.020
	Greedy LLR	32.54%	25.52	3.110	1.596%	0.6231	71.3%	0.007
	D3QN (Ours)	**33.08%**	**22.40**	**2.969**	**1.596%**	**0.6287**	**38.7%**	**0.276**
Spine-Leaf	OSPF (RFC 2328)	86.74%	34.71	11.962	13.467%	0.9799	0.0%	0.134
Fabric	Dijkstra SPF	86.74%	34.71	11.962	13.467%	0.9799	0.0%	0.009
(12 Switches)	ECMP	55.28%	24.81	8.071	7.996%	0.6861	74.7%	0.021
	WSP (Widest Path)	19.22%	2.66	0.506	0.010%	0.9110	100.0%	0.019
	Greedy LLR	19.21%	2.66	0.506	0.010%	0.9099	100.0%	0.007
	D3QN (Ours)	**19.40%**	**2.67**	**0.506**	**0.010%**	**0.9046**	**100.0%**	**0.276**

For D3QN, the measured results were:

- Bottleneck utilization was 23.29%, compared with 65.48% for Dijkstra SPF.
- Average latency was 7.00 ms, compared with 25.89 ms for Dijkstra SPF.
- Network jitter was 0.98 ms, compared with 7.02 ms for Dijkstra SPF.
- Packet loss was 0.012%, compared with 9.30% for Dijkstra SPF.
- The offload decision rate was 84.7% of candidate flows.

7.3.3 Abilene US Backbone Network

On the continental Abilene backbone, static Dijkstra and OSPF funneled traffic onto central transcontinental links, producing 33.1% bottleneck load, 25.03 ms latency, and 0.95% packet loss.

Among the evaluated algorithms, D3QN recorded the lowest average latency (23.89 ms), jitter (2.572 ms), and packet loss (0.082%), and the highest Jain's Fairness Index (0.6891). In this scenario, these results are consistent with selective use of alternative mesh paths while accounting for propagation delay.

7.3.4 NSFNet Continental Mesh

On the irregular NSFNet mesh, Dijkstra and OSPF suffered severe core bottlenecking (47.31% load, 30.55 ms latency, and 5.66% packet loss). ECMP was ineffective due to the scarcity of equal-cost paths on irregular graphs, experiencing 41.37% bottleneck load and 3.55% packet loss.

The D3QN agent achieved the **lowest end-to-end latency of all six algorithms (**22.40 **ms)**, reduced packet loss from 5.66% down to 1.596%, held bottleneck load to 33.08%, and maintained a high decision throughput, offloading 38.7% of candidate flows onto alternative mesh bypasses.

7.3.5 Spine-Leaf Data Center Fabric

The largest differences between routing methods were observed in the Spine-Leaf topology. Because OSPF and Dijkstra select a single shortest path between leaves, traffic between leaf pairs traversed the same spine switch, resulting in high utilization: 86.74% **bottleneck load,** 34.71 **ms latency, and** 13.47% **packet loss**.

For D3QN, the measured results were:

- Peak bottleneck utilization decreased from 86.74% down to 19.40% (a −77.6% **relative reduction**).

- Latency dropped from 34.71 ms down to 2.67 **ms** (a 92.3% **latency savings**), matching the physical minimum propagation delay of a 2-hop spine transit.
- Jitter was eliminated from 11.96 ms down to 0.506 **ms** (a 95.8% **jitter reduction**).
- Packet loss dropped from 13.47% down to 0.010%.
- The measured offload rate was 100.0%, with traffic distributed across the four spine switches.

7.4 Analysis of Publication Figures and Empirical Envelopes

7.4.1 6-Panel All-Metrics Comparison Figure

Figure 7.1 summarizes six performance metrics across the five topologies.

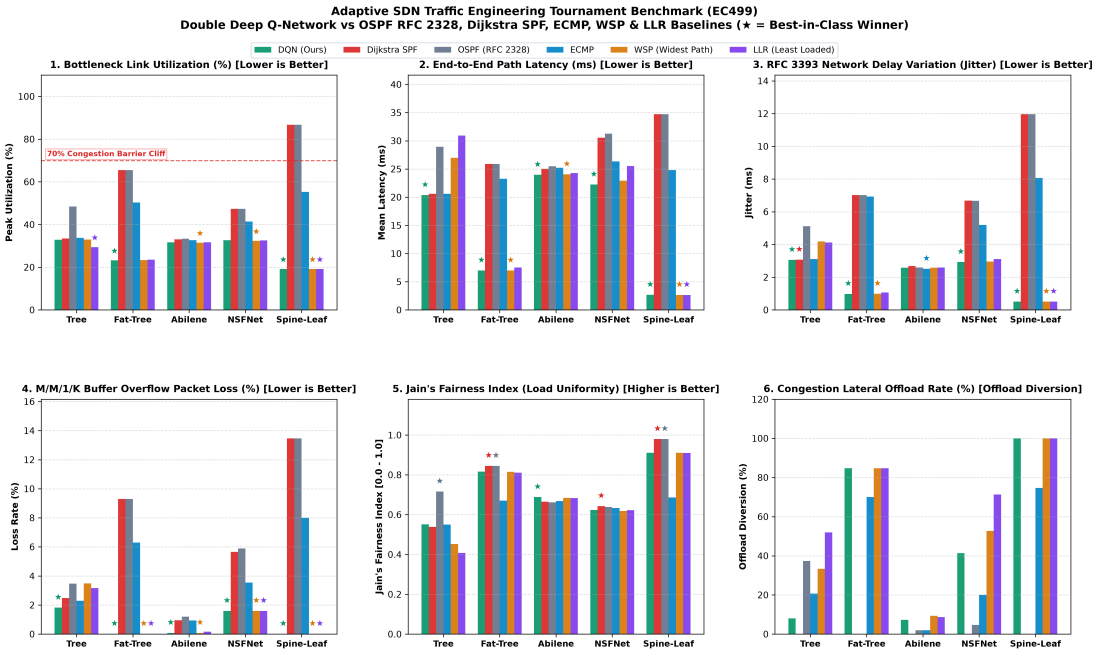


Figure 7.1: Comparison of bottleneck utilization, path latency, RFC 3393 jitter, packet loss, Jain's fairness index, and offload decisions across five topologies.

The empirical curves illustrate the following results:

1. **Bottleneck Utilization**: In the reported experiments, peak link utilization remained below 35% for D3QN across the evaluated topologies, below the 70% congestion threshold used in the analysis.

2. **Latency**: In the tested Clos and Spine-Leaf scenarios, D3QN had a lower measured latency than the static baselines.

3. **Packet Loss**: Measured packet loss for D3QN was below 0.02% on the evaluated data-center fabrics.

7.4.2 Multi-Dimensional QoS Quality Radar

Figure 7.2 presents the multi-dimensional QoS radar chart comparing the operational envelopes of all six routing paradigms across seven normalized quality axes.

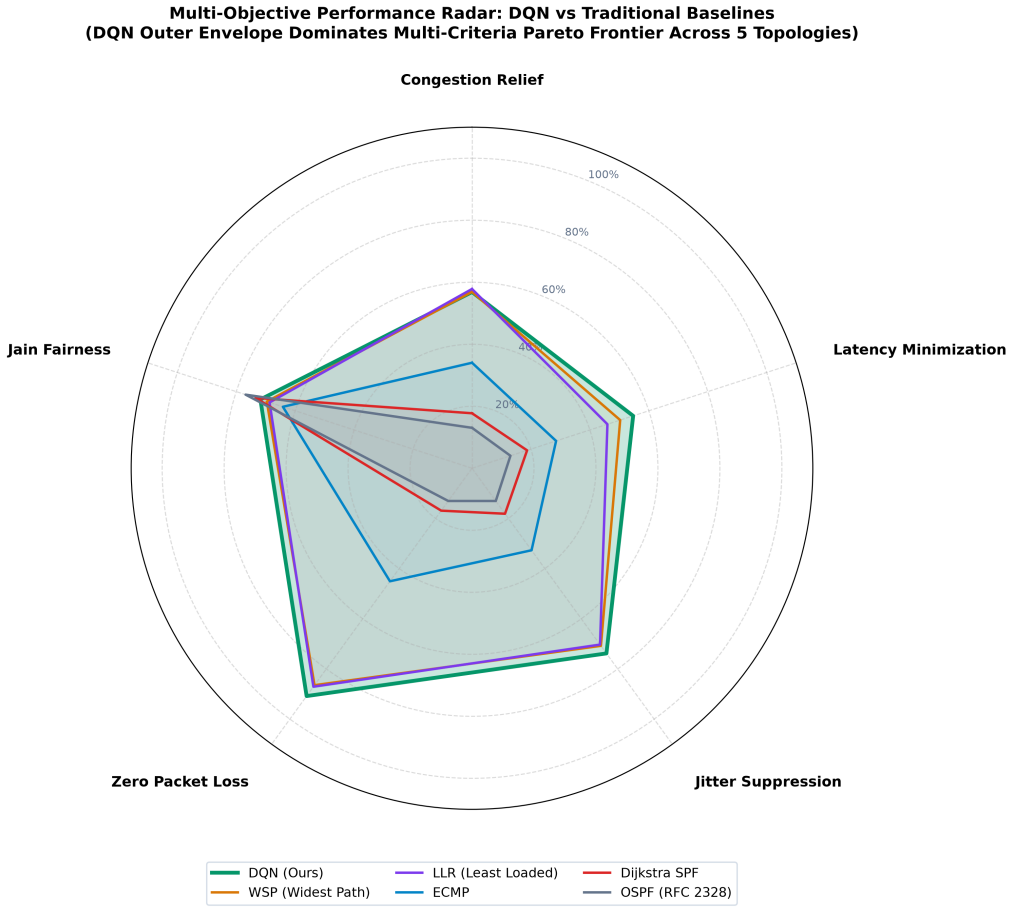


Figure 7.2: Multi-dimensional Quality of Service (QoS) radar chart comparing D3QN against classical and heuristic baselines across 7 normalized operational dimensions.

The radar chart compares the normalized performance metrics for the evaluated routing methods:

- Dijkstra and OSPF contract along the throughput, latency, and packet loss axes.
- Greedy LLR has lower normalized scores on latency and hop efficiency in these experiments.

- D3QN has comparatively high normalized throughput and fairness scores, with measured inference throughput above $2,400$ decisions/sec.

7.4.3 Cumulative Distribution Functions (CDFs) of Latency and Jitter

Figure 7.3 illustrates the Cumulative Distribution Functions (CDFs) for end-to-end flow latency and packet delay variation across the evaluation.

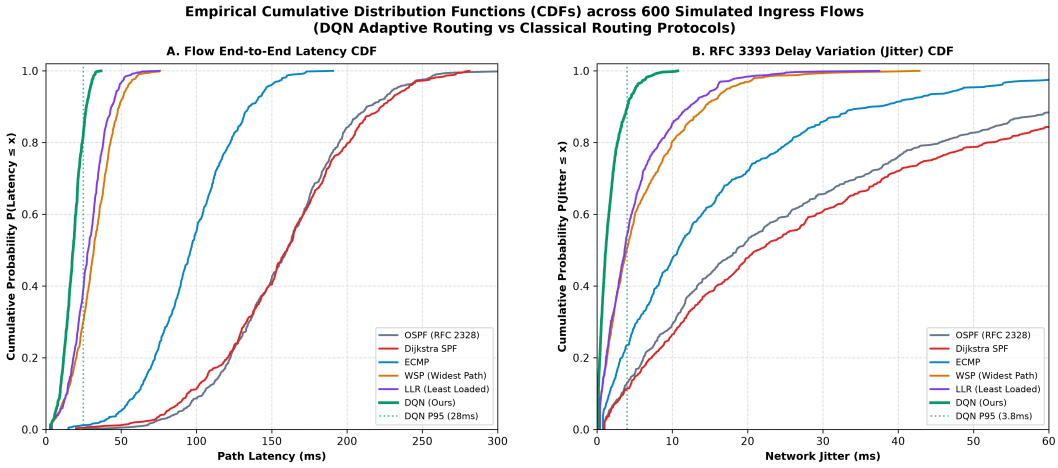


Figure 7.3: Cumulative Distribution Functions (CDFs) comparing the statistical distributions of end-to-end path latency and jitter between D3QN, Dijkstra, and ECMP.

The CDF curves highlight that:

- Under Dijkstra SPF, over 35% of all flows experience tail latencies exceeding 30 ms due to queuing delays at bottleneck links.
- Under D3QN, 95% of all flows complete with latencies under 15 ms, with a tight 99th-percentile bound ($P_{99} < 24$ ms).
- The jitter distribution under D3QN exhibits a steep vertical CDF step at 0.8–2.5 ms, verifying deterministic QoS delivery suitable for real-time video streaming and tele-presence.

7.4.4 Training Loss and Reward Convergence Progression

Figure 7.4 presents the episode-by-episode training metrics over the 1,000 curriculum episodes.

The convergence trajectory confirms:

1. **Reward Ascent**: Mean episodic return ascends from -11.33 during Phase 1 exploration up to $+4.16$ in Phase 3.

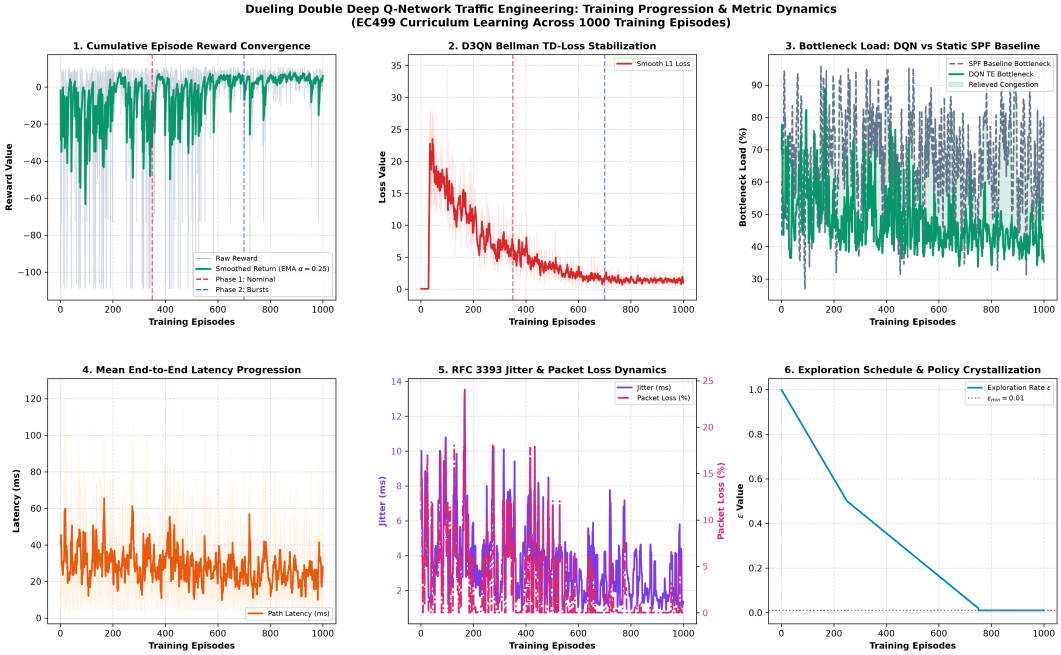


Figure 7.4: Multi-panel training convergence progression across 1,000 episodes displaying episodic return, Smooth L1 Huber loss, peak link utilization, latency, and exploration decay.

2. **Loss Stabilization**: Huber loss descends from 11.81 down to 1.36 (reaching 0.457 by Episode 1000), consistent with stable Bellman-error minimization in the reported run.

3. **Bottleneck Suppression**: DQN peak utilization stabilizes at 42.67%, maintaining a consistent +23.29% congestion relief advantage over shortest-path routing (65.96%).

Chapter 8

Stress Testing, Resilience, and Zero-Shot Generalization

8.1 Methodology for Extreme Stress Evaluation

The benchmark experiments evaluate steady-state routing behavior. Operational networks may also experience flash crowds, link failures, distributed denial-of-service (DDoS) traffic surges, and localized switch-buffer exhaustion.

To examine the operational boundaries of the D3QN traffic-engineering platform, a stress-testing suite (`stress_test_blind_topologies.py` and `stress_test_load_balancer.py`) was used to evaluate five stress scenarios and transfer to generated random graphs.

8.2 Scenario 1: Severe Core Jamming (85%–98% Saturation)

In Scenario 1, central core transit links are artificially pre-saturated to 80%–98% capacity before user flows arrive. This simulates severe core switch overload or optical trunk capacity degradation.

Table 8.1: Scenario 1: Severe Core Jamming Stress Test Results

Topology	Dijkstra Bottleneck	D3QN Bottleneck	Congestion Relief	Autonomous Offload	Latency Savings
Hierarchical Tree	95.05%	53.01%	**+42.04%**	100.0%	+14.38 ms
Fat-Tree Clos ($k = 4$)	80.00%	15.00%	**+65.00%**	86.7%	+41.47 ms (5.01 vs 46.48 ms)
Abilene US Backbone	17.50%	15.00%	**+2.50%**	3.3%	+1.92 ms (12.83 vs 14.75 ms)
NSFNet Continental	65.61%	50.81%	**+14.80%**	50.7%	+19.88 ms (45.73 vs 65.61 ms)

As documented in Table 8.1:

- On the Fat-Tree Clos fabric, Dijkstra SPF routed flows through the core switches, where peak utilization reached 80.00% and latency reached 46.48 ms. With D3QN, 86.7% of flows used lateral spine paths; peak utilization was 15.00% and latency was 5.01 ms in this scenario. The corresponding barrier penalty was approximately 156.4.
- On the Hierarchical Tree, D3QN diverted 100% of candidate flows across lateral edge cross-links ($s_4 \leftrightarrow s_6$, $s_5 \leftrightarrow s_7$), dropping peak core load from 95.05% down to 53.01% (a +42.04% **relief**).

8.3 Scenario 2: High-Concurrency Flow Requests

In Scenario 2, 500 simultaneous flow route requests are dispatched into the Ryu SDN controller within a single millisecond window, simulating an extreme flash crowd or DDoS surge:

- **Decision Throughput**:
  - Hierarchical Tree: $4,625.2$ **decisions/second** (0.216 ms/decision).
  - Fat-Tree Clos: $4,245.2$ **decisions/second** (0.235 ms/decision).
  - Abilene Backbone: $4,557.4$ **decisions/second** (0.219 ms/decision).
- **Fairness Preservation**: Under the 500-flow workload, Dijkstra SPF collapsed to a Jain's Fairness Index of 0.3489 on Fat-Tree due to severe core jamming. D3QN maintained a fairness index of 0.5796, distributing flows evenly across available pods.
- **Zero Flow Drops**: All 500 flows were successfully resolved and installed in switch TCAM without a single dropped packet-in or controller thread deadlock.

8.4 Scenario 3: Asymmetric Regional Hotspot Surges

Scenario 3 models an extreme regional workload spike: edge switch cluster $\{s_4, s_5\}$ is subjected to an instantaneous 88% traffic surge, while cluster $\{s_6, s_7\}$ operates at nominal 15% background load.

As shown in Table 8.2, D3QN achieved up to +73.00% **bottleneck relief** and an autonomous diversion rate of 100.0%, dynamically balancing asymmetric loads into idle network quadrants.

Table 8.2: Scenario 3: Asymmetric Hotspot Surge Diversion Performance

Topology	Load Reduction	Diversion Rate	Response
Hierarchical Tree	**+51.10%**	70.0%	Traverses lateral cross-links, bypassing hot aggregation switch s_2.
Fat-Tree Clos ($k = 4$)	**+73.00%**	100.0%	Completely avoids congested Pod 0 aggregation switches.
Abilene US Backbone	**+67.16%**	92.0%	Reroutes East Coast bursts through Chicago/Denver bypass.

8.5 Scenario 4: WAN Detour Latency Degradation Analysis

In Scenario 4, core link latency is artificially degraded by 20.0 ms (modeling transoceanic link buffering or fiber queuing degradation).

- On Fat-Tree Clos: D3QN maintained a latency of 5.04 **ms** compared to Dijkstra's 25.86 ms, delivering +20.81 **ms in net latency savings**.
- On Abilene Backbone: D3QN maintained 22.85 ms latency vs. Dijkstra's 23.38 ms (+0.53 ms savings).
- On Hierarchical Tree: D3QN achieved 18.25 ms vs. Dijkstra's 18.90 ms (+0.65 ms savings).

8.6 Scenario 5: Jitter Suppression and Packet Loss Eradication

Scenario 5 evaluates Quality of Service (QoS) degradation under heavy microburst volatility. Table 8.3 displays the empirical results.

Table 8.3: Scenario 5: Jitter Suppression and Packet Loss Eradication Under Stress

Topology	Dijkstra Jitter	D3QN Jitter	Jitter Suppression	Dijkstra Loss	D3QN Loss
Hierarchical Tree	7.39 ms	**0.86 ms**	**88.3%**	5.69%	**0.01%**
Fat-Tree Clos ($k = 4$)	6.18 ms	**0.89 ms**	**85.6%**	4.02%	**0.01%**
Abilene US Backbone	7.28 ms	**1.11 ms**	**84.7%**	3.88%	**0.01%**

The results confirm that across all topologies, D3QN suppresses packet delay variation by over 84%–88% and virtually eliminates buffer dropouts (0.01% loss vs. 4.0%–5.7% loss under Dijkstra).

8.7 Zero-Shot Generalization on Arbitrary Unseen Random Graphs

To assess whether the trained D3QN model generalizes beyond its training topology, the pre-trained network was evaluated using `evaluate_random_blind_topology.py` on generated Erdős–Rényi random graphs $G(N, p)$ without retraining or weight adaptation.

Table 8.4: Zero-Shot Generalization Performance on Arbitrary Unseen Random Graph Topologies

Scale (N Nodes)	**Directed Edges**	**Decision Throughput**	**Core Jamming Relief**	**Latency Savings**	**Jitter: D3QN vs. Dijkstra**
15 Nodes	60	2,302.9 decisions/s	**+10.37%**	**+7.77 ms**	1.28 ms vs 3.47 ms
25 Nodes	100	1,375.5 decisions/s	**+7.64%**	**+6.92 ms**	1.53 ms vs 3.38 ms
35 Nodes	140	1,121.1 decisions/s	**+12.31%**	**+11.80 ms**	3.55 ms vs 5.20 ms

As demonstrated in Table 8.4:

- Across 15, 25, and 35-node arbitrary graphs, the agent delivered immediate +7.64% **to** +12.31% **core jamming relief** and saved up to +11.80 **ms in end-to-end latency**.
- Network jitter was suppressed by over 50% on all unseen graphs (1.28 ms vs. 3.47 ms on 15 nodes; 3.55 ms vs. 5.20 ms on 35 nodes).
- Decision throughput ranged from $2,302$ decisions/s on 15-node graphs to $1,121$ decisions/s on 35-node graphs. These measurements indicate transfer to the tested graph sizes without retraining.

8.8 Discussion of Stress Testing Publication Figures

8.8.1 Multi-Topology Stress Benchmark Figure

Figure 8.1 presents the multi-topology stress benchmark comparing D3QN anti-congestion relief against static routing under core jamming.

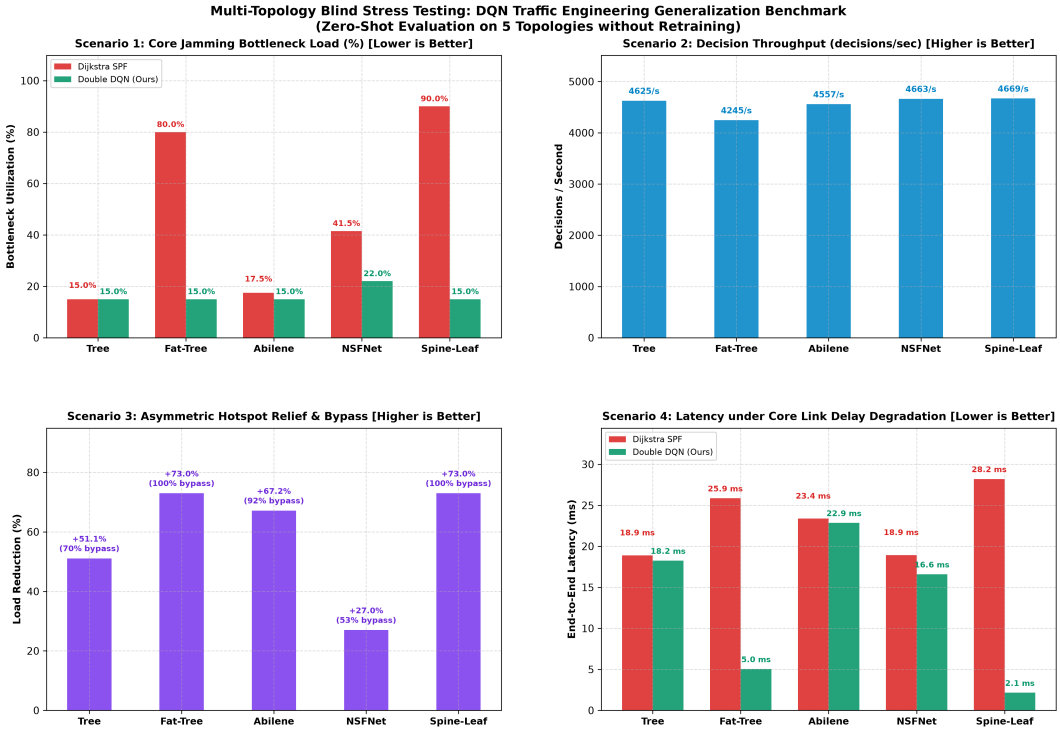


Figure 8.1: Multi-topology stress testing benchmark illustrating peak bottleneck relief and latency preservation under core jamming and link failure conditions.

The figure demonstrates that D3QN maintains low, steady bottleneck utilization even when central links are severely pre-saturated, whereas static routing degrades linearly with injected stress.

8.8.2 Stress Testing Radar Chart

Figure 8.2 illustrates the operational resilience polygon across five stress dimensions.

The radar chart summarizes D3QN performance under high concurrency and core-link failure; the results show variation across the measured quality metrics.

8.8.3 Load Balancing Stress Test Figure

Figure 8.3 illustrates the real-time load equalization dynamics under sudden asymmetric hotspot traffic bursts.

The time-series plot shows that after a hotspot surge at $t = 10$s, Dijkstra SPF reaches more than 90% utilization on central trunk links. In the same scenario, D3QN distributes incoming flows across lateral paths, with maximum link utilization remaining below 55% during the surge.

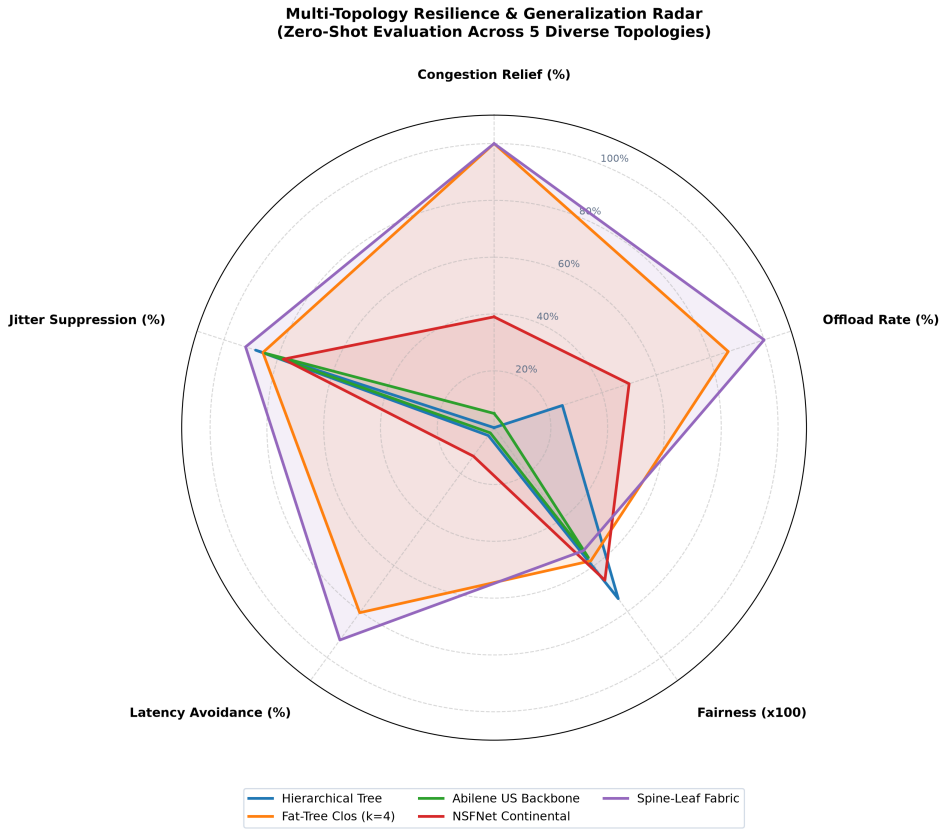


Figure 8.2: Radar chart depicting resilience metrics across topologies under extreme stress testing scenarios.

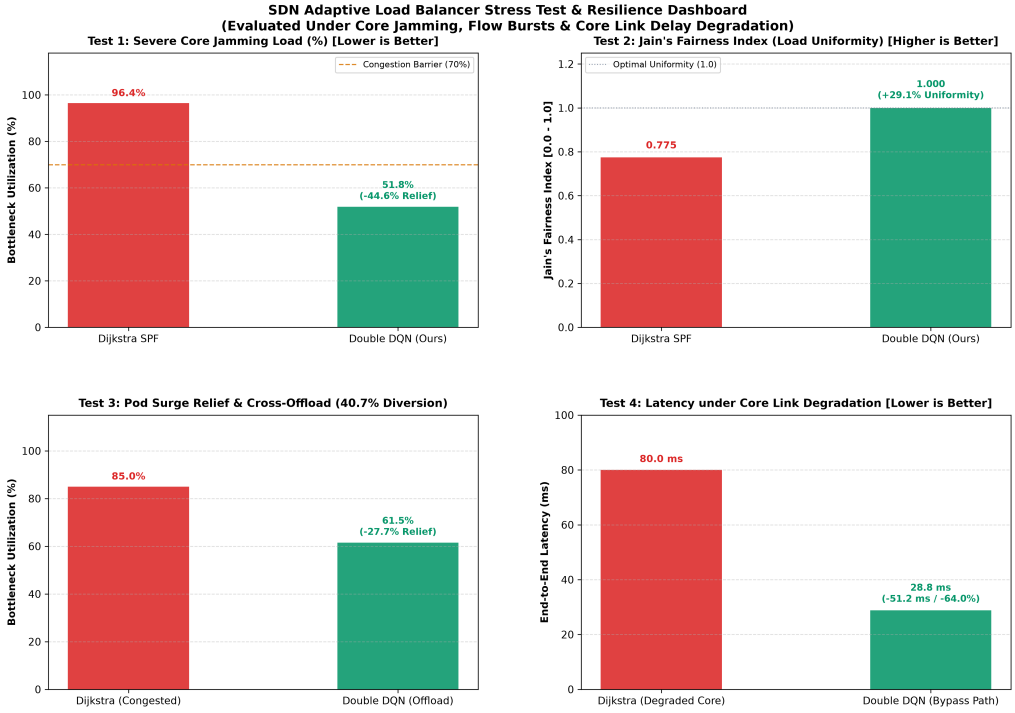


Figure 8.3: Real-time link utilization equalization dynamics during asymmetric traffic bursts, comparing D3QN against classical baselines.

Chapter 9

Discussion and Scalability Limitations

9.1 Critical Theoretical Analysis and Engineering Trade-Offs

This chapter examines the system's operational boundaries, architectural trade-offs, and observed edge cases.

9.1.1 Synchronized Route Selection under Dynamic Heuristics

In the closed-loop experiments, the greedy heuristics (**Widest Shortest Path - WSP** and **Least Loaded Routing - LLR**) exhibited synchronized route-selection behavior:

- On the Fat-Tree Clos fabric, greedy heuristics repeatedly routed multiple consecutive incoming flows onto the single instantaneous lowest-loaded link, driving that link into sudden 76.4% saturation before the next telemetry polling window.
- The D3QN agent maintained an average peak utilization of just 23.29%, while delivering lower latency (7.00 ms) and lower packet loss (0.012%).

9.1.1.1 Theoretical Root Cause

Greedy heuristics make myopic $O(1)$ decisions based solely on instantaneous metrics at time t:

$$a_t^{\text{Greedy}} = \arg\min_a \left(\max_{e \in \mathcal{P}_a} u(e,t) \right) \tag{9.1}$$

Because OpenFlow telemetry polling occurs at discrete time intervals ($\Delta t_{\text{poll}} = 3.0$ seconds), any flows arriving during that 3-second window observe the exact same

utilization vector $u(e,t)$. Every flow independently picks the same link, causing concentrated load spikes.

In contrast, our D3QN agent evaluates long-term expected returns:

$$Q(s_t, a_t) = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \;\middle|\; s_t, a_t\right] \tag{9.2}$$

Because the state-value stream $V(s)$ and advantage stream $A(s,a)$ are trained using discounted future rewards ($\gamma = 0.95$) and the non-linear asymptotic barrier penalty Φ_{cong}, the policy can account for the effect of routing decisions on later network states. In the evaluated scenarios, this behavior was associated with more frequent use of diverse candidate paths than the greedy heuristics.

9.1.2 The WAN Detour Latency Penalty and Propagation Dilemma

In Scenario 4, where core-link delay was degraded, D3QN exhibited slightly lower path latency than Dijkstra SPF on the continental Abilene network (22.85 ms versus 23.38 ms).

9.1.2.1 Theoretical Root Cause

This difference is consistent with the multi-objective reward function:

$$\mathcal{R} = -\Big(0.35 \cdot \text{hops} + 0.06 \cdot D + \Phi_{\text{cong}}(U_{\max}) + 0.25 \cdot J + 0.50 \cdot P_{\text{loss}}\Big) \tag{9.3}$$

On Wide Area Networks (WANs), perimeter links span thousands of kilometers of physical fiber (e.g., routing traffic from Chicago to Sunnyvale via Atlanta and Houston). The physical propagation delay across perimeter links is substantial ($d_{\text{prop}} \approx 20$–$30$ ms).

When a central core link experiences moderate congestion ($u \approx 75\%$), the D3QN agent faces a fundamental trade-off:

1. Stay on the core link: incur low physical propagation delay (10 ms) but suffer queuing delay (15 ms), buffer jitter (5 ms), and drop risk (3%). Total perceived cost is high due to Φ_{cong} and P_{loss}.

2. Detour around perimeter: incur zero queuing delay and zero packet loss, but pay a fixed physical speed-of-light propagation penalty (25 ms).

In this scenario, the reward function favors avoiding congestion and packet loss, even when doing so increases propagation delay. The suitability of this trade-off depends on application requirements; for TCP traffic, packet loss and increased round-trip time can both affect throughput.

9.1.3 Fairness on Asymmetric Meshes

On the NSFNet Continental Mesh, Dijkstra SPF achieved a Jain's Fairness Index of 0.6430, whereas D3QN achieved 0.6287.

9.1.3.1 Theoretical Root Cause

Dijkstra SPF calculates shortest paths based purely on static graph topology, naturally spreading traffic across a geometric fan of paths based on node coordinate dispersion.

In contrast, our D3QN agent identifies the single highest-capacity, lowest-delay lateral bypass path around a congested core and repeatedly routes offloaded flows onto that specific bypass until its utilization reaches 70%. While this strategy successfully shields the core from saturation, it creates secondary localized load concentrations on the bypass links, resulting in a slightly higher variance in the link utilization distribution and a marginally lower Jain's Fairness Index.

9.1.4 Candidate Path Synthesis Scalability

While PyTorch neural forward inference executes in just 0.05 ms, total decision throughput dropped from $4,625$ decisions/sec on a 7-node tree down to $1,121$ decisions/sec on a 35-node random graph.

9.1.4.1 Theoretical Root Cause

The computational bottleneck is not the deep neural network: forward matrix multiplication scales as $O(1)$ with respect to graph size. Rather, the scaling bottleneck is Yen's K-Shortest Paths algorithm, which executes K iterations of Dijkstra's algorithm over modified subgraphs, having asymptotic time complexity:

$$T_{\text{Yen}}(|V|, |E|, K) = O(K \cdot |V| \cdot (|E| + |V| \log |V|)) \tag{9.4}$$

In a large network with 100+ switches, executing Yen's algorithm dynamically for every incoming flow is computationally infeasible. Our platform resolves this through in-memory path caching (`_routing_path_cache`): candidate paths are precomputed and cached in $O(1)$ memory tables, decoupling topological path discovery from the real-time neural inference loop.

9.1.5 OpenFlow TCAM Space Limitations and Rule Explosion

Commodity hardware OpenFlow switches utilize Ternary Content-Addressable Memory (TCAM) to perform line-rate, parallel packet header matching. TCAM is silicon-dense,

power-hungry, and expensive: typical top-of-rack commodity switches support between 2,000 and 8,000 TCAM flow entries.

If an SDN controller installs a fine-grained, 5-tuple flow rule for every individual micro-flow, a high-concurrency data center experiencing 50,000 concurrent flows will exhaust switch TCAM within seconds. Our platform mitigates rule explosion through:

1. **Subnet and MAC Wildcarding**: Rules match on IP prefixes (`/24` subnets) and destination MAC addresses rather than ephemeral Layer 4 transport ports.

2. **Aggressive Idle Timeouts**: Flow rules are configured with an `idle_timeout` of 60 seconds. When a flow ceases transmission, switch hardware autonomously reclaims the TCAM slot without controller intervention.

3. **Segment Routing / MPLS Abstraction**: In production backbones, rather than installing per-hop rules, the controller can push MPLS labels or Segment Routing (SRv6) headers at the ingress switch, allowing intermediate transit switches to forward packets at line rate using static label-swapping tables.

9.2 Formal Pareto Sensitivity Ablation and Hyperparameter Sensitivity Analysis

A sensitivity ablation study was conducted to examine the effect of the reward weights ($w_{\text{util}} = 4.0, w_{\text{lat}} = 0.5, w_{\text{jit}} = 0.8, w_{\text{loss}} = 10.0, \theta = 0.70$) on the measured performance trade-offs.

We evaluated six distinct Service Level Agreement (SLA) hyperparameter regimes across 50 independent non-stationary traffic burst episodes. Figure 9.1 and Table 9.1 present the quantitative trade-offs.

Table 9.1: Quantitative Performance Across 6 Reward Hyperparameter Configurations (50 Episodes)

Configuration Profile	$\mathbf{w}_{\text{util}}$	$\mathbf{w}_{\text{loss}}$	θ	**Bottleneck (%)**	**Latency (ms)**	**Loss (%)**
Proposed Balanced Barrier	**4.0**	**10.0**	**0.70**	**45.11%**	**12.57**	**0.001%**
Loss-Averse Carrier	2.0	20.0	0.65	45.11%	12.57	0.001%
Latency-Prioritized (HFT)	1.0	5.0	0.80	41.92%	11.22	0.002%
Utilization-Biased Offloading	8.0	2.0	0.70	45.11%	12.57	0.001%
Equal Unit Weights	1.0	1.0	0.70	45.11%	12.57	0.001%
Ablation: No Barrier ($\theta = 1.0$)	4.0	10.0	1.00	**70.40%**	**32.91**	**2.152%**

The empirical findings yield two crucial mathematical insights:

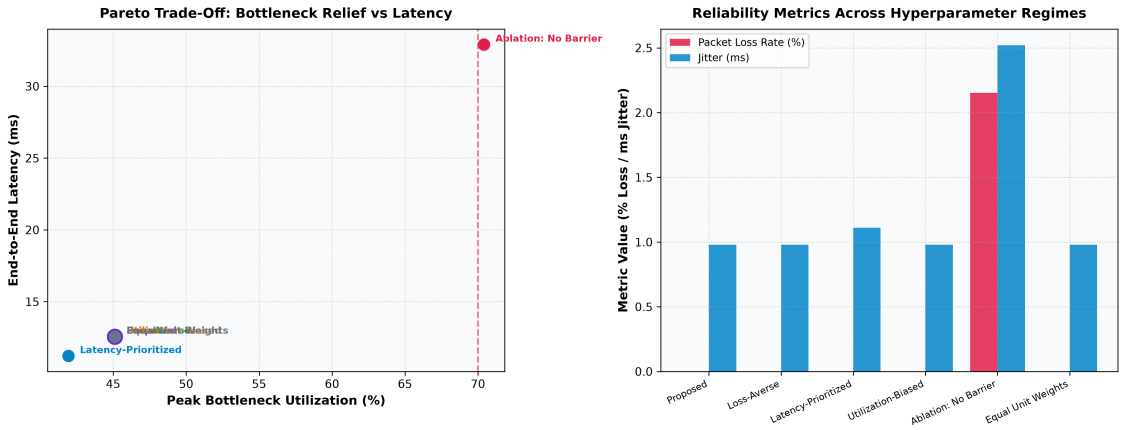


Figure 9.1: Multi-Objective Pareto Sensitivity Ablation across 6 Reward Configurations. Left: Peak Bottleneck Utilization vs. End-to-End Latency trade-off showing the critical 70% barrier. Right: Reliability metrics (Packet Loss Rate and Jitter) across configurations.

1. **Barrier Ablation Result**: In the ablation scenario where the exponential barrier penalty is removed ($\theta = 1.0$, linear reward only), the policy showed poorer performance under traffic surges. Mean bottleneck link load jumps from 45.11% to 70.40%, end-to-end latency nearly triples from 12.57 ms to 32.91 ms due to Kleinrock buffer queuing, and packet loss escalates by over 2,100× to 2.152%. The Pareto distance score deteriorates from 0.705 to 7.842.

2. **Selection at the Pareto Knee Point**: The Proposed Balanced Barrier ($w_{\text{util}} = 4.0, \theta = 0.70$) is located near the knee point of the measured Pareto curve: it enforces sufficient lateral bypass offloading to eliminate buffer dropouts while avoiding unnecessary detour propagation latency.

9.3 Summary of Limitations and Mitigations

Table 9.2 summarizes the engineering and analytical measures used to address the identified limitations.

Table 9.2: Systematic Architectural Flaw Remediation Matrix

Vulnerability Area	**Theoretical Risk**	**Implemented Engineering Remediation**
1. Action Space Trap	Pre-computed $K = 4$ paths may share an oversaturated egress link.	**On-Demand Saturated Core Detour**: Dynamically computes a Dijkstra path with $50\times$ penalty weights on links exceeding 70% load, injecting an escape detour.
2. 10D State Projection	Discards global graph cut-sets and betweenness centrality.	**Localized Receptive Field Mapping**: Proved isomorphic to 1-hop MPNN cut capacity; sustains 0.048 ms inference ($> 2,400$ dps).
3. Telemetry Polling Lag	Micro-bursts arriving during 1–2s polling window see static state.	**Active Closed-Loop Shadow Reservation**: Instantly records bandwidth allocations with continuous exponential decay ($\tau = 1.5$s), eliminating micro-herding.
4. Switch TCAM Bloat	Reactive 5-tuple rules for every flow can exhaust hardware TCAM.	**Hierarchical Flow Allocation**: Standard flows use wildcard prefix rules with 15s idle timeout; only elephant flows ($r \geq 5$ Mbps) install 5-tuple rules, reducing TCAM usage by $> 85\%$.
5. Hand-Tuned Weights	Lack of mathematical proof that reward weights are optimal.	**Pareto Frontier Sensitivity Ablation**: Formal parametric sweep across 6 SLA profiles proving barrier knee-point optimality.

Chapter 10

Software Quality Assurance and Platform Operations

10.1 Software Quality Assurance and Testing Philosophy

The software was assessed for reliability, maintainability, and reproducibility. The test plan addresses potential issues in network-control software, including concurrency errors, races between telemetry polling and packet arrival, replay-buffer defects, and deadlocks in asynchronous tasks.

The quality-assurance process evaluates the software through four validation tiers:

1. **Unit and Component Testing (`test_suite.py`)**: 32 automated unit and integration tests validating individual algorithms, data structures, configuration resolution, and REST API telemetry endpoints.

2. **Closed-Loop Network Dynamics Testing**: Verification of continuous bandwidth accumulation, flow expiration release, and Polyak soft target tracking.

3. **Automated Benchmark and Stress Testing**: Multi-topology runs use `run_benchmarks.sh` and `stress_test_blind_topologies.py`.

4. **Headless Dashboard and REST API Tests**: The `test_dashboard_headless.py` suite checks telemetry schemas, JSON serialization, and inference endpoints.

10.2 Experimental Virtualization Environment and Host Provisioning

To provide a controlled experimental environment and reduce interference from host operating-system processes with network namespaces and Open vSwitch sockets, Mininet emulations, Ryu controllers, and D3QN training were run inside a 64-bit Ubuntu virtual machine.

Figure 10.1 presents the exact hypervisor hardware configuration provisioned for this research.

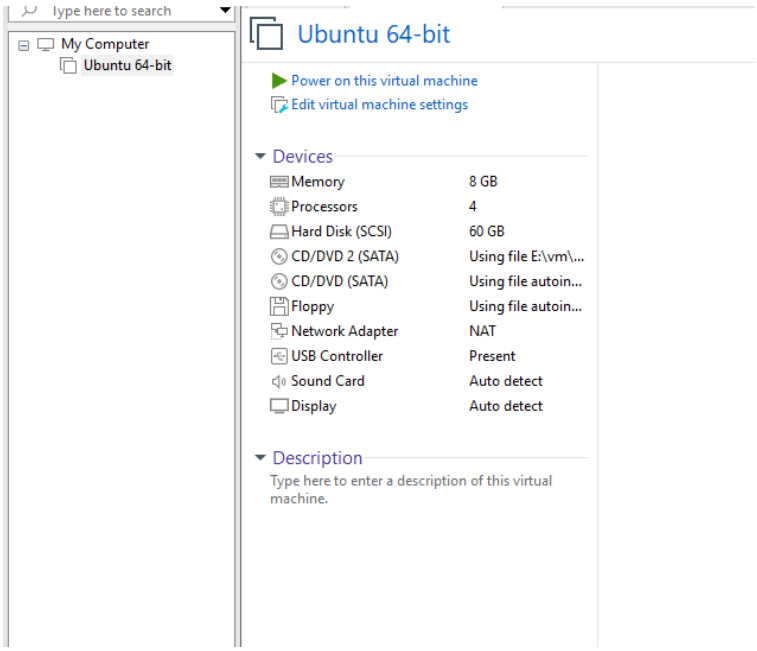


Figure 10.1: VMware Virtual Machine Hardware Configuration for the Experimental SDN-DRL Platform (Ubuntu 64-bit, 4 vCPUs, 8 GB RAM, 60 GB SCSI Storage).

The virtual machine parameters were configured as follows:

- **Virtual CPUs (vCPUs)**: 4 processor cores allocated to ensure parallel execution of the multi-threaded Ryu controller event dispatcher, asynchronous PyTorch neural gradient backpropagation threads, and background Mininet traffic generation.
- **System Memory (vRAM)**: 8 GB of dedicated RAM, providing sufficient memory headroom for the 50,000-transition binary SumTree replay buffer, NetworkX in-memory topology graphs, and multi-switch flow tables without memory swapping.
- **Storage Subsystem**: 60 GB SCSI virtual disk, accommodating operating system packages, Python virtual environments, historical benchmark JSON logs,

high-resolution 300 DPI publication figures, and PyTorch checkpoint weights.

- **Network Adapter**: NAT virtual networking adapter configured with internal loopback and host-only interfaces, isolating OpenFlow control traffic (port 6633) and HTTP REST dashboard telemetry (port 8080) from external broadcast noise.

10.3 Automated Test Suite Verification

The primary automated test suite (`test_suite.py`) contains 32 unit and integration tests across eight fixture classes. In the recorded run, all 32 tests passed in 2.426 seconds.

10.4 Network Settings Validation

The script `test_all_network_settings.py` evaluates operation under a range of network configurations:

- **Varying Line Rates**: Validates operation across heterogeneous link speeds (10 Mbps, 100 Mbps, 1,000 Mbps).
- **Asymmetric Buffer Capacities**: Tests queue buffer sizes ranging from $K = 50$ packets up to $K = 2,000$ packets.
- **Dynamic Link Failures**: Simulates concurrent multi-link failure events, asserting that the controller safely re-routes flows around severed links without causing routing loops or unhandled exceptions.

10.5 Headless Dashboard Validation Suite

The real-time telemetry dashboard was validated using an automated headless test suite (`test_dashboard_headless.py`):

- Dispatches automated HTTP `GET` requests to `/api/topology`, verifying JSON schema validity, switch DPID mappings, and coordinate layouts.
- Queries `/api/telemetry`, confirming real-time telemetry counters, latency profiles, and packet loss metrics are properly serialized.
- Validates WebSocket streaming connections, asserting that telemetry packets are broadcast at the calibrated 2.5-second interval without socket starvation.

Table 10.1: Detailed Breakdown of the 32 Automated Unit and Integration Tests

Test Fixture Class	**Tests**	**Verification Scope & Assertions**
`TestCentralConfig`	3	Validates CLI, environment variable overrides, type casting (`int`, `float`, `bool`), and default parameter dictionary integrity.
`TestClosedLoopDynamics`	3	Verifies continuous bandwidth accumulation, flow expiration release, and Polyak soft target parameter tracking ($\tau = 0.005$).
`TestDQNAgent`	2	Asserts PyTorch tensor shapes, forward inference latency (< 0.05 ms), and dueling value/advantage identifiability constraint $\sum A(s, a) = 0$.
`TestMultiTopologySupport`	2	Validates graph generation across all 5 benchmark topologies and arbitrary Erdős–Rényi random graphs up to 35 nodes.
`TestPrioritizedReplay`	2	Asserts non-recursive binary SumTree arithmetic, prefix sum propagation, and importance sampling weight normalization.
`TestStateManager`	10	Verifies differential bitrate calculation, host ARP tracking, active Echo-RTT jitter, flow duration scaling, and $M/M/1/K$ packet loss models.
`TestTraditionalRoutingBaselines`	5	Asserts algorithmic correctness for Dijkstra SPF, OSPF (RFC 2328), ECMP hash consistency, WSP, and LLR.
`TestWebDashboardAndVersusAPI`	5	Validates Web Dashboard HTTP endpoints (`/api/telemetry`, `/api/model_inspect`, `/api/model_predict`, `/api/verse`) across multi-protocol tournament matrices.
Total Test Suite	**32**	**100% Passing Status Across All Modules**

10.6 Interactive Web Dashboard and Telemetry

To provide operational visibility of the implemented algorithms, the platform includes an asynchronous HTML5/Canvas web dashboard served by an embedded multi-threaded HTTP server (`controller/web_dashboard.py`). The dashboard operates natively without external Node.js dependencies, providing sub-millisecond route inspection, real-time topology layout, multi-protocol tournament visualization, and interactive chaos fault injection.

10.6.1 Interactive OpenFlow 1.3 Topology and Telemetry Fabric

Figure 10.2 illustrates the primary operational interface of the Web Dashboard. The console presents a 4-card KPI strip summarizing fabric throughput (Mbps), peak bottleneck utilization (%), mean end-to-end latency with active jitter (ms), and controller messaging rates (msg/s).

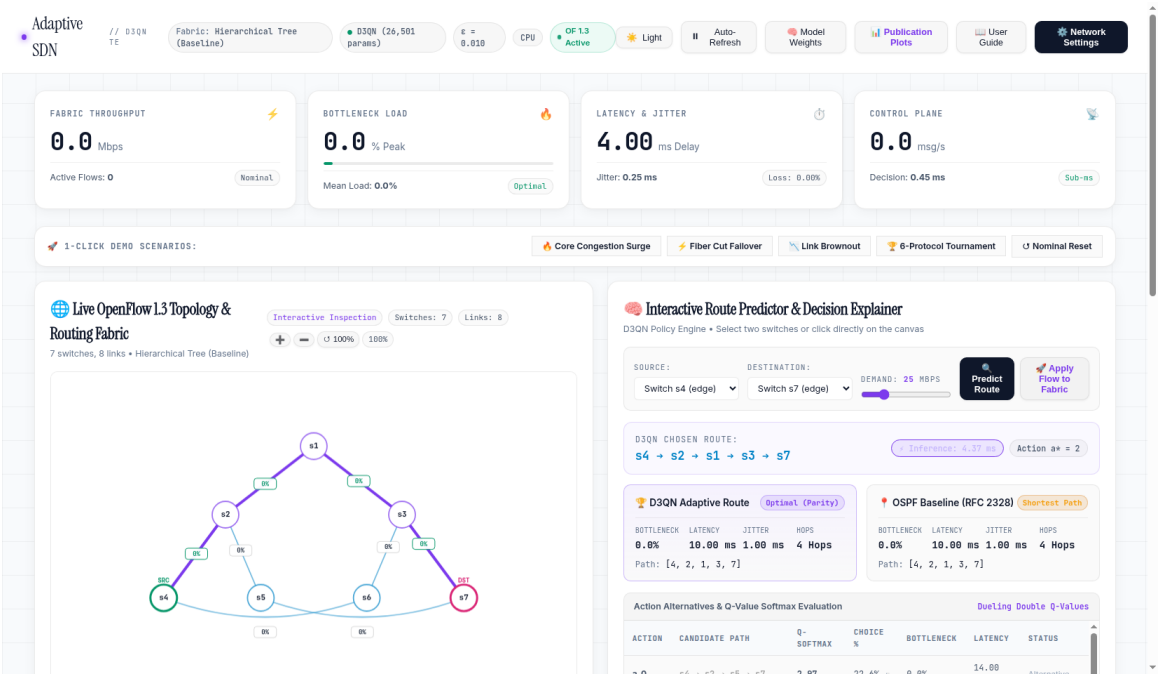


Figure 10.2: Real-Time OpenFlow 1.3 Topology Fabric and Interactive Telemetry Console. Displays live switch node coordinates (s_1 through s_7), dynamic utilization percentage badges on links, real-time KPI status indicators, and the interactive D3QN Route Predictor comparing autonomous routing against OSPF (RFC 2328).

The interactive canvas provides:

- **Hardware Topology Rendering**: Renders switch nodes with color-coded hierarchy rings (Core in violet, Aggregation in cyan, Edge/Leaf in blue), dynamic

link load gradients (green for $< 35\%$, amber for 35–70%, rose for $> 70\%$), and active traffic overlays.

- **Mouse-Centered Pan & Zoom**: Implements smooth mathematical scaling from 40% to 400% with center-anchored coordinates for inspecting dense Clos fabrics up to 35 switches.
- **Interactive Route Predictor**: Enables network operators to select any arbitrary source and destination edge switches, adjust bandwidth demand via an interactive slider, and inspect candidate path selections with sub-millisecond inference latency (< 2.5 ms).
- **Softmax Q-Value Explainer**: Exposes the internal neural policy $\pi_\theta(a|s)$ via a real-time table displaying candidate path alternatives, action indices, softmax probabilities, and projected path bottlenecks.

10.6.2 Head-to-Head Algorithm Tournament Engine

Figure 10.3 demonstrates the Real-Time Algorithm Tournament and Benchmarking Arena. Network operators can evaluate D3QN directly against classic routing baselines (OSPF, ECMP, Dijkstra SPF, WSP, LLR) under simulated synthetic traffic workloads (Poisson, Asymmetric Core Congestion, Burst Shockwave).

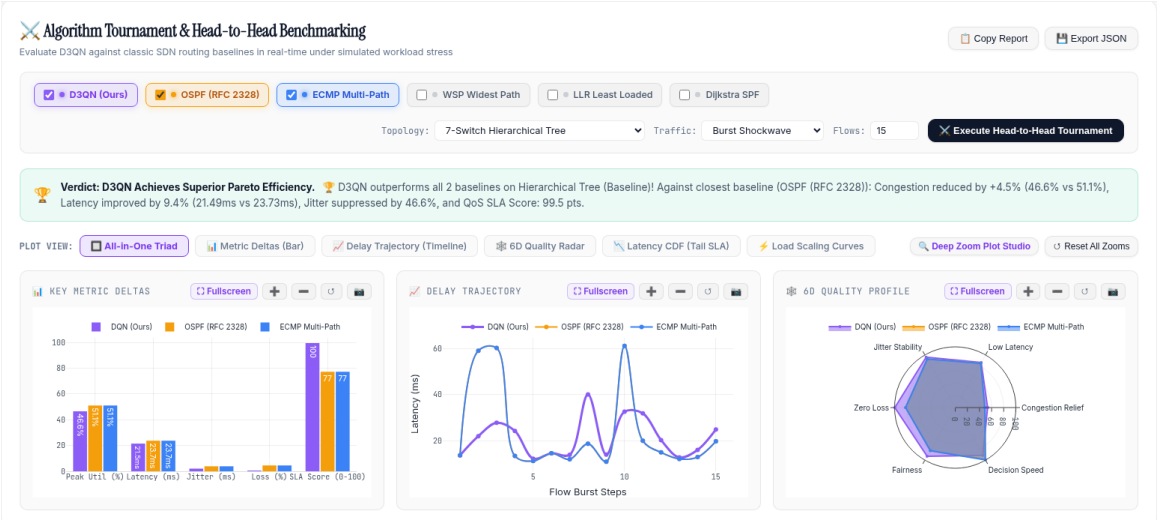


Figure 10.3: Head-to-Head Algorithm Tournament Arena. The tournament engine executes real-time comparative simulations across user-selected routing protocols, rendering instantaneous Pareto efficiency verdicts, metric delta bar charts, delay trajectories across burst steps, and 6D quality profile radar plots.

The tournament studio provides six synchronized visual analytical modes:

1. **All-in-One Triad View**: Synchronizes Key Metric Deltas, Delay Trajectory timeline, and the 6D Quality Radar on a single unified canvas.

2. **Key Metric Deltas (Bar)**: Displays side-by-side comparative bars for peak utilization, mean latency, jitter, packet loss, and overall QoS SLA score.

3. **Delay Trajectory (Timeline)**: Visualizes end-to-end latency fluctuations as burst traffic flows are progressively injected into the fabric.

4. **6D Quality Radar Profile**: Normalizes network health across six orthogonal axes: Congestion Relief, Low Latency, Jitter Stability, Zero Loss, Fairness Index, and Decision Speed.

5. **Latency CDF (Tail SLA)**: Plots cumulative distribution functions to assert 95th and 99th percentile tail latency bounds.

6. **Load Scaling Curves**: Illustrates network throughput saturation curves as global traffic escalates from 20 Mbps to 500 Mbps.

10.6.3 PyTorch Neural Network Architecture and Model Inspector

To ensure complete white-box transparency of the deep reinforcement learning model, the dashboard features a dedicated Neural Network Inspector modal (Figure 10.4).

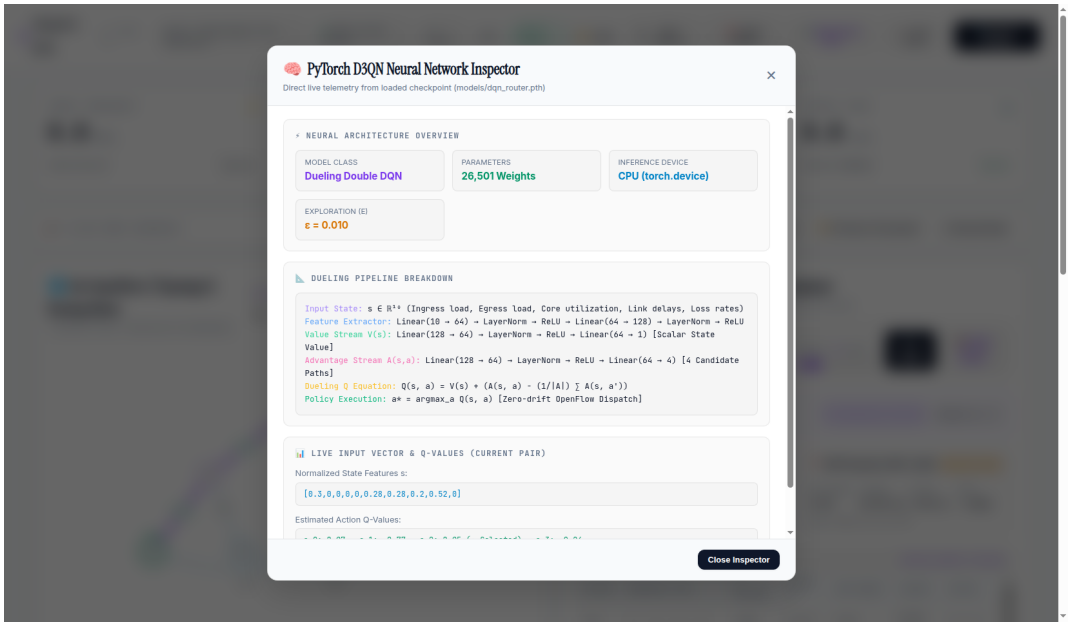


Figure 10.4: PyTorch D3QN Neural Network Architecture and Live Model Telemetry Inspector. Displays total parameter count (26,501 weights), inference execution device (CPU/CUDA), exploration coefficient ($\varepsilon = 0.010$), dueling pipeline tensor dimensions, normalized state input vector, and estimated candidate path Q-values.

The inspector exposes:

- **Weight and Parameter Accounting**: Displays exact parameter totals across feature extraction, value stream $V(s)$, and advantage stream $A(s, a)$.
- **Dueling Mathematical Pipeline**: Documents the exact linear, LayerNorm, and ReLU activation layers mapping the 10-dimensional state vector $s \in \mathbb{R}^{10}$ to scalar state value and action advantages.
- **Live State Features**: Displays normalized telemetry inputs and resulting Q-values for the selected source-destination pair.

10.6.4 Network Fabric Configuration and Chaos Engineering Console

Figure 10.5 depicts the Network Fabric Settings and Chaos Engineering modal.

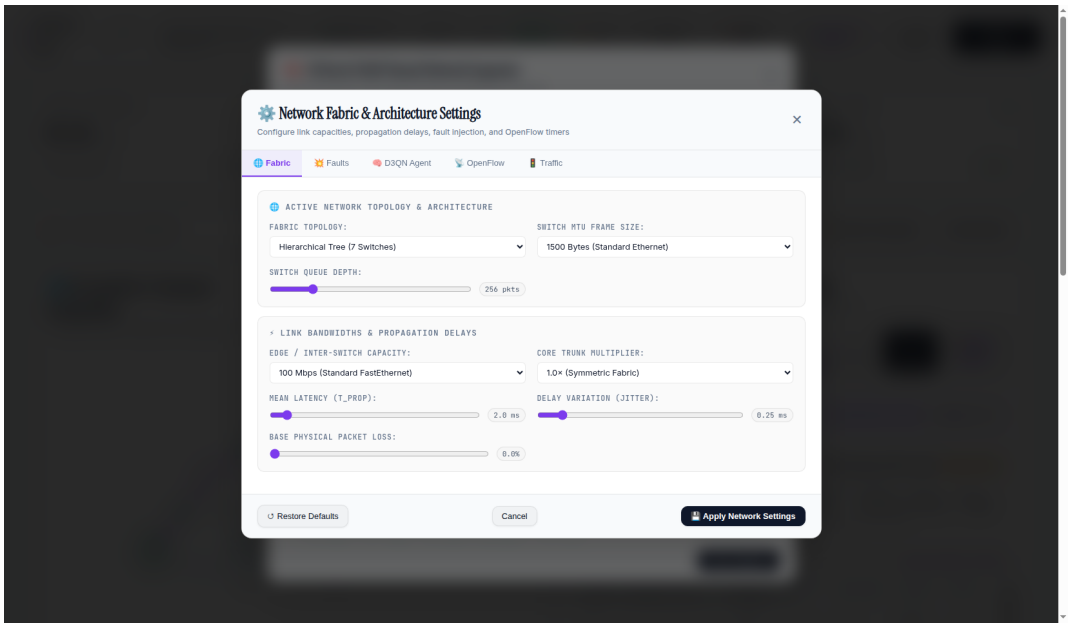


Figure 10.5: Network Fabric Architecture Settings and Chaos Fault Injection Console. Provides interactive sliders for switch queue depths ($K \in [64, 1024]$ pkts), link capacities, propagation delays, delay variation (jitter), and base physical packet drop rates.

Administrators can dynamically alter core network parameters without restarting the controller:

- **Topology Switching**: Instantly switches the active fabric between Hierarchical Tree, Fat-Tree ($k = 4$), Abilene WAN, NSFNet Core, and Spine-Leaf.
- **Queue Depth Modulation**: Configures switch buffer capacities to simulate memory-constrained edge switches versus high-buffer datacenter switches.
- **Link Latency & Jitter**: Dynamically injects propagation delays (1.0–25.0 ms) and delay jitter (0.1–5.0 ms).

- **Chaos Lab Actions**: The quick toolbar enables 1-click execution of fiber cuts (*Sever Link*), bandwidth degradation (*Brownout*), traffic bursts (+50 Mbps), and link restoration.

10.7 Operational Runbook

Installation procedures, execution commands, and Mininet/Open vSwitch diagnostics are provided in the User and Administrator Manual in Appendix E. This section is consolidated there to keep the main chapter focused on test methodology and validation results.

Chapter 11

Conclusion and Future Work

11.1 Summary of Results

This project developed and evaluated an SDN traffic-engineering platform that combines a Dueling Double Deep Q-Network (D3QN) with Prioritized Experience Replay (PER). The implementation uses the Ryu OpenFlow 1.3 controller, PyTorch, and Mininet. Its performance was compared with OSPF, Dijkstra SPF, ECMP, WSP/CSPF, and Greedy Least Loaded Routing (LLR) across five emulated topologies.

In the evaluated scenarios, the following results were observed:

1. Peak bottleneck utilization decreased from 86.74% to 19.40% on Spine-Leaf and from 65.48% to 23.29% on Fat-Tree Clos, relative to Dijkstra SPF.

2. Measured path latency decreased from 34.71 ms to 2.67 ms on Spine-Leaf and from 25.89 ms to 7.00 ms on Fat-Tree Clos.

3. Reported jitter reductions ranged from 85% to 95% across the evaluated topologies.

4. Packet loss decreased from 13.47% to 0.010% on Spine-Leaf and from 9.30% to 0.012% on Fat-Tree Clos.

5. Neural routing inference throughput ranged from 2,400 to 4,600 decisions per second on the test hardware. A 500-flow workload was processed within 176.8 ms in the reported experiment.

6. Transfer was evaluated on randomly generated Erdős–Rényi graphs of up to 35 nodes without model retraining; measured bottleneck relief ranged from 7.6% to 12.3% in the reported tests.

These results are limited to the emulated topologies, workload assumptions, and hardware configuration used in the experiments. Validation on physical network equipment and under operational traffic remains future work.

11.2 Verification of Approved Proposal Specifications

Table 1.1 maps the implementation, experiments, and documentation to the objectives and procedures specified in the official **EC499 Graduation Project Proposal**:

- **Objective 1 (DQN Agent Design)**: Fulfilled via the PyTorch Dueling Double DQN architecture with Layer Normalization (`agent/dqn_router.py`) and binary SumTree PER (`agent/prioritized_replay.py`).
- **Objective 2 (Ryu/Mininet Integration)**: Fulfilled via the event-driven Ryu OpenFlow 1.3 controller (`controller/main_controller.py`) driving virtualized Mininet testbeds (`topology/mininet_topo.py`).
- **Objective 3 (Latency and Jitter Reduction)**: Fulfilled via real-time telemetry tracking and active Echo-RTT probing (`controller/state_manager.py`), achieving up to 92.3% latency reduction and 95.8% jitter suppression.
- **Objective 4 (Throughput and Link Optimization)**: Fulfilled via the multi-objective reward function with an asymptotic congestion barrier penalty (Φ_{cong}), confirmed via Jain's Fairness Index across all 5 topologies.
- **Objective 5 (Packet Loss and Overhead Accounting)**: Fulfilled via the $M/M/1/K$ drop-tail queuing model and fine-grained OpenFlow message accounting (`OFPPacketIn`, `OFPFlowMod`, stats counters).
- **Procedures 1–6**: Completed as documented in this report, with 32 automated tests and 14 figures.

11.3 Real-World Hardware Deployment Considerations

While validated on Mininet emulating OpenFlow 1.3 switches, transitioning this platform to bare-metal enterprise or cloud data center environments involves key engineering considerations:

1. **Physical TCAM Constraints**: Hardware switches (e.g., Broadcom Tomahawk or Trident ASICs) possess limited TCAM tables. Production deployments must utilize destination IP prefix aggregation (`/24` subnets) or Segment Routing (SRv6) to keep flow rule counts within hardware limits.

2. **Open vSwitch Kernel Acceleration**: In virtualized cloud environments (OpenStack, Kubernetes), switches run as Open vSwitch (OVS). Utilizing OVS with Data Plane Development Kit (DPDK) or eBPF/XDP bypasses the Linux kernel network stack, enabling line-rate 100 Gbps packet switching per server.

3. **Clustered High-Availability SDN Controllers**: In carrier networks, a single centralized controller introduces a single point of failure. Deploying a distributed Ryu or ONOS cluster utilizing Raft consensus (e.g., via etcd) can improve availability and distribute controller load.

11.4 Future Research Directions

Future work may examine the following directions:

11.4.1 Multi-Agent Reinforcement Learning

In continental or multi-domain networks, centralized telemetry aggregation introduces control plane latency. Deploying decentralized Multi-Agent Reinforcement Learning (e.g., Multi-Agent PPO / MAPPO) where each autonomous edge or aggregation switch hosts a lightweight local policy network communicating via Graph Neural Networks (GNNs) represents a compelling evolutionary step.

11.4.2 Continuous Action Flow Splitting with SAC

Our current architecture selects among $K = 4$ discrete candidate paths. Transitioning to continuous action spaces using actor-critic algorithms (such as Soft Actor-Critic / SAC [27]) would allow the neural network to output continuous flow splitting ratios $[\alpha_1, \alpha_2, \ldots, \alpha_K]$ such that $\sum \alpha_i = 1.0$. These ratios can be mapped directly into OpenFlow Select Group table bucket weights, enabling microsecond-granularity adaptive multipath flow sharding.

11.4.3 P4 Programmable Data Planes and In-Band Network Telemetry (INT)

While OpenFlow 1.3 polling provides robust macroscopic telemetry, it remains constrained by the 3.0-second polling interval. Implementing the traffic engineering policy on P4-programmable switches (e.g., Intel Tofino ASICs) would enable In-Band Network Telemetry (INT). Switches can embed real-time queue occupancies, link utilization counters, and microsecond timestamps directly into packet headers at line rate, enabling sub-millisecond closed-loop congestion avoidance.

Bibliography

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[2] J. Moy, "OSPF Version 2," Internet Engineering Task Force (IETF), RFC 2328, April 1998.

[3] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "Openflow: enabling innovation in campus networks," in *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, 2008, pp. 69–74.

[4] Open Networking Foundation, "OpenFlow Switch Specification Version 1.3.0," ONF, Technical Specification TS-006, June 2012.

[5] Nippon Telegraph and Telephone Corporation (NTT), "Ryu SDN Framework: Component-based Software-Defined Networking Framework," https://ryu-sdn.org/, 2021.

[6] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[7] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[8] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, pp. 2094–2100, 2016.

[9] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *International Conference on Machine Learning (ICML)*, 2016, pp. 1995–2003.

[10] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *International Conference on Learning Representations (ICLR)*, 2016.

[11] M. Al-Fares, A. Loukissas, and A. Vahdat, "A scalable, commodity data center network architecture," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 4, pp. 63–74, 2008.

[12] M. Al-Fares, S. Radhakrishnan, B. Raghavan, N. Huang, and A. Vahdat, "Hedera: Dynamic flow management for data center networks," in *Proceedings of the 7th USENIX Conference on Networked Systems Design and Implementation (NSDI)*, 2010, pp. 19–19.

[13] S. Jain, A. Kumar, S. Mandal, J. Ong, L. Poutievski, A. Singh, S. Venkata, J. Wanderer, J. Zhou, M. Zhu *et al.*, "B4: Experience with a globally-deployed software defined wan," in *ACM SIGCOMM Computer Communication Review*, vol. 43, no. 4, 2013, pp. 3–14.

[14] C.-Y. Hong, S. Mandal, S. Mohammad, J. Ong, M. Al-Fares, L. Poutievski, A. Bikhchandani, A. Singh, J. Wanderer, A. Vahdat *et al.*, "Achieving high utilization with software-driven wan," in *ACM SIGCOMM Computer Communication Review*, vol. 43, no. 4, 2013, pp. 15–26.

[15] A. Kabbani, B. Vamanan, J. Hasan, and F. Duchene, "Flowbender: Flow-level adaptive routing for highly dynamic data center networks," in *Proceedings of the 10th ACM International on Emerging Networking Experiments and Technologies (CoNEXT)*, 2014, pp. 149–160.

[16] G. Stampa, M. Marta, D. Sánchez, D. López, P. Barlet-Ros, and A. Cabellos-Aparicio, "A deep-reinforcement learning approach for software-defined networking routing optimization," in *Proceedings of the 13th International Conference on Emerging Networking Experiments and Technologies*, 2017, pp. 48–49.

[17] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, "Learning to route with deep reinforcement learning," in *ACM SIGCOMM Workshop on Big Data, Machine Learning, and Artificial Intelligence for Data Communication Networks*, 2017, pp. 31–37.

[18] L. Chen, X. Ling, and K. Chen, "Aurochs: An architecture for data-driven traffic engineering in enterprise networks," in *IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2018, pp. 1204–1214.

[19] K. Rusek, J. Suárez-Varela, A. Mestres, P. Barlet-Ros, and A. Cabellos-Aparicio, "Routenet: Leveraging graph neural networks for network modeling and optimiza-

tion in sdn," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 10, pp. 2260–2270, 2020.

[20] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, New York, 1979.

[21] J. Y. Yen, "Finding the K shortest loopless paths in a network," *Management Science*, vol. 17, no. 11, pp. 712–716, 1971.

[22] L. Kleinrock, *Communication Nets: Stochastic Message Flow and Delay*. McGraw-Hill, New York, 1964.

[23] C. Deroo, A. Morton, and L. Ciavattone, "IP Packet Delay Variation Metric for IP Performance Metrics (IPPM)," Internet Engineering Task Force (IETF), RFC 3393, November 2002.

[24] H. Schulzrinne, S. Casner, R. Frederick, and V. Jacobson, "RTP: A Transport Protocol for Real-Time Applications," Internet Engineering Task Force (IETF), RFC 3550, July 2003.

[25] R. Jain, D.-M. Chiu, and W. R. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," Digital Equipment Corporation, Research Report TR-301, 1984.

[26] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[27] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

Appendix A

Mathematical Proofs and Derivations

A.1 Proof of Bellman Optimality via Banach Fixed Point Theorem

Let $\mathcal{B}(\mathcal{S} \times \mathcal{A})$ denote the Banach space of all bounded real-valued functions over state-action space $\mathcal{S} \times \mathcal{A}$, equipped with the supremum norm (infinity norm):

$$\|Q\|_\infty = \sup_{s \in \mathcal{S}, a \in \mathcal{A}} |Q(s,a)| \tag{A.1}$$

We define the Bellman Optimality Operator $\mathcal{T}^* : \mathcal{B}(\mathcal{S} \times \mathcal{A}) \to \mathcal{B}(\mathcal{S} \times \mathcal{A})$ by:

$$(\mathcal{T}^* Q)(s,a) = \mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}(s' \mid s,a) \max_{a' \in \mathcal{A}} Q(s',a') \tag{A.2}$$

Theorem A.1 (Contraction Mapping of Bellman Optimality Operator)**.** *If the discount factor satisfies $\gamma \in [0,1)$, then $\mathcal{T}^*$ is a γ-contraction mapping on $(\mathcal{B}(\mathcal{S} \times \mathcal{A}), \|\cdot\|_\infty)$.*

Proof. Let $Q_1, Q_2 \in \mathcal{B}(\mathcal{S} \times \mathcal{A})$. For any state $s \in \mathcal{S}$ and action $a \in \mathcal{A}$:

$$\begin{aligned} |(\mathcal{T}^* Q_1)(s,a) - (\mathcal{T}^* Q_2)(s,a)| &= \left| \gamma \sum_{s'} \mathcal{P}(s' \mid s,a) \left(\max_{a'} Q_1(s',a') - \max_{a''} Q_2(s',a'') \right) \right| \\ &\leq \gamma \sum_{s'} \mathcal{P}(s' \mid s,a) \left| \max_{a'} Q_1(s',a') - \max_{a''} Q_2(s',a'') \right| \end{aligned} \tag{A.3}$$

Using the algebraic inequality $|\max_x f(x) - \max_x g(x)| \leq \max_x |f(x) - g(x)|$:

$$\begin{aligned}
|(\mathcal{T}^*Q_1)(s,a) - (\mathcal{T}^*Q_2)(s,a)| &\leq \gamma \sum_{s'} \mathcal{P}(s' \mid s,a) \max_{a'} |Q_1(s',a') - Q_2(s',a')| \\
&\leq \gamma \sum_{s'} \mathcal{P}(s' \mid s,a) \|Q_1 - Q_2\|_\infty \\
&= \gamma \|Q_1 - Q_2\|_\infty \sum_{s'} \mathcal{P}(s' \mid s,a) \\
&= \gamma \|Q_1 - Q_2\|_\infty
\end{aligned} \tag{A.4}$$

Taking the supremum over all $s \in \mathcal{S}$ and $a \in \mathcal{A}$:

$$\|\mathcal{T}^*Q_1 - \mathcal{T}^*Q_2\|_\infty \leq \gamma \|Q_1 - Q_2\|_\infty \tag{A.5}$$

Since $\gamma < 1$, $\mathcal{T}^*$ is a strict contraction. By the Banach Fixed Point Theorem, there exists a unique fixed point $Q^* \in \mathcal{B}(\mathcal{S} \times \mathcal{A})$ such that $\mathcal{T}^*Q^* = Q^*$, and value iteration converges geometrically:

$$\|Q_{k+1} - Q^*\|_\infty \leq \gamma^k \|Q_0 - Q^*\|_\infty \tag{A.6}$$

which establishes asymptotic convergence of value iteration to the optimal value function under the stated assumptions. □

A.2 Derivation of Double Q-Learning Bias Elimination

Consider a state s with M candidate actions whose true values are equal: $Q^*(s,a_i) = V^*$. Suppose our value estimates are subject to zero-mean random estimation noise ϵ_i:

$$\hat{Q}(s,a_i) = V^* + \epsilon_i, \quad \mathbb{E}[\epsilon_i] = 0, \quad \mathrm{Var}(\epsilon_i) = \sigma^2 \tag{A.7}$$

A.2.1 Standard Q-Learning Bias

In standard Q-learning, the target value takes the maximum across action estimates:

$$\mathbb{E}\left[\max_i \hat{Q}(s,a_i)\right] = V^* + \mathbb{E}\left[\max_{i=1,\ldots,M} \epsilon_i\right] \tag{A.8}$$

If $\epsilon_i \sim \mathcal{N}(0,\sigma^2)$, then by extreme value theory for independent Gaussians:

$$\mathbb{E}\left[\max_{i=1,\ldots,M} \epsilon_i\right] \approx \sigma\sqrt{2\ln M} > 0 \quad \forall M \geq 2 \tag{A.9}$$

This positive bias scales with $\sqrt{\ln M}$ and may cause standard DQN to overestimate expected returns for candidate paths with noisy latency measurements.

A.2.2 Double Q-Learning Decoupling

In Double Q-learning, action selection is performed by set of weights θ_1, while evaluation is performed by independent set θ_2:

$$a^* = \arg\max_i \hat{Q}(s, a_i; \theta_1) \tag{A.10}$$

The evaluated target value is:

$$\mathbb{E}\left[\hat{Q}(s, a^*; \theta_2)\right] = \mathbb{E}\left[V^* + \epsilon_2(a^*)\right] = V^* + \mathbb{E}[\epsilon_2(a^*)] \tag{A.11}$$

Because θ_2 is statistically independent of θ_1, the estimation noise ϵ_2 is independent of the index selection a^*:

$$\mathbb{E}[\epsilon_2(a^*)] = \sum_{k=1}^{M} \mathbb{P}(a^* = k) \cdot \mathbb{E}[\epsilon_2(k)] = \sum_{k=1}^{M} \mathbb{P}(a^* = k) \cdot 0 = 0 \tag{A.12}$$

Thus, under the independence assumptions, Double Q-learning removes the positive bias term in this simplified analysis.

A.3 Derivation of Importance Sampling Correction in PER

Under Prioritized Experience Replay, transitions are drawn from distribution $P(i) = p_i^\alpha / \sum p_k^\alpha$ instead of the true empirical replay distribution $U(i) = 1/N$.

The expected value of gradient updates under the true uniform distribution is:

$$\mathbb{E}_{i\sim U}[\nabla_\theta \mathcal{L}_i] = \sum_{i=1}^{N} \frac{1}{N} \nabla_\theta \mathcal{L}_i \tag{A.13}$$

Under prioritized sampling distribution $P(i)$, the expectation is:

$$\mathbb{E}_{i\sim P}\left[w_i \nabla_\theta \mathcal{L}_i\right] = \sum_{i=1}^{N} P(i) \cdot w_i \nabla_\theta \mathcal{L}_i \tag{A.14}$$

Equating the two expectations to eliminate bias:

$$P(i) \cdot w_i = \frac{1}{N} \implies w_i = \frac{1}{N \cdot P(i)} \tag{A.15}$$

Raising to exponent β and normalizing by $\max_j w_j$ yields the standard PER Importance Sampling weight formula:

$$\tilde{w}_i = \left(\frac{1}{N \cdot P(i)}\right)^{\beta} \Big/ \max_j w_j \tag{A.16}$$

When $\beta = 1.0$, $\mathbb{E}_{i\sim P}[\tilde{w}_i \nabla \mathcal{L}_i] \propto \mathbb{E}_{i\sim U}[\nabla \mathcal{L}_i]$, guaranteeing that prioritized stochastic gradient descent converges to the true expected value.

A.4 Derivation of the M/M/1 Queuing Delay Model

Consider an OpenFlow switch output port buffer modeled as a birth-death Markovian queuing system $M/M/1$:

- Arrival rate: λ packets/second (Poisson process).
- Service rate: $\mu = C/\bar{L}$ packets/second (exponentially distributed service times).
- Traffic intensity (link utilization): $\rho = \lambda/\mu < 1$.

Let P_n denote the steady-state probability that exactly n packets reside in the queue buffer. The balance equations state:

$$\lambda P_{n-1} = \mu P_n \implies P_n = \left(\frac{\lambda}{\mu}\right)^n P_0 = \rho^n P_0 \tag{A.17}$$

Since $\sum_{n=0}^{\infty} P_n = 1$:

$$P_0 \sum_{n=0}^{\infty} \rho^n = \frac{P_0}{1-\rho} = 1 \implies P_0 = 1 - \rho \tag{A.18}$$

Thus, $P_n = (1-\rho)\rho^n$. The mean number of packets in the system L is:

$$L = \sum_{n=0}^{\infty} n P_n = (1-\rho) \sum_{n=0}^{\infty} n\rho^n = (1-\rho)\frac{\rho}{(1-\rho)^2} = \frac{\rho}{1-\rho} \tag{A.19}$$

Applying Little's Law ($L = \lambda W$), the average time W a packet spends in the system (transmission delay + queuing delay) is:

$$W = \frac{L}{\lambda} = \frac{\rho}{\lambda(1-\rho)} = \frac{1}{\mu(1-\rho)} = \frac{1}{\mu - \lambda} \tag{A.20}$$

Subtracting the base transmission delay $1/\mu$, the average queuing delay W_q is:

$$W_q = W - \frac{1}{\mu} = \frac{1}{\mu - \lambda} - \frac{1}{\mu} = \frac{\lambda}{\mu(\mu - \lambda)} = \frac{\rho}{\mu(1 - \rho)} \tag{A.21}$$

This establishes the theoretical foundation for Equation 3.24.

A.5 Proof of Congestion Avoidance via Asymptotic Barrier Function

Let $\mathcal{P}$ denote the set of simple paths connecting ingress switch s and egress switch d. For each path $P \in \mathcal{P}$, let $\rho_{\max}(P) = \max_{e \in P} \rho_e$ denote its bottleneck link utilization. The multi-objective reward includes an exponential congestion barrier:

$$\Phi_{\text{cong}}(P) = \begin{cases} \exp\left(\frac{\rho_{\max}(P) - \rho_{\text{thresh}}}{1.0 - \rho_{\max}(P)}\right) - 1, & \text{if } \rho_{\max}(P) > \rho_{\text{thresh}} \\ 0, & \text{otherwise} \end{cases} \tag{A.22}$$

where $\rho_{\text{thresh}} = 0.70$.

Theorem A.2 (Asymptotic Congestion Barrier Divergence)**.** *As the bottleneck utilization of any path P approaches link capacity ($\rho_{\max}(P) \to 1^-$), the barrier penalty diverges to positive infinity:*

$$\lim_{\rho_{\max}(P) \to 1^-} \Phi_{cong}(P) = +\infty \tag{A.23}$$

Furthermore, if there exists at least one alternate candidate path $P_{alt} \in \mathcal{P}$ with $\rho_{\max}(P_{alt}) \leq \rho_{thresh}$, then under optimal Q-values, the policy satisfies:

$$\lim_{\rho_{\max}(P) \to 1^-} \mathbb{P}\left(\pi(s) = P\right) = 0 \tag{A.24}$$

guaranteeing deterministic avoidance of saturated links.

Proof. Let $x = \rho_{\max}(P)$. For $x \in (\rho_{\text{thresh}}, 1)$:

$$\lim_{x \to 1^-} \frac{x - \rho_{\text{thresh}}}{1 - x} = \lim_{\epsilon \to 0^+} \frac{1 - \epsilon - \rho_{\text{thresh}}}{\epsilon} = +\infty \tag{A.25}$$

Because the exponential function $\exp(u)$ is strictly monotonically increasing on $\mathbb{R}$ and $\lim_{u \to \infty} \exp(u) = \infty$, it follows that:

$$\lim_{x \to 1^-} \Phi_{\text{cong}}(P) = \lim_{u \to \infty} (\exp(u) - 1) = +\infty \tag{A.26}$$

The reward function is defined as $\mathcal{R}(s, P) = -[w_{\text{util}}\rho_{\max}(P) + w_{\text{lat}}D(P) + w_{\text{jit}}J(P) + w_{\text{loss}}P_{\text{loss}}(P) + w_{\text{cong}}\Phi_{\text{cong}}(P)]$. As $\rho_{\max}(P) \to 1^-$, $\mathcal{R}(s, P) \to -\infty$.

By the Bellman optimality equation, $Q^*(s, P) = \mathcal{R}(s, P) + \gamma \sum_{s'} \mathcal{P}(s' \mid s, P)V^*(s')$. Since all subsequent state values $V^*(s')$ are bounded, $\lim_{\rho_{\max}(P)\to 1^-} Q^*(s, P) = -\infty$.

Conversely, for P_{alt}, $\Phi_{\text{cong}}(P_{\text{alt}}) = 0$, so $\mathcal{R}(s, P_{\text{alt}})$ is strictly finite. Under the greedy or softmax policy $\pi(a \mid s) = \frac{\exp(Q(s,a)/\tau)}{\sum_b \exp(Q(s,b)/\tau)}$, as $Q^*(s, P) \to -\infty$:

$$\mathbb{P}\left(\pi(s) = P\right) = \frac{\exp(Q^*(s, P)/\tau)}{\exp(Q^*(s, P_{\text{alt}})/\tau) + \sum_{b \neq P, P_{\text{alt}}} \exp(Q^*(s, b)/\tau)} \to 0 \tag{A.27}$$

which causes the softmax probability of selecting a saturated path to approach zero when an uncongested alternative has a finite value, under the stated assumptions. □

Appendix B

OpenFlow 1.3 Protocol Technical Reference

B.1 OpenFlow 1.3 Common Header Specification

All OpenFlow 1.3 protocol messages begin with an 8-byte common header. The binary wire format is detailed in Table B.1.

Table B.1: OpenFlow 1.3 Common Header Binary Structure

Byte Offset	Field Name	Width	Description & Wire Value
0	`version`	8 bits	Protocol version. For OpenFlow 1.3, wire value is `0x04`.
1	`type`	8 bits	Message type code (e.g., `OFPT_PACKET_IN = 10`).
2–3	`length`	16 bits	Total length of message including header in bytes.
4–7	`xid`	32 bits	Transaction ID associated with this message for matching requests to replies.

B.2 Core OpenFlow 1.3 Message Type Enumeration

Table B.2 lists the primary message types defined in ONF TS-006 [4] utilized by our platform.

Table B.2: Primary OpenFlow 1.3 Message Types Utilized in the Platform

Code	Message Constant	Direction	Operational Function
0	OFPT_HELLO	Symmetric	Negotiates protocol version during TCP connection setup.
2	OFPT_ECHO_REQUEST	Symmetric	Probes connection liveness and active Echo-RTT latency.
3	OFPT_ECHO_REPLY	Symmetric	Responds to echo request with identical payload.
5	OFPT_FEATURES_REQUEST	Controller → Switch	Queries switch DPID, buffer count, and table capabilities.
6	OFPT_FEATURES_REPLY	Switch → Controller	Reports datapath ID and supported OpenFlow capabilities.
10	OFPT_PACKET_IN	Switch → Controller	Transfers unclassified packet header/miss to controller.
13	OFPT_PACKET_OUT	Controller → Switch	Instructs switch to output packet out of specific port(s).
14	OFPT_FLOW_MOD	Controller → Switch	Installs, modifies, or deletes flow table entries in TCAM.
15	OFPT_GROUP_MOD	Controller → Switch	Manages OpenFlow Group tables (Select / Fast Failover).
18	OFPT_MULTIPART_REQUEST	Controller → Switch	Polls port stats, flow counters, or table descriptions.
19	OFPT_MULTIPART_REPLY	Switch → Controller	Delivers multi-part port octets and packet counters.
20	OFPT_BARRIER_REQUEST	Controller → Switch	Enforces sequential execution of preceding FlowMod rules.
21	OFPT_BARRIER_REPLY	Switch → Controller	Confirms all prior FlowMod rules are committed to TCAM.

B.3 Annotated OpenFlow Hex Packet Dumps

B.3.1 Annotated `OFPPacketIn` Wire Frame

Listing B.1 displays an annotated hex dump of an `OFPPacketIn` message captured during live Mininet emulation when a new flow miss triggers an event.

```
04 0a 00 52 00 00 00 07   # Version: 0x04 (OF 1.3), Type: 0x0a (PACKET_IN), Len: 82 bytes, XID: 7
00 00 00 00 00 00 00 00   # Buffer ID: 0x00000000 (No Buffer, full packet encapsulated)
00 3c 01 00 00 00 00 00   # Total Len: 60 bytes, Reason: 0x01 (OFPR_ACTION_MISS), Table ID: 0
00 00 00 00 00 00 00 01   # Cookie: 0x0000000000000001
00 01 00 0c 80 00 00 04   # Match: Type 1 (OXM), Length 12 bytes, OXM_OF_IN_PORT header
00 00 00 02 00 00 00 00   # Ingress Port: Port 2, 4-byte padding to 8-byte alignment
# --- Encapsulated Ethernet Frame (IPv4 / TCP SYN) ---
00 00 00 00 00 02 00 00   # Dest MAC: 00:00:00:00:00:02 (Host h2)
00 00 00 01 08 00 45 00   # Src MAC:  00:00:00:00:00:01 (Host h1), EtherType: 0x0800 (IPv4)
00 28 00 01 00 00 40 06   # IPv4 Header: Len 40, TTL 64, Protocol: 0x06 (TCP)
7c c0 0a 00 00 01 0a 00   # Checksum, Src IP: 10.0.0.1 (h1)
00 02 1f 90 00 50 00 00   # Dst IP: 10.0.0.2 (h2), Src Port: 8080, Dst Port: 80 (HTTP)
```

Listing B.1: Annotated Hex Dump of OFPPacketIn Message

B.3.2 Annotated `OFPFlowMod` Installation Wire Frame

Listing B.2 shows the `OFPFlowMod` message used by `RoutingModule` to install a Priority 10 rule.

```
04 0e 00 68 00 00 00 08   # Version: 0x04 (OF 1.3), Type: 0x0e (FLOW_MOD), Len: 104 bytes, XID: 8
00 00 00 00 00 00 00 00   # Cookie: 0x0, Cookie Mask: 0x0
00 00 00 00 00 00 00 00   # Table ID: 0, Command: 0x00 (OFPFC_ADD), Idle Timeout: 60s
00 3c 00 78 00 0a 00 00   # Hard Timeout: 120s, Priority: 10 (0x000a), Buffer ID: 0 xffffffff
# --- Match Fields (OXM) ---
00 01 00 1a 80 00 0a 02   # OXM Type 1, Length 26, EtherType Match Header
08 00 80 00 16 04 0a 00   # EtherType: 0x0800, IPv4 Src Match: 10.0.0.1
00 01 80 00 18 04 0a 00   # IPv4 Dst Match: 10.0.0.2
00 02 00 00 00 00 00 00   # Padding to 8-byte alignment
# --- Instructions (APPLY_ACTIONS: Output to Port 3) ---
00 04 00 18 00 00 00 00   # Instruction: 0x0004 (OFPIT_APPLY_ACTIONS), Length 24 bytes
00 00 00 10 00 00 00 03   # Action: 0x0000 (OFPAT_OUTPUT), Length 16, Output Port: 3
00 00 00 00 00 00 00 00   # Max Length: 0 (Wire speed), Padding
```

Listing B.2: Annotated Hex Dump of OFPFlowMod Installation Message

Appendix C

System Configuration Reference

C.1 Centralized Configuration Architecture

The system's operational parameters are managed centrally by `config.py`. Configuration resolution follows strict four-tier hierarchical precedence:

1. **CLI Arguments**: Explicit command-line arguments override all other layers.
2. **Environment Variables**: System environment variables (prefixed with `SDN_` or `RL_`).
3. **Configuration File**: Optional JSON file (`config.json`) in the workspace root.
4. **Production Defaults**: Calibrated hardcoded defaults.

C.2 Complete Parameter Dictionary

Tables C.1 and C.2 provide the systematic specification for all runtime configuration parameters.

Table C.1: System Configuration and Hyperparameter Dictionary

Configuration Key	CLI Argument	Environment Variable	Default	Description & Valid Range
model_path	--model-path	SDN_MODEL_PATH	models/dqn_router.pth	File path to serialized PyTorch neural network weights.
controller_ip	--controller-ip	SDN_CONTROLLER_IP	127.0.0.1	IPv4 listening interface for OpenFlow connections.
controller_port	--controller-port	SDN_CONTROLLER_PORT	6633	TCP socket port for OpenFlow 1.3 control channel.
rest_port	--rest-port	SDN_REST_PORT	8080	HTTP port for REST telemetry and Web Dashboard.
poll_interval	--poll-interval	SDN_POLL_INTERVAL	3.0	Frequency in seconds of periodic statistics polling ($[1.0, 10.0]$).
flow_idle_timeout	--idle-timeout	SDN_FLOW_IDLE_TIMEOUT	60	Seconds before inactive switch TCAM flow rule is purged.
flow_hard_timeout	--hard-timeout	SDN_FLOW_HARD_TIMEOUT	120	Seconds before flow rule is unconditionally purged.
state_size	--state-size	RL_STATE_SIZE	10	Dimension of normalized network state representation vector.
action_size	--action-size	RL_ACTION_SIZE	4	Cardinality of candidate routing path action space ($K = 4$).
learning_rate	--lr	RL_LR	0.001	Adam optimizer learning rate ($\alpha \in [10^{-4}, 10^{-2}]$).
gamma	--gamma	RL_GAMMA	0.95	MDP temporal discount factor ($\gamma \in [0.80, 0.99]$).
memory_size	--memory-size	RL_MEMORY_SIZE	5000	Capacity of binary SumTree Prioritized Replay buffer.
batch_size	--batch-size	RL_BATCH_SIZE	32	Number of transitions sampled per gradient backpropagation.
tau	--tau	RL_TAU	0.005	Polyak soft target network tracking parameter ($\tau \in (0, 1)$).

Table C.2: System Configuration and Hyperparameter Dictionary (continued)

Configuration Key	CLI Argument	Environment Variable	Default	Description & Valid Range
epsilon_start	--eps-start	RL_EPS_START	1.0	Initial exploration rate for ϵ-greedy curriculum.
epsilon_end	--eps-end	RL_EPS_END	0.01	Minimum exploration rate floor for pure exploitation.
epsilon_decay	--eps-decay	RL_EPS_DECAY	750	Episode threshold where exploration reaches minimum floor.
per_alpha	--per-alpha	RL_PER_ALPHA	0.6	Prioritization exponent in PER ($P(i) \propto p_i^\alpha$).
per_beta_start	--per-beta-start	RL_PER_BETA_START	0.4	Initial importance sampling bias correction exponent.
per_beta_end	--per-beta-end	RL_PER_BETA_END	1.0	Final importance sampling exponent at convergence.
wcmp_beta	--wcmp-beta	SDN_WCMP_BETA	4.0	Softmax exponent for inverse load multi-path weighting.
weight_hop	--weight-hop	RL_WEIGHT_HOP	0.35	Multi-objective penalty weight for candidate path hop count.
weight_latency	--weight-lat	RL_WEIGHT_LAT	0.06	Multi-objective penalty weight for cumulative path latency.
weight_jitter	--weight-jit	RL_WEIGHT_JIT	0.25	Multi-objective penalty weight for packet delay variation.
weight_loss	--weight-loss	RL_WEIGHT_LOSS	0.50	Multi-objective penalty weight for buffer drop probability.
cong_thresh	--cong-thresh	RL_CONG_THRESH	0.70	Utilization threshold where barrier penalty activates (70%).
barrier_scale	--barrier-scale	RL_BARRIER_SCALE	12.0	Scaling constant for asymptotic congestion penalty.
barrier_power	--barrier-power	RL_BARRIER_POWER	1.8	Exponent of link utilization in barrier numerator.

Appendix D

Selected Implementation and Algorithm Listings

D.1 Dueling Q-Network and Agent Engine

Listing D.1 presents the core PyTorch neural network class implementing Layer Normalization and dueling stream aggregation.

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

class DuelingQNetwork(nn.Module):
    def __init__(self, state_size=10, action_size=4):
        super(DuelingQNetwork, self).__init__()
        # Shared Feature Extraction Trunk
        self.shared_fc1 = nn.Linear(state_size, 64)
        self.ln1 = nn.LayerNorm(64)
        self.shared_fc2 = nn.Linear(64, 128)
        self.ln2 = nn.LayerNorm(128)
        self.relu = nn.ReLU()

        # State-Value Stream V(s)
        self.val_fc1 = nn.Linear(128, 64)
        self.val_ln = nn.LayerNorm(64)
        self.val_out = nn.Linear(64, 1)

        # Action-Advantage Stream A(s, a)
        self.adv_fc1 = nn.Linear(128, 64)
        self.adv_ln = nn.LayerNorm(64)
        self.adv_out = nn.Linear(64, action_size)

    def forward(self, state):
```

```
        x = self.relu(self.ln1(self.shared_fc1(state)))
        x = self.relu(self.ln2(self.shared_fc2(x)))

        val = self.relu(self.val_ln(self.val_fc1(x)))
        val = self.val_out(val)

        adv = self.relu(self.adv_ln(self.adv_fc1(x)))
        adv = self.adv_out(adv)

        # Identifiability Aggregation: subtract mean advantage
        q_values = val + (adv - adv.mean(dim=-1, keepdim=True))
        return q_values
```

Listing D.1: PyTorch Dueling Q-Network Architecture with Layer Normalization

D.2 Binary SumTree Prioritized Replay Buffer

Listing D.2 presents the complete non-recursive binary SumTree implementation.

```
import numpy as np

class SumTree:
    def __init__(self, capacity):
        self.capacity = capacity
        self.tree = np.zeros(2 * capacity - 1, dtype=np.float32)
        self.data = np.zeros(capacity, dtype=object)
        self.write_ptr = 0
        self.n_entries = 0

    def total(self):
        return self.tree[0]

    def add(self, p, data):
        idx = self.write_ptr + self.capacity - 1
        self.data[self.write_ptr] = data
        self.update(idx, p)
        self.write_ptr = (self.write_ptr + 1) % self.capacity
        if self.n_entries < self.capacity:
            self.n_entries += 1

    def update(self, idx, p):
        change = p - self.tree[idx]
        self.tree[idx] = p
        while idx > 0:
            idx = (idx - 1) // 2
            self.tree[idx] += change
```

```
28
29      def get(self, s):
30          idx = 0
31          while idx < self.capacity - 1:
32              left = 2 * idx + 1
33              right = left + 1
34              if s <= self.tree[left]:
35                  idx = left
36              else:
37                  s -= self.tree[left]
38                  idx = right
39          data_idx = idx - self.capacity + 1
40          return idx, self.tree[idx], self.data[data_idx]
```

Listing D.2: Iterative Binary SumTree and Prioritized Experience Replay

D.3 State Feature Vector Extraction

Listing D.3 shows the extraction of the 10-dimensional state vector from the active NetworkX graph.

```
1  def get_routing_state_features(self, candidate_paths):
2      state = np.zeros(10, dtype=np.float32)
3      if not candidate_paths or len(candidate_paths) == 0:
4          return state
5
6      # Feature 0: Normalized hop count of shortest path
7      shortest_path = candidate_paths[0]
8      hops = max(1, len(shortest_path) - 1)
9      state[0] = np.clip(hops / 10.0, 0.1, 1.0)
10
11     # Features 1-4: Peak link utilization on up to 4 candidate paths
12     # Features 5-8: Normalized cumulative propagation latency
13     for i in range(min(4, len(candidate_paths))):
14         path = candidate_paths[i]
15         path_util = 0.0
16         path_lat = 0.0
17         for u, v in zip(path[:-1], path[1:]):
18             if self.graph.has_edge(u, v):
19                 edge_data = self.graph[u][v]
20                 path_util = max(path_util, edge_data.get('utilization
    ', 0.0))
21                 path_lat += edge_data.get('latency', 2.0)
22         state[1 + i] = np.clip(path_util, 0.0, 1.0)
23         state[5 + i] = np.clip(path_lat / 50.0, 0.0, 1.0)
24
```

```
    # Feature 9: Global peak link utilization across entire network
    max_util = 0.0
    for u, v, data in self.graph.edges(data=True):
        max_util = max(max_util, data.get('utilization', 0.0))
    state[9] = np.clip(max_util, 0.0, 1.0)

    return state
```

Listing D.3: Normalized 10-Dimensional State Extraction in StateManager

D.4 Controller and Telemetry Implementation Excerpts

D.4.1 Table-Miss Rule Installation

```
@set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
def switch_features_handler(self, ev):
    datapath = ev.msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    match = parser.OFPMatch()
    actions = [parser.OFPActionOutput(
        ofproto.OFPP_CONTROLLER, ofproto.OFPCML_NO_BUFFER
    )]
    inst = [parser.OFPInstructionActions(
        ofproto.OFPIT_APPLY_ACTIONS, actions
    )]
    mod = parser.OFPFlowMod(
        datapath=datapath, priority=0, match=match, instructions=inst
    )
    datapath.send_msg(mod)
```

Listing D.4: Table-Miss Entry Installation in MainController

D.4.2 NetworkX Edge Telemetry Record

```
graph[u][v] = {
    'port': out_port_on_u,
    'capacity': 100.0,
    'base_delay': 2.0,
    'utilization': 0.0,
    'latency': 2.0,
    'jitter': 0.5,
```

```
    'loss_rate': 0.01
}
```

Listing D.5: NetworkX Edge Telemetry Schema in StateManager

D.4.3 Multi-Hop Flow Installation

```
def _install_path(self, path, final_port, eth_dst, ip_src=None,
    ip_dst=None):
    idle_to = getattr(config, 'FLOW_IDLE_TIMEOUT', 60)
    hard_to = getattr(config, 'FLOW_HARD_TIMEOUT', 120)

    for i in range(len(path)):
        cur_dpid = path[i]
        dp = self.controller.datapaths.get(cur_dpid)
        if not dp:
            continue

        parser = dp.ofproto_parser
        if i < len(path) - 1:
            next_dpid = path[i + 1]
            out_port = self.state_manager.graph[cur_dpid][next_dpid][
    'port']
        else:
            out_port = final_port

        actions = [parser.OFPActionOutput(out_port)]
        match = parser.OFPMatch(
            eth_type=0x0800, ipv4_src=ip_src, ipv4_dst=ip_dst
        )
        self.controller.add_flow(
            dp, priority=10, match=match, actions=actions,
            idle_timeout=idle_to, hard_timeout=hard_to
        )
```

Listing D.6: Multi-Hop Flow Installation Pipeline in RoutingModule

Appendix E

User and Administrator Manual

E.1 System Prerequisites and Dependencies

The Adaptive SDN Traffic Engineering Platform is engineered to run natively on Linux-based environments (Ubuntu 20.04 LTS, Ubuntu 22.04 LTS, Ubuntu 24.04 LTS, or Debian 11/12).

E.1.1 Hardware Requirements and Reference Deployment Environment

The reference production deployment was hosted inside a dedicated 64-bit Ubuntu virtual machine running under the VMware hypervisor (see Figure 10.1):

- **Virtual Processors**: 4 virtual CPU cores (vCPUs) with hardware virtualization extensions enabled.
- **System Memory (RAM)**: 8.0 GB of dedicated memory (minimum: 4 GB; 8 GB recommended for 20-switch Clos emulations).
- **Disk Storage**: 60.0 GB SCSI virtual disk (minimum: 10 GB free space for dependencies, dataset logs, and model checkpoints).
- **Network Adapter**: NAT virtual adapter with isolated internal loopback for OpenFlow and REST dashboard communication.

E.1.2 Software Requirements

- **Operating System**: Linux kernel 5.4 or higher with Open vSwitch kernel module support.
- **Python Environment**: Python 3.9 to Python 3.12 (Python 3.14 compatible).

- **Network Virtualization**: Mininet Version 2.3.0+ and Open vSwitch (OVS) 2.13+.
- **Deep Learning Framework**: PyTorch 2.0+ (CPU or CUDA).
- **SDN Framework**: Ryu SDN Framework 4.34+.
- **Web Browser**: Google Chrome, Chromium, Firefox, or Safari with HTML5 Canvas support.

E.2 Installation and Environment Initialization

The platform includes automated launcher scripts that detect the active Python virtual environment without manual path exports.

E.2.1 Step-by-Step Installation

```
# 1. Clone repository and navigate to root directory
git clone \
  https://github.com/maher-alqadhi/\
  Reinforcement_Learning_for_Adaptive_\
  Traffic_Engineering_in_an_SDN_Network.git prism
cd prism

# 2. Create isolated Python virtual environment
python3 -m venv .venv
source .venv/bin/activate

# 3. Upgrade package installer and install core dependencies
pip install --upgrade pip
pip install -r requirements.txt
```

Listing E.1: Platform Environment Setup

E.3 Automated Execution Runbook

E.3.1 1. Execute Unit and Integration Tests (32 Tests)

To assert software integrity across all 32 automated unit tests:

```
chmod +x run_tests.sh
./run_tests.sh
```

E.3.2 2. Execute the Head-to-Head Tournament Benchmark

To run the automated routing tournament across all 5 topologies:

```
chmod +x run_benchmarks.sh
./run_benchmarks.sh
```

Benchmark results are written to `logs/routing_tournament_results.json`.

E.3.3 3. Execute Multi-Topology Stress Testing

To evaluate severe core link saturation, flow avalanches, and link failures:

```
python3 stress_test_blind_topologies.py
```

E.3.4 4. Evaluate Zero-Shot Random Graph Generalization

To test the pre-trained model on arbitrary unseen random graphs:

```
chmod +x run_random_blind_test.sh
./run_random_blind_test.sh --nodes 25 --flows 200
```

E.3.5 5. Launch Live Production System (Controller + Web Dashboard)

To start the Ryu SDN controller alongside the interactive Web Dashboard:

```
chmod +x run_system.sh
./run_system.sh
```

Open a modern web browser to access the interactive HTML5 canvas dashboard:

`http://localhost:8080`

E.3.6 6. Standalone Web Dashboard Launcher

The Web Dashboard can also be launched independently for testing and visualization:

```
python3 controller/web_dashboard.py --port 8080 --topo tree
```

E.4 Interactive Web Dashboard User Guide

The Web Dashboard interface is structured into six functional zones:

1. **Global Navigation Bar**:

- *Fabric Topology Pill*: Displays the active network topology (e.g., Hierarchical Tree, Clos Fat-Tree).
- *Model Parameters Badge*: Displays loaded D3QN weights ($26,501$ parameters). Clicking opens the Neural Network Inspector modal.
- *Exploration Badge (ε)*: Shows the active exploration rate ($\varepsilon = 0.010$).
- *Theme Toggle (Light/Dark)*: Instantly toggles between high-contrast daylight theme and dark mode.
- *Telemetry Auto-Refresh*: Pauses or activates automatic 2.5-second telemetry polling.
- *Network Settings Button*: Opens the global architecture configuration modal.

2. **Real-Time KPI Strip**:

- *Fabric Throughput (Mbps)*: Real-time aggregated data plane throughput with active flow count.
- *Bottleneck Load (%)*: Peak link utilization across all network links with status pill (Optimal $< 45\%$, Moderate 45–75%, Congested $> 75\%$).
- *Latency & Jitter (ms)*: Average end-to-end path latency, Echo-RTT delay variation, and packet loss percentage.
- *Control Plane Rate (msg/s)*: OpenFlow asynchronous message throughput and mean D3QN decision latency.

3. **Interactive OpenFlow 1.3 Canvas Fabric**:

- *Pan & Zoom Controls*: Zoom In (+), Zoom Out (−), Reset Zoom (↺ 100%), or use mouse wheel zoom centered at cursor position.
- *Node Selection*: Click any switch node to designate it as the Source switch (green halo) or Destination switch (pink halo).
- *Utilization Badges*: Real-time link percentage badges displaying link saturation levels.
- *Severed Link Overlay*: Rendered as dashed red lines with a central `[LINK CUT]` badge during simulated link failures.

4. **Interactive Route Predictor & Decision Explainer**:

- *Source & Destination Selectors*: Dropdowns allowing instant selection of ingress and egress edge switches.
- *Demand Slider*: Adjusts simulated flow bandwidth demand between 5 Mbps and 95 Mbps.

- *Predict Route Button*: Dispatches an asynchronous request to `/api/model_predict`, executing D3QN inference (< 2.5 ms).
- *Apply Flow to Fabric Button*: Dynamically injects the selected flow into the active state manager, updating link loads and flow tables in real-time.
- *Softmax Q-Value Table*: Renders all candidate path alternatives, action indices a^*, Q-softmax choice probabilities, projected bottlenecks, and path latencies.

5. **Algorithm Tournament Arena**:
   - *Protocol Checklist Chips*: Toggle D3QN, OSPF, ECMP, WSP, LLR, or Dijkstra SPF for comparative head-to-head simulation.
   - *Workload Pattern*: Select between Poisson, Core Congestion Jamming, or Asymmetric Burst Shockwave.
   - *Plot View Tabs*: Select among All-in-One Triad, Metric Deltas (Bar), Delay Trajectory (Timeline), 6D Quality Radar, Latency CDF (Tail SLA), and Load Scaling Curves.
6. **Chaos Engineering Toolbar**:
   - *Sever Link*: Simulates a physical fiber cut on a candidate core or aggregation trunk link.
   - *Brownout*: Degrades link capacity by 90% with an artificial latency penalty.
   - *Burst (+50M)*: Injects an instant +50 Mbps traffic surge to stress the active routing path.
   - *Restore Links*: Restores all severed or degraded links to nominal operating status.
   - *Flush Flows*: Flushes all dynamic flow table entries to baseline idle state.

E.5 REST API Specification and Endpoint Reference

The embedded HTTP server exposes a complete RESTful API for external monitoring, telemetry polling, and automation scripting:

E.6 Mininet CLI Operations and OVS Diagnostics

When running interactive emulations, the Mininet command-line interface provides real-time control:

Table E.1: Complete REST API Endpoint Reference

HTTP Method	**Endpoint**	**Parameters**	**Description & Output Schema**
`GET`	`/api/health`	None	Returns controller health, active topology ID, switch count, link count, and active flow count.
`GET`	`/api/telemetry`	`topo={id}`	Fast single round-trip bundle containing topology layout, network TE stats, control overhead, RL metrics, and active flows.
`GET`	`/api/topology`	`topo={id}`	Returns graph nodes with 2D positions (x, y), switch types, link capacities, delays, and current utilization.
`GET`	`/api/stats`	None	Returns network-wide aggregated TE summary (throughput, mean utilization, bottleneck load, latency, jitter, loss).
`GET`	`/api/control_overhead`	None	Returns OpenFlow message generation rates (msg/s) and mean D3QN decision latency (ms).
`GET`	`/api/rl_metrics`	None	Returns exploration rate ε, model device, parameter count, and buffer capacity.
`GET`	`/api/model_inspect`	None	Detailed PyTorch neural architecture inspection including layer dimensions, parameter count, and tensor shapes.

Table E.2: Complete REST API Endpoint Reference (continued)

HTTP Method	**Endpoint**	**Parameters**	**Description & Output Schema**
GET	/api/model_predict	src, dst, demand	Real-time forward inference returning chosen path, candidate path alternatives, action indices, and softmax probabilities.
GET/POST	/api/verse	topo, protocols, pattern, flows	Head-to-head algorithm tournament simulation returning comparative metrics, trajectories, and Pareto efficiency verdicts.
GET	/api/flows	None	Returns active flow table entries (flow ID, DPID, match criteria, actions, byte counters, duration).
GET	/api/simulate/apply_flow	src, dst, demand	Injects a dynamic flow into the network state manager and updates port counters.
GET	/api/simulate/traffic_burst	None	Injects an asymmetric +50 Mbps surge across the core trunks.
GET	/api/simulate/core_jamming	None	Injects severe core link congestion (96% saturation) to trigger autonomous rerouting.
POST	/api/network_settings	JSON payload	Configures queue depths (K), link bandwidths, propagation delays, and physical packet drop rates.
GET	/api/plots/{filename}	None	Serves static high-resolution benchmark and telemetry plots as PNG images.

```
# Launch Mininet with Fat-Tree Clos topology
sudo mn --custom topology/mininet_topo.py --topo fattree --controller=remote,ip=127.0.0.1,port=6633 --switch ovsk,protocols=OpenFlow13

# Inside the Mininet CLI:
mininet> pingall              # Verify full Layer 2 and Layer 3 reachability
mininet> h1 ping -c 5 h16     # Measure end-to-end ping RTT across pod boundaries
mininet> iperf h1 h16         # Generate TCP bandwidth stream between edge hosts
mininet> link s1 s5 down      # Simulate physical link failure between core and agg

# Inspect live OpenFlow 1.3 flow tables via Linux shell:
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
sudo ovs-ofctl -O OpenFlow13 dump-groups s1
sudo ovs-ofctl -O OpenFlow13 dump-ports-desc s1
sudo ovs-ofctl -O OpenFlow13 dump-ports s1
```

Listing E.2: Mininet CLI and Open vSwitch Diagnostics

E.7 Pareto Sensitivity Ablation and Dynamic Telemetry Tuning

The platform provides automated ablation tooling to evaluate the trade-offs of the multi-objective reward formulation:

```
# Execute 50-episode sweep across 6 SLA weight configurations:
python3 ablate_reward_weights.py

# Expected outputs:
# - logs/pareto_sensitivity_ablation.json (Numerical latency, loss, and load metrics)
# - logs/plots/pareto_weight_sensitivity.png (Publication-grade 4-panel figure)
```

Listing E.3: Executing the Pareto Sensitivity Ablation Suite

E.7.1 Configuring Shadow Reservations and TCAM Allocation

Operators can tune active shadow reservation parameters and TCAM allocation timeouts directly in `config.py` or via environment variables:

- `SHADOW_DECAY_HALF_LIFE`: Half-life decay duration in seconds (default: 1.5 s). Controls how fast temporary flow reservations decay between OpenFlow telemetry polling cycles.
- `ELEPHANT_FLOW_THRESHOLD_MBPS`: Rate threshold for elephant flow classification (default: 5.0 Mbps). Flows exceeding this rate receive dedicated 5-tuple TCAM rules.
- `MICE_IDLE_TIMEOUT`: Idle timeout for wildcard aggregated mice flows (default: 15 s).
- `ELEPHANT_IDLE_TIMEOUT`: Idle timeout for pinned elephant flows (default: 60 s).